UMBRA



MANUAL FUNCIONAL Y TÉCNICO

La aplicación, explicada.

Todas sus funciones, el papel de cada componente y el código fuente completo para consultar y estudiar.

Edición del 09/11/2026 · React y Vite · API Spring Boot y datos locales
62 archivos de texto reproducidos · 5943 líneas de código y documentación
Incluye accesibilidad por perfil, autenticación HTTP y personajes públicos.

MAPA DEL DOCUMENTO

Contenido

Los títulos y enlaces permiten saltar entre explicación y código. El PDF conserva texto seleccionable; el ZIP contiene los archivos originales para editarlos.

1. Qué incluye este documento

2. Todas las funciones de la app

3. Pantallas y rutas

4. Cómo se conecta el código

5. Modelo de datos y almacenamiento

6. Recorridos explicados paso a paso

7. Accesibilidad por perfil

8. Ejecución y pruebas

9. Configuración e integración actual con Spring Boot

10. Índice de archivos y código

> Las explicaciones se basan en los archivos presentes al generar este documento. Cada listado tiene su ruta y números de línea; los cambios posteriores de la app requieren regenerar el manual.

UMBRA · EDICIÓN 09/11/2026

Qué incluye este documento

UMBRA permite crear, organizar y conservar personajes de fantasía. La edición del 09/11/2026 combina autenticación y consulta de personajes públicos mediante Spring Boot con una biblioteca personal y ajustes guardados en el navegador.

Se documentan las funciones, rutas, arquitectura, configuración y responsabilidades de los archivos actuales. Los listados íntegros reproducen sus bytes de texto con números de línea; los originales editables, imágenes, fuentes y licencias están en el ZIP.

Estado de la integración

Registro, login, comprobación inicial y logout ya usan servicios HTTP. El CRUD de la biblioteca personal, sus favoritos, copias JSON y preferencias siguen siendo locales. No existe sincronización de esas fichas entre dispositivos.

Recordarme todavía no se envía al servidor y Recuperar acceso sigue enlazado al módulo de cuentas locales antiguas. La identidad contiene rol, pero el frontend no ofrece controles de permisos por rol. Algunas etiquetas y textos de ayuda todavía describen el flujo local anterior.

Este repositorio no contiene el backend Spring Boot, su ORM ni la configuración de MariaDB. Se documenta el contrato que consume el frontend; no se ha verificado el servicio en ejecución.

El PDF permite leer y buscar; el HTML incorpora fuentes y portada. El ZIP incluye codigo/ con la instantánea editable, excluyendo node_modules, dist, .git y datos de navegadores. package-lock.json se conserva sin imprimirse en los listados.

UMBRA · EDICIÓN 09/11/2026

Funciones de la app

Función	Comportamiento
Inicio	Portada, enlace a crear y dos personajes recientes; estado vacío y acceso a la cuenta. Archivo: src/pages/Home.jsx
Biblioteca	Busca por nombre, clase, raza u origen; ignora tildes y mayúsculas; filtra clase y favoritos; ordena por fecha, nombre o nivel. Archivo: src/pages/Library.jsx
Consultar ficha	Ficha local en /biblioteca/:id con acciones y confirmación en la página. Archivo: src/pages/CharacterDetail.jsx
Crear y editar	Valida los campos, personaliza la ficha y guarda en la biblioteca del perfil activo. Archivo: src/pages/CreateCharacter.jsx
Retratos	Dos recursos locales, iniciales o archivo PNG/JPEG/WebP de hasta 750 KB, con vista previa. Archivo: src/components/charactercard/Portrait.jsx
Aleatorio	Generador completo compartido por el editor e Inspiración; respeta campos fijados. Archivo: src/utils/generateCharacter.js
Favoritos	Marca y desmarca fichas; el filtro de Biblioteca muestra solo las marcadas. Archivo: src/hooks/useCharacters.js
Duplicado	Crea otro id, añade (copia), desmarca favorito y vuelve a Biblioteca. Archivo: src/pages/CharacterDetail.jsx
Eliminación	Solicita confirmación dentro de la ficha local y guarda la nueva biblioteca. Archivo: src/pages/CharacterDetail.jsx
Inspiración	Ficha editable, bloqueo automático al editar, historial, retratos y guardado directo local. Archivo: src/pages/Inspiration.jsx
Registro	Crea la cuenta mediante POST /api/auth/register; muestra enlace a Login y no emite código local. Archivo: src/pages/Auth.jsx
Acceso	POST /api/auth/login; Recordarme permanece visible pero no se envía al backend. Archivo: src/pages/Login.jsx
Recuperación	Solo recupera cuentas locales antiguas; no cambia contraseñas del backend. Archivo: src/utils/localAccounts.js

Función	Comportamiento
Perfil	Identidad o Invitado, recuento de fichas y favoritos, cierre de sesión y accesos a ajustes. Archivo: src/pages/Profile.jsx
Copias JSON	Descarga biblioteca y retratos; valida una copia y la restaura solo tras confirmación. Archivo: src/utils/characterStore.js
Texto	Pequeño, normal, grande y extra grande mediante radios y deslizador. Archivo: src/pages/Accessibility.jsx
Contraste	Normal, alto o invertido; tiene prioridad sobre los colores de tema y acento. Archivo: src/pages/Accessibility.jsx
Lectura	Anuncios de cambio de pantalla, lectura voluntaria de fichas y velocidades 0.75, 1 y 1.25. Archivo: src/components/feedback/ReadAloud.jsx
Teclado	Tab, Mayús+Tab, opciones nativas, Escape, foco reforzado y flechas/Inicio/Fin en barra inferior. Archivo: src/components/bottomnavigation/BottomNavigation.jsx
Apariencia	Oscuro, claro o sepia; controles normales, grandes o extra grandes; seis acentos; movimiento reducido. Archivo: src/pages/Accessibility.jsx
Perfiles independientes	Cada cuenta e Invitado conservan biblioteca y accesibilidad diferentes. Archivo: src/utils/preferenceStore.js
Menús y diálogos	Cabecera desplegable, barra inferior activa y diálogos que devuelven el foco. Archivo: src/components/modal/Modal.jsx
Ayuda y privacidad	Pantallas informativas con textos locales anteriores pendientes de adaptar. Archivo: src/pages/Information.jsx
Conexión y 404	Avisa de eventos de red y presenta una pantalla para rutas inexistentes. Archivo: src/components/feedback/ConnectionStatus.jsx
Personajes públicos	Lista remota independiente en Biblioteca; consulta detalle en /personajes-publicos/:id. Sin edición ni favoritos locales.
Sesión del servidor	GET /api/auth/me al abrir; POST /api/auth/logout al salir. Workspace conserva biblioteca y ajustes locales por idUsuario.

UMBRA · EDICIÓN 09/11/2026

Pantallas y rutas

App.jsx define las rutas; navigation.js aporta títulos y enlaces comunes. Los filtros locales q, class, favorites y sort se guardan en la URL de Biblioteca y se conservan al volver de una ficha local.

Ruta	Contenido
/	Redirección a /inicio
/inicio	Home
/biblioteca	Library
/biblioteca/:id	Ficha local: consulta, favoritos, edición, duplicado y eliminación.
/personajes-publicos/:id	Consulta HTTP del detalle público del servidor.
/crear-personaje	CreateCharacter + CharacterEditor
/personaje/:id/editar	El mismo editor con una ficha existente
/inspiracion	Inspiration
/perfil	Profile
/login	Login; redirige a Perfil si hay sesión
/registro	Auth; redirige a Perfil si hay sesión
/recuperar	Auth con recovery; redirige si hay sesión
/accesibilidad	Menú de cinco secciones
/accesibilidad/texto	Tamaño de texto
/accesibilidad/contraste	Modo de contraste
/accesibilidad/lectura	Lectores de pantalla y voz
/accesibilidad/teclado	Atajos y foco
/accesibilidad/apariencia	Tema, controles, acentos y movimiento

Ruta	Contenido
/ayuda	Help
/privacidad	Privacy
Cualquier otra ruta	NotFound

Las rutas públicas se añaden en App, pero todavía no tienen un título específico en el cálculo global de Workspace; su encabezado puede mostrar Página no encontrada aunque el detalle se cargue. El alojamiento debe resolver las rutas internas a index.html, reservando /api para el backend.

UMBRA · EDICIÓN 09/11/2026

Cómo se conecta el código

index.html → main.jsx → BrowserRouter → App → Workspace → páginas y componentes

App consulta la sesión HTTP y adapta idUsuario a id de cadena, nombreUsuario a name, email y rol. Conserva loading y accountError; al terminar monta Workspace con la identidad o Invitado.

Autenticación y personajes públicos → services/api.js → /api → proxy Vite → Spring Boot

Biblioteca personal y accesibilidad → useCharacters / usePreferences → characterStore / preferenceStore → localStorage

Workspace usa key={user?.id || guest} para volver a cargar la biblioteca y preferencias del propietario al cambiar de sesión. Estas bibliotecas no se suben al backend.

CharacterCard crea un enlace para fichas locales. Con interactive=false representa un article dentro del enlace público. Portrait admite imageDataUrl, image, retrato incluido, imagePosition y respaldo con iniciales si falta o falla la imagen.

useEffect conecta consultas, foco, navegación y eventos con su limpieza. Los componentes públicos usan una bandera activo para ignorar respuestas después de desmontarse.

UMBRA · EDICIÓN 09/11/2026

Modelo y almacenamiento

El modelo siguiente corresponde a fichas personales locales; no es el DTO remoto de personajes públicos.

Propiedad	Tipo y significado
id	string: Identificador estable; nuevo UUID cuando se crea una ficha.
name	string: Nombre entre 2 y 60 caracteres tras recortar extremos.
type / race	string: Valores de characterTypes y characterRaces.
level	number: Entero entre 1 y 20.
realm / story	string: Origen hasta 100 caracteres e historia hasta 3000.
portrait	number: Índice del retrato base; sin recurso disponible se muestran iniciales.
imageDataUrl / imageName	string: Imagen local serializada y su nombre; se guardan dentro de la ficha.
traits	string[]: Hasta seis rasgos; la validación admite etiquetas de hasta 40 caracteres.
stats	number[4]: Fuerza, Destreza, Inteligencia y Carisma; enteros de 1 a 20.
favorite	boolean: Estado que utiliza el filtro de favoritos.
updatedAt	number: Fecha en milisegundos; sirve para ordenar fichas recientes.

Dato	Ubicación
umbra.characters.v1.<owner>	localStorage: biblioteca personal; owner es idUsuario convertido a cadena o guest.
umbra.accessibility.v2.<owner>	localStorage: accesibilidad y apariencia por perfil.
umbra.accessibility.v1	Legado: solo migración de ajustes de Invitado.
umbra.accounts.v1 / umbra.session.v1	Legado: cuentas y sesiones locales antiguas. El login y registro actuales no usan estas claves.

Dato	Ubicación
Sesión actual	Cookie gestionada por el servidor; React conserva identidad y GET /api/auth/me la comprueba al recargar.

Las respuestas de sesión esperan idUsuario, nombreUsuario, email y rol. La lista pública usa idPersonaje y nombre; el detalle muestra visibilidad, historia, personalidad, fuerza, destreza, inteligencia y carisma. No se normaliza el DTO público como ficha editable local.

Las claves locales dependen del origen: localhost:5173, 127.0.0.1:5173 y otros puertos tienen almacenamientos diferentes. Borrar datos del sitio elimina bibliotecas y ajustes locales, sin eliminar cuentas del backend. La exportación JSON solo incluye personajes.

PBKDF2-SHA-256 con sal y 100000 iteraciones y SHA-256 de recuperación pertenecen al módulo local antiguo. No describen la seguridad ni el almacenamiento de contraseñas de Spring Boot.

UMBRA · EDICIÓN 09/11/2026

Recorridos explicados

Crear o editar una ficha personal

CharacterEditor → AppForm → submit → saveCharacter → commit → writeCharacters → localStorage

El editor valida, construye la ficha y guarda antes de volver a Biblioteca. Si falla el guardado se conserva el estado anterior y aparece el error. Inspiración también valida y llama onSave directamente; los cambios manuales fijan campos y el historial permite recuperar ideas.

Registro y acceso

Auth → registrarUsuario → POST /api/auth/register
Login → App.login → iniciarSesion → POST /api/auth/login → Workspace

Registro valida nombre de 2 a 50 caracteres, correo hasta 100 y contraseña de 8 a 72 con carácter especial y confirmación. Envía nombreUsuario, email y password; el éxito muestra enlace a Login, sin código de recuperación. Login adapta la identidad devuelta y abre Perfil.

Restaurar y cerrar sesión

Al abrir se muestra Comprobando sesión… mientras GET /api/auth/me responde. Un 401 continúa como Invitado. Si hay otro error se permite continuar como Invitado con aviso. Logout espera POST /api/auth/logout; solo después elimina user de React. Si falla, Perfil muestra un error.

Personajes públicos

Biblioteca monta PublicCharacters y consulta GET /api/personajes/publicos. Las tarjetas muestran nombre, Personaje público e iniciales, sin nivel. Al abrir una tarjeta se consulta GET /api/personajes/{id}. El detalle presenta texto, visibilidad y atributos, con carga y errores. Los filtros de biblioteca personal no afectan a este listado.

Recordarme y recuperación

Recordarme se recoge en Login, pero api.js no lo serializa. No modifica la duración de sesión. Recuperar acceso sigue llamando recoverAccount sobre cuentas locales antiguas: no restablece las cuentas de la API y no envía correo.

Copias JSON

Perfil exporta app: UMBRA, version: 1 y characters, con favoritos y retratos. Importar admite hasta 10 MB y 500 fichas, valida campos e ids únicos y pide confirmación antes de reemplazar toda la biblioteca activa. No incluye cuentas, sesiones ni accesibilidad; una biblioteca vacía se conserva vacía.

Errores y conexión

Los fallos de biblioteca conservan el estado anterior. Accesibilidad puede aplicar cambios temporalmente si no puede guardarlos. ConnectionStatus informa de eventos del navegador, sin comprobar el backend. No hay service worker; las funciones HTTP requieren el servidor.

7 · AJUSTES INDEPENDIENTES

Accesibilidad por perfil

Menú → Accesibilidad abre cinco entradas. Cada una muestra el nombre del perfil al que se aplican los cambios.

Menú	Opciones
Tamaño de texto	small, normal, large y extra-large. El deslizador y los radios cambian el mismo valor.
Modo de contraste	normal, high e inverted. Los modos de contraste sustituyen los colores de la interfaz y mantienen la selección de tema.
Lectores de pantalla	announceNavigation añade el nombre de la pantalla a una región de estado. readAloud muestra lectura voluntaria de fichas. readingRate elige 0.75, 1 o 1.25.
Navegación por teclado	enhancedFocus refuerza el contorno. La guía explica Tab, Mayús+Tab, Enter/Espacio, flechas, Inicio/Fin y Escape.
Personalización visual	theme: dark/light/sepia; elementSize: normal/large/extra-large; accent: red/purple/blue/green/orange/slate; reduceMotion.

Control → Accessibility.update → usePreferences.updatePreferences → preferenceStore → data-* de Workspace → variables CSS

La normalización admite solo valores del catálogo. Cada propietario utiliza su clave v2. Invitado puede recuperar la antigua configuración global; una cuenta nueva recibe defaults. Restablecer modifica únicamente el perfil activo.

Las etiquetas semánticas, textos alternativos y el diálogo nativo se ofrecen siempre. La voz adicional depende de las capacidades y las voces del dispositivo; no activa ni sustituye TalkBack, VoiceOver o NVDA.

Las reglas CSS de contraste aparecen después de los temas y acentos para que prevalezcan. Al elegir contraste normal vuelven a verse los colores guardados. La app también respeta la preferencia de movimiento reducido del sistema.

8 · TRABAJAR CON EL PROYECTO

Ejecución y pruebas

Extrae codigo/ en una carpeta y abre una terminal allí. En PowerShell puede utilizarse npm.cmd para evitar el bloqueo de npm.ps1 por la política de ejecución.

Comando	Para qué sirve
`npm.cmd install`	Instala dependencias según el proyecto. Con un lock compatible, npm.cmd ci ofrece instalación reproducible.
`npm.cmd run dev`	Inicia Vite para editar; se abre la URL que muestre la terminal.
`npm.cmd test`	Ejecuta las 17 pruebas unitarias declaradas mediante node --test.
`npm.cmd run lint`	Comprueba reglas de JavaScript, JSX y hooks.
`npm.cmd run build`	Compila el frontend en dist/.
`npm.cmd run preview`	Sirve la compilación para comprobarla localmente.
`npm.cmd run test:browser`	Ejecuta recorridos con app activa y Chrome/Edge con CDP en el puerto 9222.

Pruebas unitarias incluidas: 17

Archivo	Caso
characterStore.test.js	una biblioteca vacía persiste y las cuentas conservan bibliotecas independientes
characterStore.test.js	una copia restaura los datos y rechaza identificadores repetidos o contenido inválido
characterStore.test.js	un fallo de almacenamiento no destruye la copia anterior
localAccounts.test.js	registro, verificación de contraseña, sesiones temporales y Recordarme
localAccounts.test.js	recuperación requiere el código y cambia la contraseña conservando la identidad
preferenceStore.test.js	cada perfil y el invitado conservan todos sus ajustes independientes
preferenceStore.test.js	los ajustes globales antiguos se recuperan solo para el invitado
preferenceStore.test.js	los valores desconocidos no se aplican y el fallo de guardado no sobrescribe ajustes anteriores

Archivo	Caso
validateForm.test.js	un nombre vacío o compuesto de espacios requiere una corrección
validateForm.test.js	el correo exige usuario, arroba y dominio
validateForm.test.js	el registro exige longitud suficiente, un carácter especial y contraseñas iguales
validateForm.test.js	el nivel debe ser entero y estar entre 1 y 20
validateForm.test.js	una casilla obligatoria debe estar marcada
generateCharacter.test.js	genera fichas completas válidas y varía todos los campos
generateCharacter.test.js	cada nueva combinación cambia todos los campos libres incluso con una secuencia aleatoria constante
generateCharacter.test.js	fijar campos conserva sus valores y sus imágenes sin mutar el borrador anterior
generateCharacter.test.js	regenerar solo los atributos conserva todos los demás campos

Recorridos de navegador

browser-smoke.mjs utiliza un contexto temporal aislado mediante CDP. Contiene recorridos de fichas locales, copias, perfiles y accesibilidad. El recorrido de cuentas espera el antiguo registro local, su código de recuperación y etiquetas anteriores; queda pendiente adaptarlo al flujo HTTP y a los personajes públicos. UMBRA_URL cambia la URL base y UMBRA_SCREENSHOT guarda una captura.

Las voces se simulan en algunos casos; esas pruebas comprueban la conexión de los controles, no una salida de audio real ni el uso de un lector de pantalla externo. La existencia de pruebas no garantiza por sí sola que se hayan ejecutado después de cada cambio.

Publicación

build produce dist/. El alojamiento debe resolver las rutas de BrowserRouter a index.html y utilizar HTTPS cuando se sirva fuera de localhost. Vite ya declara server.proxy para /api hacia http://localhost:8080; en producción se requiere un proxy inverso equivalente y fallback de rutas a index.html.

UMBRA · EDICIÓN 09/11/2026

Configuración e integración actual con Spring Boot

La capa HTTP ya existe en src/services/api.js. El frontend consulta el backend; MariaDB y el ORM pertenecen al servidor y no se incluyen en este repositorio.

Configuración de desarrollo

Arranca Spring Boot por separado en http://localhost:8080 y Vite con npm.cmd run dev. server.proxy reenvía /api a ese servidor con changeOrigin: true y sin rewrite; /api/auth/me conserva el prefijo /api. server.watch.ignored mantiene **/public/fonts/**.

.env.development declara VITE_API_URL=http://localhost:8080/api, pero api.js no lee esa variable: usa rutas relativas /api/... y el destino efectivo lo determina el proxy de Vite. Cambiar solo la variable no cambia las solicitudes. Reinicia Vite después de cambiar su configuración.

Método y ruta	Contrato consumido por el cliente
GET /api/personajes/publicos	Lista JSON; tarjetas usan idPersonaje y nombre.
GET /api/personajes/{id}	Detalle JSON: nombre, visibilidad, historia, personalidad y cuatro atributos. El servicio codifica el id.
POST /api/auth/register	JSON de entrada: nombreUsuario, email, password. Éxito espera JSON; 409: correo existente, 400: datos incorrectos.
POST /api/auth/login	JSON de entrada: email, password. Respuesta: idUsuario, nombreUsuario, email, rol. 401: credenciales incorrectas.
GET /api/auth/me	Misma identidad que login; 401 devuelve null. Cabecera X-Requested-By: XMLHttpRequest.
POST /api/auth/logout	Cierra sesión; el cliente comprueba el estado, sin leer JSON de respuesta.

Sesión

Login, me y logout declaran credentials: same-origin. El cliente espera una sesión mediante cookie del backend y no guarda tokens. La duración y atributos de cookies y las reglas de autorización requieren consultar el backend; no se deducen de este código.

Publicación

El contenido de dist requiere un alojamiento con fallback a index.html para las rutas React y un proxy inverso de /api a Spring Boot en el mismo origen. server.proxy es de desarrollo y no forma parte de la

compilación; Vite hereda server.proxy para la vista previa local al no declarar preview.proxy. VITE_API_URL no cambia las rutas del cliente. Utiliza HTTPS y verifica la sesión con el servidor real.

Trabajo pendiente

CRUD remoto de fichas personales, sincronización de ajustes, recuperación de contraseña del servidor, integración de Recordarme, adaptación de textos de ayuda y privacidad y actualización del recorrido de navegador. rol se conserva en memoria, sin controles por rol en las pantallas actuales.

10 · CONSULTA DEL CÓDIGO

Índice de archivos

A continuación aparecen las explicaciones y los listados completos. Las líneas son las del archivo original; una línea larga puede ocupar varias líneas impresas conservando un único número.

Ruta	Responsabilidad
index.html	Documento HTML que recibe el navegador y punto de entrada de Vite.
src/main.jsx	Arranca React, conecta el enrutador y carga los estilos globales.
src/App.jsx	Coordina la sesión del servidor, las rutas y el espacio local de cada perfil.
src/pages/Home.jsx	Pantalla de inicio: portada, acceso a crear personajes y las dos fichas más recientes.
src/pages/Library.jsx	Explora toda la biblioteca con búsqueda, filtros, ordenación y estados vacíos.
src/pages/CreateCharacter.jsx	Comparte un editor entre la creación y la modificación de fichas.
src/pages/Login.jsx	Formulario de acceso que delega la autenticación en la API.
src/pages/Auth.jsx	Registro mediante API y recuperación local antigua en un formulario compartido.
src/pages/Profile.jsx	Muestra la identidad activa, sus recuentos, el cierre de sesión y los controles de copia de seguridad.
src/pages/Accessibility.jsx	Contiene el menú principal de accesibilidad y los cinco submenús, con configuración independiente por perfil.
src/pages/Information.jsx	Agrupa Inspiración, Ayuda, Privacidad y la pantalla de ruta inexistente.
src/components/header/Header.jsx	Cabecera compartida con marca, menú de secciones y acceso a avisos o ayuda.
src/components/bottomnavigation/BottomNavigation.jsx	Barra inferior para Inicio, Biblioteca, Inspiración y Perfil.
src/components/hero/Hero.jsx	Portada de Inicio con paisaje, mensaje de bienvenida y acceso al editor.
src/components/charactercard/CharacterCard.jsx	Tarjeta reutilizable para fichas locales y públicas.

Ruta	Responsabilidad
src/components/charactercard/Portrait.jsx	Retrato con prioridades de imagen, encuadre configurable e iniciales de respaldo.
src/components/modal/Modal.jsx	Ventana modal reutilizable basada en el elemento nativo dialog.
src/components/forms/AppForm.jsx	Motor de formularios reutilizable para personajes, registro y recuperación.
src/components/feedback/ConnectionStatus.jsx	Aviso de pérdida y recuperación de conexión del navegador.
src/components/feedback/ReadAloud.jsx	Botón para leer texto en español mediante las voces del navegador o dispositivo.
src/components/icon/Icon.jsx	Catálogo de iconos SVG reutilizables sin archivos externos.
src/hooks/useCharacters.js	Mantiene la biblioteca React del perfil y coordina las escrituras persistentes.
src/hooks/usePreferences.js	Carga y aplica los ajustes de accesibilidad del perfil activo.
src/utils/characterStore.js	Valida fichas, lee y escribe bibliotecas y prepara las copias JSON.
src/utils/localAccounts.js	Módulo local anterior; la recuperación todavía lo usa, el acceso y registro actuales usan la API.
src/utils/preferenceStore.js	Define valores predeterminados y persistencia de accesibilidad separada por perfil.
src/utils/validateForm.js	Validación declarativa compartida por AppForm y Login.
src/data/characters.js	Seis personajes de ejemplo para inicializar bibliotecas locales.
src/data/characterOptions.js	Catálogos de clases, razas, rasgos y nombres de atributos, además de la ficha vacía.
src/data/navigation.js	Fuente común de enlaces y títulos de pantalla.
src/data/accessibility.js	Catálogos de valores y etiquetas de los cinco menús de accesibilidad.
src/index.css	Fuentes locales, reinicio de estilos y variables base de color, texto y controles.
src/App.css	Composición general, tarjetas, formularios, diálogos y reglas globales de accesibilidad.

Ruta	Responsabilidad
src/pages/Login.css	Composición específica de la pantalla de acceso.
src/pages/Accessibility.css	Presentación del menú y los controles de accesibilidad.
package.json	Identidad del paquete, dependencias directas y comandos de desarrollo.
vite.config.js	Plugin React, proxy de API de desarrollo y exclusión de fuentes del observador.
eslint.config.js	Reglas de análisis estático para JavaScript y JSX.
tests/characterStore.test.js	Pruebas del almacenamiento y las copias de personajes.
tests/localAccounts.test.js	Pruebas del módulo de cuentas locales anterior; no prueban la autenticación de Spring Boot.
tests/preferenceStore.test.js	Pruebas de accesibilidad independiente por perfil.
tests/validateForm.test.js	Pruebas de las reglas compartidas de formularios.
scripts/browser-smoke.mjs	Recorridos de integración en Chrome o Edge mediante Chrome DevTools Protocol, sin dependencia de Playwright.
public/favicon.svg	Icono vectorial de la pestaña del navegador.
public/images/ASSETS.md	Descripción y procedencia de los recursos gráficos incluidos.
.gitignore	Patrones que Git ignora para evitar añadir dependencias, compilados y archivos temporales.
README.md	Guía bilingüe del frontend, configuración de API y límites de la integración actual.
LICENSE	Licencia del proyecto incluida en el repositorio.
public/fonts/Inter-OFL.txt	Licencia de fuente distribuida con los recursos.
public/fonts/Cormorant-Garamond-OFL.txt	Licencia de fuente distribuida con los recursos.
src/data/inspiration.js	Catálogos narrativos y campos del generador.
src/pages/CharacterDetail.css	Estilos de la ficha local y su confirmación de borrado.
src/pages/CharacterDetail.jsx	Pantalla propia de cada ficha local, con acciones y confirmación de borrado.

Ruta	Responsabilidad
src/pages/Inspiration.css	Estilos de la ficha editable de Inspiración.
src/pages/Inspiration.jsx	Generador visual editable que guarda directamente en la biblioteca local.
src/pages/PublicCharacterDetail.jsx	Consulta por id una ficha del backend.
src/services/api.js	Servicios fetch para personajes públicos y autenticación de Spring Boot.
src/utils/generateCharacter.js	Genera todos los campos respetando bloqueos y evitando repetir valores anteriores.
src/utils/readPortrait.js	Lectura y comprobación compartida de retratos subidos.
src/components/publiccharacters/PublicCharacters.jsx	Listado remoto independiente al final de la biblioteca.
tests/generateCharacter.test.js	Pruebas del generador completo y sus campos fijados.
.env.development	Variable de entorno declarada para la dirección de la API.

ARCHIVO 01 DE 62

index.html

`index.html`

Documento HTML que recibe el navegador y punto de entrada de Vite.

- Declara el idioma español, la codificación UTF-8, el viewport móvil, el icono y la descripción.
- El div #root es el lugar donde React monta la interfaz. El script de tipo module carga src/main.jsx.

Funciones y elementos principales

Nombre	Qué hace	Línea
`#root`	Contenedor de montaje de React; no es una función de JavaScript.	—

Código completo

15 líneas · SHA-256 del original: `a4b5b04a163ff023f3c8102426002f20b1b158dc381205f4221eb128a55c1dac`

```
1  <!doctype html>
2  <html lang="es">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <meta name="theme-color" content="#090f0f" />
7      <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
8      <meta name="description" content="UMBRA: crea personajes y da vida a tus historias." />
9      <title>UMBRA · Tus historias</title>
10   </head>
11   <body>
12     <div id="root"></div>
13     <script type="module" src="/src/main.jsx"></script>
14   </body>
15 </html>
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 02 DE 62

main.jsx

`src/main.jsx`

Arranca React, conecta el enrutador y carga los estilos globales.

- createRoot(document.getElementById('root')) crea la raíz de React.
- BrowserRouter permite interpretar las rutas de la barra de direcciones. StrictMode ayuda a detectar efectos y limpiezas incorrectos durante el desarrollo.

Funciones y elementos principales

Nombre	Qué hace	Línea
`createRoot(...).render(...)`	Monta App dentro de BrowserRouter y StrictMode.	—

Código completo

13 líneas · SHA-256 del original: `a0c30b0734ced3997d30279cc261a3ab1c2dc13a7a16f736b62a6d8e132b012d`

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import { BrowserRouter } from 'react-router'
4  import App from './App.jsx'
5  import './index.css'
6
7  createRoot(document.getElementById('root')).render(
8    <StrictMode>
9      <BrowserRouter>
10       <App />
11     </BrowserRouter>
12   </StrictMode>,
13 )
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 03 DE 62

App.jsx

`src/App.jsx`

Coordina la sesión del servidor, las rutas y el espacio local de cada perfil.

Entradas / props. Workspace recibe user, onLogin, onLogout y accountError.

Estado. App: user, loading y accountError. Workspace: notificationsOpen; biblioteca y preferencias proceden de hooks.

- Al montar consulta obtenerSesion; transforma idUsuario en id de cadena y conserva nombreUsuario, email y rol.
- Muestra Comprobando sesión… hasta terminar. Un 401 equivale a Invitado; otros fallos muestran un aviso sin impedir el uso local.
- Workspace usa una key igual al id o guest y monta useCharacters y usePreferences para ese propietario.
- La ficha local abre /biblioteca/:id; la pública abre /personajes-publicos/:id. El diálogo de notificaciones sigue siendo local.
- El login llama a iniciarSesion; el logout espera cerrarSesion antes de volver a Invitado. rol se conserva sin controles de autorización en el frontend.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Workspace`	Compone rutas, navegación, avisos y ajustes.	24
`App`	Comprueba la sesión HTTP inicial y conserva user, loading y accountError.	79
`login`	Envía credenciales al backend y adapta la identidad devuelta.	112
`logout`	Espera el cierre de sesión HTTP y cambia a Invitado.	123

Código completo

141 líneas · SHA-256 del original: `c03c4b5c97374ed7652227a4bffd9eaadf3db84a16b3f2add0ea4ffeade88b96`

```
1  import { useEffect, useRef, useState } from 'react'
2  import { Link, Navigate, Route, Routes, useLocation, useMatch } from 'react-router'
3  import Header from './components/header/Header.jsx'
4  import BottomNavigation from './components/bottomnavigation/BottomNavigation.jsx'
```

```
import ConnectionStatus from './components/feedback/ConnectionStatus.jsx'
import Modal from './components/modal/Modal.jsx'
import Home from './pages/Home.jsx'
import Login from './pages/Login.jsx'
import Library from './pages/Library.jsx'
import CharacterDetail from './pages/CharacterDetail.jsx'
import CreateCharacter from './pages/CreateCharacter.jsx'
import Auth from './pages/Auth.jsx'
import Profile from './pages/Profile.jsx'
import Accessibility from './pages/Accessibility.jsx'
import Inspiration from './pages/Inspiration.jsx'
import { Help, NotFound, Privacy } from './pages/Information.jsx'
import { useCharacters } from './hooks/useCharacters.js'
import { usePreferences } from './hooks/usePreferences.js'
import { iniciarSesion, obtenerSesion, cerrarSesion } from './services/api.js'
import { pageTitles } from './data/navigation.js'
import PublicCharacterDetail from './pages/PublicCharacterDetail.jsx'
import './App.css'

function Workspace({ user, onLogin, onLogout, accountError }) {
  const { pathname } = useLocation()
  const main = useRef(null)
  const [notificationsOpen, setNotificationsOpen] = useState(false)
  const { characters, storageError, saveCharacter, deleteCharacter, toggleFavorite, replaceCharacters } = useCharacters(user?.id || 'guest')
  const { preferences, updatePreferences, preferenceError } = usePreferences(user?.id || 'guest')
  const path = pathname.replace(/\/$/, '') || '/'
  const detailMatch = useMatch('/biblioteca/:id')
  const character = detailMatch ? characters.find(item => item.id === detailMatch.params.id) : null
  const editing = /^\/personaje\/[^/]+\/editar$/.test(path)
  const page = pageTitles[path] || (detailMatch ? character?.name || 'Personaje no encontrado' : editing ? 'Editar personaje' : 'Página no encontrada')
  const isLogin = path === '/login'

  useEffect(() => {
    document.title = `UMBRA · ${page}`
    main.current.focus({ preventScroll: true })
    window.scrollTo(0, 0)
  }, [pathname, page])

  const className = ['app-frame', isLogin && 'login-frame', preferences.enhancedFocus && 'enhanced-focus', preferences.reduceMotion && 'reduce-motion'].filter(Boolean).join(' ')

  return <div className={className} data-text-size={preferences.textSize} data-contrast={preferences.contrast} data-theme={preferences.theme} data-element-size={preferences.elementSize} data-accent={preferences.accent}>
    <a className="skip-link" href="#main-content">Saltar al contenido</a>
    <Header login={isLogin} user={user} onNotify={() => setNotificationsOpen(true)} />
    <ConnectionStatus />
    {preferences.announceNavigation && <p className="visually-hidden" role="status" aria-live="polite" aria-atomic="true">Página: {page}</p>}
    {(storageError || accountError) && <p className="storage-notice" role="alert">{storageError || accountError}</p>}
    <main ref={main} className="main-content" id="main-content" tabIndex={-1}>
      {path !== '/inicio' && path !== '/' && !isLogin && <div className="page-heading"><p className="eyebrow">UMBRA · TUS HISTORIAS</p><h1>{page}</h1></div>}
      <Routes>
        <Route path="/personajes-publicos/:id" element={<PublicCharacterDetail />}/>
        <Route path="/" element={<Navigate to="/inicio" replace />} />
```

```
        <Route path="/inicio" element={<Home characters={characters} user={user} />} />
        <Route path="/biblioteca" element={<Library characters={characters} />} />
        <Route path="/biblioteca/:id" element={<CharacterDetail key={path} characters={characters} onSave={saveCharacter} onDelete={deleteCharacter} onToggleFavorite={toggleFavorite} preferences={preferences} />} />
        <Route path="/crear-personaje" element={<CreateCharacter key="create" characters={characters} onSave={saveCharacter} />} />
        <Route path="/personaje/:id/editar" element={<CreateCharacter key={path} characters={characters} onSave={saveCharacter} />} />
        <Route path="/inspiracion" element={<Inspiration preferences={preferences} onSave={saveCharacter} />} />
        <Route path="/perfil" element={<Profile user={user} characters={characters} onLogout={onLogout} onImport={replaceCharacters} />} />
        <Route path="/login" element={user ? <Navigate to="/perfil" replace /> : <Login onLogin={onLogin} />} />
        <Route path="/registro" element={user ? <Navigate to="/perfil" replace /> : <Auth key="register" />} />
        <Route path="/recuperar" element={user ? <Navigate to="/perfil" replace /> : <Auth key="recover" recovery />} />
        <Route path="/ayuda" element={<Help />} />
        <Route path="/accesibilidad/:section?" element={<Accessibility preferences={preferences} onChange={updatePreferences} error={preferenceError} user={user} />} />
        <Route path="/privacidad" element={<Privacy />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </main>
    {!isLogin && <BottomNavigation />}
    {notificationsOpen && <Modal title="Notificaciones" onClose={() => setNotificationsOpen(false)}>
      <div className="action-stack"><p className="page-description">No tienes avisos nuevos. Tu biblioteca tiene {characters.length} personajes y se guarda en este navegador.</p><p className="form-hint">Recuerda exportar una copia desde Perfil para conservar tus historias.</p><Link className="secondary-button" to="/perfil" onClick={() => setNotificationsOpen(false)}>Ir a Perfil</Link></div>
    </Modal>}
  </div>
}

export default function App() {
  const [user, setUser] = useState(null)
  const [loading, setLoading] = useState(true)
  const [accountError, setAccountError] = useState('')

  useEffect(() => {
    let activo = true

    obtenerSesion()
      .then(datos => {
        if (activo && datos) {
          setUser({
            id: String(datos.idUsuario),
            name: datos.nombreUsuario,
            email: datos.email,
            rol: datos.rol,
          })
        }
      })
      .catch(() => {
        if (activo) {
          setAccountError('No se pudo comprobar la sesión con el servidor.')
```

```
        }
      })
      .finally(() => {
        if (activo) setLoading(false)
      })

    return () => {
      activo = false
    }
  }, [])

  async function login(datos) {
    const respuesta = await iniciarSesion(datos)

    setUser({
      id: String(respuesta.idUsuario),
      name: respuesta.nombreUsuario,
      email: respuesta.email,
      rol: respuesta.rol,
    })
  }

  async function logout() {
    await cerrarSesion()
    setUser(null)
  }

  if (loading) {
    return <p role="status">Comprobando sesión...</p>
  }

  return (
    <Workspace
      key={user?.id || 'guest'}
      user={user}
      onLogin={login}
      onLogout={logout}
      accountError={accountError}
    />
  )
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 04 DE 62

Home.jsx

`src/pages/Home.jsx`

Pantalla de inicio: portada, acceso a crear personajes y las dos fichas más recientes.

Entradas / props. characters: biblioteca local activa; user: identidad del servidor o null.

- Ordena una copia de characters por updatedAt y toma los dos primeros. No cambia el orden del array original.
- Utiliza Hero y CharacterCard. Muestra un estado vacío cuando no existen personajes y un acceso al perfil o al login.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Home`	Construye Inicio a partir de la biblioteca y de la sesión activa.	5

Código completo

21 líneas · SHA-256 del original: `757cb1e048186f6b9ea7e76afa9f84aba37321c9e688593ea117bd6ca920f78b`

```
1  import { Link } from 'react-router'
2  import Hero from '../components/hero/Hero.jsx'
3  import CharacterCard from '../components/charactercard/CharacterCard.jsx'
4
5  export default function Home({ characters, user }) {
6    const recent = [...characters].sort((a, b) => b.updatedAt - a.updatedAt).slice(0, 2)
7    return <>
8      <Hero />
9      <div className="ornament" aria-hidden="true"><span>❖</span></div>
10     <section aria-labelledby="characters-title">
11       <div className="section-heading">
12         <h2 id="characters-title">Personajes recientes</h2>
13         <Link className="text-button" to="/biblioteca">Ver todos <span aria-hidden="true">→</span></Link>
14       </div>
15       <div className="character-grid">{recent.map(character => <CharacterCard key={character.id}
   character={character} />)}</div>
16       {!recent.length && <p className="page-description">Tu biblioteca está vacía. Crea un personaje para
   comenzar tu historia.</p>}
17     </section>
18     <p className="closing-quote">«Incluso en la oscuridad, una historia perdura.»</p>
19     <p className="guest-note">{user ? <>Bienvenido, {user.name}. <Link to="/perfil">Ver perfil</Link></>
   : <>Estás explorando como invitado. <Link to="/login">Iniciar sesión</Link></>}</p>
20   </>
21 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 05 DE 62

Library.jsx

`src/pages/Library.jsx`

Explora toda la biblioteca con búsqueda, filtros, ordenación y estados vacíos.

Entradas / props. characters: biblioteca local del perfil.

Estado. Los parámetros q, class, favorites y sort de la URL conservan los filtros.

- Busca en nombre, clase, raza y origen de la biblioteca local, normalizando tildes y mayúsculas.
- Combina clase, favoritos y orden; updateFilter actualiza los parámetros con replace para conservarlos al volver de una ficha.
- Al final monta PublicCharacters, que consulta su propia lista remota. Los filtros locales no afectan a los personajes públicos.

Funciones y elementos principales

Nombre	Qué hace	Línea
`simplify`	Normaliza una cadena con NFD, elimina marcas diacríticas y convierte a minúsculas españolas.	7
`Library`	Calcula los resultados y presenta los filtros, el contador y las tarjetas.	9
`updateFilter`	Modifica un parámetro de filtro en la URL.	16

Código completo

44 líneas · SHA-256 del original: `7e195469aeb6c10d0f32c2391e46c8a26b8e1ccf5d3cb1757eb6a9a66bbd6330`

```
1  import { Link, useLocation, useSearchParams } from 'react-router'
2  import CharacterCard from '../components/charactercard/CharacterCard.jsx'
3  import Icon from '../components/icon/Icon.jsx'
4  import { characterTypes } from '../data/characterOptions.js'
5  import PublicCharacters from '../components/publiccharacters/PublicCharacters.jsx'
6
7  const simplify = value => value.normalize('NFD').replace(/[\u0300-\u036f]/g, '').toLocaleLowerCase('es')
8
9  export default function Library({ characters }) {
10   const [params, setParams] = useSearchParams()
11   const search = params.get('q') || ''
12   const type = characterTypes.includes(params.get('class')) ? params.get('class') : ''
13   const favorites = params.get('favorites') === '1'
14   const sort = ['recent', 'name', 'level'].includes(params.get('sort')) ? params.get('sort') : 'recent'
15   const { state } = useLocation()
```

```
16   function updateFilter(key, value) {
17     setParams(previous => {
18       const next = new URLSearchParams(previous)
19       if (value) next.set(key, value)
20       else next.delete(key)
21       return next
22     }, { replace: true })
23   }
24   const results = characters.filter(character =>
25     simplify(`${character.name} ${character.type} ${character.race}
${character.realm}`).includes(simplify(search.trim())) &&
26     (!type || character.type === type) && (!favorites || character.favorite))
27     .sort((a, b) => sort === 'name' ? a.name.localeCompare(b.name, 'es') : sort === 'level' ? b.level -
a.level : b.updatedAt - a.updatedAt)
28
29   return <section aria-labelledby="library-title">
30     <div className="section-heading"><h2 id="library-title">Tus personajes</h2><Link className="text-
button" to="/crear-personaje">+ Crear personaje</Link></div>
31     {state?.message && <p className="feedback success" role="status">{state.message}</p>}
32     <label className="search-field"><Icon name="search" /><input aria-label="Buscar por nombre, clase,
raza o reino" type="search" value={search} onChange={event => updateFilter('q', event.target.value)}
placeholder="Buscar personaje…" /></label>
33     <div className="form-row library-filters">
34       <label>Clase<select value={type} onChange={event => updateFilter('class', event.target.value)}>
<option value="">Todas las clases</option>{characterTypes.map(item => <option key={item}>{item}
</option>)}</select></label>
35       <label>Ordenar<select value={sort} onChange={event => updateFilter('sort', event.target.value)}>
<option value="recent">Más recientes</option><option value="name">Nombre</option><option
value="level">Mayor nivel</option></select></label>
36     </div>
37     <label className="preference-option"><input type="checkbox" checked={favorites} onChange={event =>
updateFilter('favorites', event.target.checked ? '1' : '')} />Solo favoritos</label>
38     <p className="search-results" role="status">{results.length} {results.length === 1 ? 'personaje
encontrado' : 'personajes encontrados'}.</p>
39     <div className="character-grid">{results.map(character => <CharacterCard key={character.id}
character={character} />)}</div>
40     {results.length === 0 && <div className="empty-state"><p>{characters.length ? 'No hay personajes que
coincidan con los filtros.' : 'Tu biblioteca está vacía. Empieza una nueva leyenda.'}</p>
{characters.length ? <button className="secondary-button" onClick={() => setParams({}, { replace: true
})}>Limpiar filtros</button> : <Link className="primary-button" to="/crear-personaje">Crear mi primer
personaje</Link>}</div>}
41     <p className="page-description">Tus cambios se guardan en este navegador. Puedes exportar una copia
de seguridad desde Perfil.</p>
42     <PublicCharacters />
43   </section>
44 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 06 DE 62

CreateCharacter.jsx

`src/pages/CreateCharacter.jsx`

Comparte un editor entre la creación y la modificación de fichas.

Entradas / props. CreateCharacter recibe characters y onSave. CharacterEditor recibe initial, onSave y editing.

Estado. El contenedor mantiene draft y revision. El editor mantiene el retrato, el estilo, el error de imagen y si está leyéndose un archivo.

- La ruta con :id busca la ficha existente. Sin :id utiliza blankCharacter o el borrador enviado desde Inspiración. Un id ausente muestra NotFound.
- Declara campos de nombre, clase, raza, nivel, origen, historia y cuatro atributos para AppForm. Ofrece hasta seis rasgos de personalidad.
- readPortrait valida y decodifica PNG/JPEG/WebP de hasta 750 KB y devuelve imageDataUrl e imageName para la vista previa.
- Generar personaje aleatorio usa utils/generateCharacter.js y renueva el borrador completo; se guarda al enviar el editor.
- Cancelar muestra confirmación si AppForm ha detectado cambios; no existe un bloqueo general de todos los enlaces internos ni una recuperación automática de borradores.

Funciones y elementos principales

Nombre	Qué hace	Línea
`CharacterEditor`	Presenta AppForm, los rasgos y la selección o carga del retrato.	10
`upload`	Revisa tipo y tamaño del archivo, lee su contenido, comprueba la imagen y actualiza la vista previa.	26
`submit`	Construye la ficha con FormData, llama onSave y navega a Biblioteca con un mensaje.	38
`CreateCharacter`	Decide si se está creando o editando y prepara la ficha inicial.	64

Código completo

76 líneas · SHA-256 del original: `95d2e845d25ee8042a17a92bbc6a21d3320ea5cd18331a409d88e3efc4fca4ad`

```
import { useState } from 'react'
import { useLocation, useNavigate, useParams } from 'react-router'
import AppForm from '../components/forms/AppForm.jsx'
import Portrait from '../components/charactercard/Portrait.jsx'
import { attributeNames, blankCharacter, characterRaces, characterTypes, personalityTraits } from '../data/characterOptions.js'
import { randomCharacter } from '../utils/generateCharacter.js'
import { readPortrait } from '../utils/readPortrait.js'
import { NotFound } from './Information.jsx'

function CharacterEditor({ initial, onSave, editing }) {
  const navigate = useNavigate()
  const [image, setImage] = useState({ imageDataUrl: initial.imageDataUrl || '', imageName: initial.imageName || '' })
  const [portrait, setPortrait] = useState(initial.portrait <= 1 ? initial.portrait : 2)
  const [imageError, setImageError] = useState('')
  const [readingImage, setReadingImage] = useState(false)
  const fields = [
    { name: 'name', label: 'Nombre del personaje', required: true, minLength: 2, maxLength: 60, defaultValue: initial.name, placeholder: 'El nombre de tu leyenda' },
    { name: 'type', label: 'Clase', required: true, options: characterTypes, defaultValue: initial.type },
    { name: 'race', label: 'Raza', required: true, options: characterRaces, defaultValue: initial.race },
    { name: 'level', label: 'Nivel', type: 'number', required: true, min: 1, max: 20, defaultValue: initial.level, hint: 'Un número entero entre 1 y 20.' },
    { name: 'realm', label: 'Reino u origen', maxLength: 100, defaultValue: initial.realm },
    { name: 'story', label: 'Historia', type: 'textarea', maxLength: 3000, defaultValue: initial.story, placeholder: 'Toda leyenda tiene un comienzo…' },
    ...attributeNames.map((label, index) => ({ name: `stat${index}`, label, type: 'number', required: true, min: 1, max: 20, defaultValue: initial.stats[index], hint: 'Entre 1 y 20.' })),
  ]

  async function upload(event) {
    const input = event.currentTarget
    const file = input.files[0]
    if (!file) return
    setImageError('')
    setReadingImage(true)
    try {
      setImage(await readPortrait(file))
    } catch (error) { setImageError(error.message); input.value = '' }
    finally { setReadingImage(false) }
  }

  function submit(values) {
    if (readingImage) throw new Error('Espera a que termine de cargarse el retrato.')
    const character = onSave({
      ...initial, ...image, portrait,
      name: values.get('name'), type: values.get('type'), race: values.get('race'),
      level: Number(values.get('level')), realm: values.get('realm'), story: values.get('story'),
      stats: attributeNames.map((_, index) => Number(values.get(`stat${index}`))), traits: values.getAll('traits'),
    })
    navigate('/biblioteca', { state: { message: `${character.name}: ${editing ? 'cambios guardados' : 'personaje creado'}.` } })
  }

  return <AppForm fields={fields} submitLabel={editing ? 'Guardar cambios' : 'Guardar personaje'} onSubmit={submit} cancelTo="/biblioteca">
```

```
50      <fieldset className="traits-field"><legend>Rasgos de personalidad (hasta 6)</legend><div
    className="traits-grid">{[...new Set([...personalityTraits, ...initial.traits])].map(trait => <label
    className="checkbox-label" key={trait}><input type="checkbox" name="traits" value={trait} defaultChecked=
    {initial.traits.includes(trait)} />{trait}</label>)}</div></fieldset>
51      <div className="portrait-editor">
52        <h2>Retrato</h2>
53        <Portrait character={{ ...initial, ...image, portrait, name: initial.name || 'tu personaje' }} />
54        <label>Estilo de retrato<select value={portrait} onChange={event => {
    setPortrait(Number(event.target.value)); setImage({ imageDataUrl: '', imageName: '' }) }}><option value=
    {0}>Hechicera</option><option value={1}>Pícaro</option><option value={2}>Iniciales del personaje</option>
    </select></label>
55        <label>Subir imagen<input type="file" accept="image/png,image/jpeg,image/webp" onChange={upload}
    disabled={readingImage} /></label>
56        <p className="form-hint">PNG, JPG o WebP. Máximo 750 KB.{image.imageName && ` Imagen:
    ${image.imageName}`}</p>
57        {readingImage && <p role="status">Cargando retrato…</p>}
58        {image.imageDataUrl && <button type="button" className="secondary-button" onClick={() => setImage({
    imageDataUrl: '', imageName: '' })}>Quitar imagen subida</button>}
59        {imageError && <p className="field-error" role="alert">{imageError}</p>}
60      </div>
61    </AppForm>
62  }
63
64  export default function CreateCharacter({ characters, onSave }) {
65    const { id } = useParams()
66    const { state } = useLocation()
67    const [draft, setDraft] = useState(() => ({ ...blankCharacter(), ...state?.draft, id: undefined }))
68    const [revision, setRevision] = useState(0)
69    const initial = id ? characters.find(character => character.id === id) : draft
70    if (!initial) return <NotFound />
71
72    return <section className="editor-page">
73      {!id && <button type="button" className="secondary-button random-button" onClick={() => { setDraft({
    ...randomCharacter(), id: undefined }); setRevision(value => value + 1) }}>Generar personaje
    aleatorio</button>}
74      <CharacterEditor key={`${id || 'new'}-${revision}`} initial={initial} onSave={onSave} editing=
    {Boolean(id)} />
75    </section>
76  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 07 DE 62

Login.jsx

`src/pages/Login.jsx`

Formulario de acceso que delega la autenticación en la API.

Entradas / props. onLogin: callback asíncrono de App.

Estado. showPassword, errors, message y pending.

- Valida correo y contraseña, evita envíos repetidos y permite mostrar u ocultar la contraseña.
- handleSubmit entrega email, password y remember a onLogin; el servicio HTTP solo serializa email y password.
- Recordarme no cambia actualmente la duración de la sesión. El texto de cuenta local sigue pendiente de actualizar en la interfaz.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Login`	Presenta el formulario de acceso.	12
`handleSubmit`	Valida, espera onLogin y navega a Perfil o muestra un error.	19

Código completo

90 líneas · SHA-256 del original: `cf0768a1da6013d2d61e81398a606491589ef07d03658fef6b4e72b62dd55d7d`

```
1  import { useState } from 'react'
2  import { Link, useNavigate } from 'react-router'
3  import Icon from '../components/icon/Icon.jsx'
4  import { validateForm } from '../utils/validateForm.js'
5  import './Login.css'
6
7  const fields = [
8    { name: 'email', label: 'Correo electrónico', type: 'email', required: true },
9    { name: 'password', label: 'Contraseña', type: 'password', required: true },
10 ]
11
12 export default function Login({ onLogin }) {
13   const navigate = useNavigate()
14   const [showPassword, setShowPassword] = useState(false)
15   const [errors, setErrors] = useState({})
16   const [message, setMessage] = useState('')
17   const [pending, setPending] = useState(false)
18
```

```
  async function handleSubmit(event) {
    event.preventDefault()
    if (pending) return
    const form = event.currentTarget
    const nextErrors = validateForm(fields, new FormData(form))
    setErrors(nextErrors)
    setMessage('')
    const firstError = Object.keys(nextErrors)[0]
    if (firstError) {
      form.elements.namedItem(firstError).focus()
      return
    }
    setPending(true)
    try {
      const values = new FormData(form)
      await onLogin({ email: values.get('email'), password: values.get('password'), remember: values.has('remember') })
      navigate('/perfil', { replace: true })
    } catch (error) {
      setMessage(error.message || 'No se pudo acceder. Comprueba el almacenamiento del navegador.')
    } finally { setPending(false) }
  }

  return <section className="login-page" aria-labelledby="login-title">
    <div className="login-intro">
      <Icon name="sigil" size={60} />
      <h1 id="login-title">Cruza el umbral</h1>
      <p>Tus historias te esperan en la oscuridad.</p>
    </div>

    <div className="login-divider" aria-hidden="true"><span>❖</span></div>

    <p className="form-hint" id="login-notice">Accede a tu cuenta local de este navegador. Los campos con * son obligatorios.</p>
    <form className="login-form" noValidate aria-describedby="login-notice" onSubmit={handleSubmit}>
      <div className="login-field">
        <label htmlFor="login-email">Correo electrónico *</label>
        <div className="login-input">
          <Icon name="mail" size={18} />
          <input id="login-email" name="email" type="email" autoComplete="email" required maxLength={254}
            placeholder="Tu correo electrónico" aria-invalid={Boolean(errors.email)} aria-describedby={errors.email ? 'login-email-error' : undefined} />
        </div>
        {errors.email && <p className="field-error" id="login-email-error" role="alert">{errors.email}</p>}
      </div>

      <div className="login-field">
        <label htmlFor="login-password">Contraseña *</label>
        <div className="login-input">
          <Icon name="lock" size={18} />
          <input id="login-password" name="password" type={showPassword ? 'text' : 'password'} autoComplete="current-password" required
            placeholder="Introduce tu contraseña" aria-invalid={Boolean(errors.password)} aria-describedby={errors.password ? 'login-password-error' : undefined} />
          <button type="button" className="login-password-toggle" aria-label={showPassword ? 'Ocultar contraseña' : 'Mostrar contraseña'}
            aria-pressed={showPassword} onClick={() => setShowPassword(!showPassword)}>
            <Icon name={showPassword ? 'eye' : 'eye-off'} size={18} />
```

```
71            </button>
72          </div>
73          {errors.password && <p className="field-error" id="login-password-error" role="alert">
    {errors.password}</p>}
74        </div>
75
76        <div className="login-options">
77          <label className="login-remember"><input name="remember" type="checkbox" />
    <span>Recordarme</span></label>
78          <Link className="login-recovery" to="/recuperar">¿Olvidaste tu contraseña?</Link>
79        </div>
80
81        <button className="primary-button login-submit" type="submit" disabled={pending}><Icon name="login"
    size={18} />{pending ? 'Entrando…' : 'Iniciar sesión'}</button>
82        {message && <p className="feedback error" role="alert">{message}</p>}
83      </form>
84
85      <div className="login-divider login-account-divider"><span>¿Aún no tienes una cuenta?</span></div>
86      <Link className="login-create-account" to="/registro"><Icon name="user-plus" size={18} />Crear
    cuenta</Link>
87      <p className="login-terms"><Link to="/privacidad">Privacidad y condiciones de uso</Link></p>
88      <p className="login-motto">Cada alma tiene una historia.</p>
89    </section>
90  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 08 DE 62

Auth.jsx

`src/pages/Auth.jsx`

Registro mediante API y recuperación local antigua en un formulario compartido.

Entradas / props. recovery: false para registro y true para recuperación.

Estado. registrado y recovered controlan las confirmaciones.

- Registro: nombre entre 2 y 50 caracteres, email hasta 100, contraseña entre 8 y 72 con carácter especial y confirmación.
- submit llama registrarUsuario, que transforma name en nombreUsuario. Al terminar muestra éxito y enlace a Login; no genera un código de recuperación.
- La rama recovery sigue usando recoverAccount y solo restablece cuentas locales antiguas. No modifica contraseñas del backend.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Auth`	Elige campos y muestra confirmación de registro o recuperación.	39
`submit`	Envía el registro al servidor o ejecuta la recuperación local antigua.	69

Código completo

117 líneas · SHA-256 del original: `fd93435ae44ca3f70f96020dd5e87bfb6d8506c24b90665aaee7d4619ca36c46`

```
1  import { useState } from 'react'
2  import { Link } from 'react-router'
3  import AppForm from '../components/forms/AppForm.jsx'
4  import { recoverAccount } from '../utils/localAccounts.js'
5  import { registrarUsuario } from '../services/api.js'
6
7
8  const email = {
9    name: 'email',
10   label: 'Correo electrónico',
11   type: 'email',
12   required: true,
13   autoComplete: 'email',
14   maxLength: 100
15 }
16
17 const password = {
```

```
  name: 'password',
  label: 'Contraseña',
  type: 'password',
  required: true,
  minLength: 8,
  maxLength: 72,
  requireSpecial: true,
  autoComplete: 'new-password',
  hint: 'Al menos 8 caracteres y un carácter especial.'
}

const confirmation = {
  name: 'confirmPassword',
  label: 'Repetir contraseña',
  type: 'password',
  required: true,
  matches: 'password',
  autoComplete: 'new-password'
}


export default function Auth({ recovery = false }) {
  const [recovered, setRecovered] = useState(false)
  const [registrado, setRegistrado] = useState(false)
  const fields = recovery
    ? [
      email,
      {
        name: 'recoveryCode',
        label: 'Código de recuperación',
        required: true,
        maxLength: 32
      },
      password,
      confirmation
    ]
    : [
      {
        name: 'name',
        label: 'Nombre',
        required: true,
        minLength: 2,
        maxLength: 50,
        autoComplete: 'name'
      },
      email,
      password,
      confirmation
    ]


  async function submit(values) {
    const data = Object.fromEntries(values)

    if (recovery) {
      await recoverAccount(data)
      setRecovered(true)
    } else {
      await registrarUsuario(data)
```

```
77        setRegistrado(true)
78      }
79    }
80
81    return <section className="auth-page">
82      {registrado ? (
83        <div className="action-stack">
84          <p className="feedback success" role="status">
85            Tu cuenta se ha creado correctamente.
86          </p>
87          <Link className="primary-button" to="/login">
88            Iniciar sesión
89          </Link>
90        </div>
91      ) : recovered ? (
92        <div className="action-stack">
93          <p className="feedback success" role="status">
94            Contraseña local actualizada.
95          </p>
96          <Link className="primary-button" to="/login">
97            Iniciar sesión
98          </Link>
99        </div>
100     ) : (
101       <AppForm
102         fields={fields}
103         onSubmit={submit}
104         submitLabel={recovery ? 'Cambiar contraseña' : 'Crear cuenta'}
105         notice="Introduce tus datos para crear una cuenta de UMBRA."
106       />
107     )}
108
109
110     {!registrado && !recovered && (
111       <p className="account-link">
112         ¿Ya tienes cuenta? <Link to="/login">Iniciar sesión</Link>
113       </p>
114     )}
115   </section>
116 }
117
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 09 DE 62

Profile.jsx

`src/pages/Profile.jsx`

Muestra la identidad activa, sus recuentos, el cierre de sesión y los controles de copia de seguridad.

Entradas / props. user, characters, onLogout y onImport. onImport está conectado a replaceCharacters en App.

Estado. backup conserva temporalmente la copia validada; message y error dan feedback.

- Exportar prepara un JSON con la biblioteca actual. Importar lee un archivo de hasta 10 MB, lo valida y abre un diálogo antes de sustituir datos.
- La restauración reemplaza toda la biblioteca del perfil activo, incluidos sus favoritos y retratos; no mezcla listas ni afecta a otras cuentas.
- Las copias no incluyen cuentas, contraseñas ni ajustes de accesibilidad. La pantalla incluye enlaces a accesibilidad y privacidad.
- El botón Cerrar sesión espera onLogout y muestra un error si falla. El enlace de creación para Invitado conserva la etiqueta antigua Crear cuenta local, aunque el registro usa la API.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Profile`	Presenta la información del perfil y los controles de biblioteca y copia.	7
`readBackup`	Lee el archivo elegido, revisa su tamaño y utiliza importCharacters para validarlo.	12
`restore`	Llama onImport con la copia validada y muestra el resultado; los errores no se anuncian como éxito.	24

Código completo

56 líneas · SHA-256 del original: `1ce1d2e196b649b3024c1b312e40ab857ecc727b47b7ea40276613cdfc0c2594`

```
1  import { useState } from 'react'
2  import { Link } from 'react-router'
3  import Icon from '../components/icon/Icon.jsx'
4  import Modal from '../components/modal/Modal.jsx'
5  import { downloadCharacters, importCharacters } from '../utils/characterStore.js'
6
```

```
export default function Profile({ user, characters, onLogout, onImport }) {
  const [backup, setBackup] = useState(null)
  const [message, setMessage] = useState('')
  const [error, setError] = useState('')

  async function readBackup(event) {
    const file = event.target.files[0]
    event.target.value = ''
    if (!file) return
    setError('')
    setMessage('')
    try {
      if (file.size > 10 * 1024 * 1024) throw new Error('La copia no puede superar los 10 MB.')
      setBackup(importCharacters(await file.text()))
    } catch (failure) { setError(failure.message) }
  }

  function restore() {
    try {
      onImport(backup)
      setBackup(null)
      setError('')
      setMessage('Copia restaurada. Tu biblioteca ya está actualizada.')
    } catch (failure) { setBackup(null); setError(failure.message) }
  }

  return <section className="profile-panel">
    <Icon name="user" size={36} />
    <h2>{user ? user.name : 'Exploras como invitado'}</h2>
    <p>{user ? user.email : 'Puedes crear y guardar personajes sin una cuenta. Cada cuenta local y el modo invitado tienen su propia biblioteca.'}</p>
    <div className="profile-stats"><p><strong>{characters.length}</strong> personajes</p><p><strong>{characters.filter(character => character.favorite).length}</strong> favoritos</p></div>
    <Link className="primary-button" to="/biblioteca">Abrir mi biblioteca</Link>
    {user ? <button className="secondary-button" onClick={async () => {
      try {
        await onLogout()
      } catch {
        setError('No se pudo cerrar la sesión. Inténtalo de nuevo.')
      }
    }}>Cerrar sesión</button> : <><Link className="secondary-button" to="/login">Iniciar sesión</Link> <Link className="secondary-button" to="/registro">Crear cuenta local</Link></>}
    <h2>Copia de seguridad</h2>
    <p>Exporta tus personajes, favoritos y retratos para conservarlos o llevarlos a otro navegador. Las cuentas y contraseñas no se incluyen.</p>
    <button className="secondary-button" onClick={() => { try { downloadCharacters(characters); setMessage('Copia de seguridad preparada para descargar.'); setError('') } catch { setError('No se pudo preparar la descarga.') } }}>Exportar biblioteca (JSON)</button>
    <label className="form-field">Importar copia de seguridad<input type="file" accept=".json,application/json" onChange={readBackup} /></label>
    {message && <p className="feedback success" role="status">{message}</p>}
    {error && <p className="feedback error" role="alert">{error}</p>}
    <Link className="inline-link" to="/accesibilidad">Preferencias de accesibilidad</Link>
    <Link className="inline-link" to="/privacidad">Privacidad y uso</Link>
    {backup && <Modal title="¿Restaurar la biblioteca?" onClose={() => setBackup(null)}><div className="action-stack"><p className="page-description">La copia tiene {backup.length} personajes y sustituirá los {characters.length} actuales. Exporta tu biblioteca primero si quieres conservarla.</p><button className="primary-button" onClick={restore}>Sustituir y restaurar</button><button className="secondary-button" onClick={() => setBackup(null)}>Cancelar</button></div></Modal>}
```

```
  </section>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 10 DE 62

Accessibility.jsx

`src/pages/Accessibility.jsx`

Contiene el menú principal de accesibilidad y los cinco submenús, con configuración independiente por perfil.

Entradas / props. preferences, onChange, error y user; useParams aporta el submenú solicitado.

Estado. message confirma cambios. Los valores permanentes viven en usePreferences y no en esta pantalla.

- Texto: cuatro tamaños y deslizador. Contraste: normal, alto e invertido. Lectura: avisos de pantalla, voz voluntaria y velocidad.
- Teclado: foco reforzado y tabla de atajos nativos. Apariencia: tres temas, tres tamaños de elementos, seis acentos y movimiento reducido.
- Choices utiliza radios reales dentro de fieldset y legend para que se puedan recorrer con teclado y lectores de pantalla. Preview permite ver el efecto.
- Restablecer entrega defaultPreferences al perfil activo. Un submenú desconocido muestra NotFound. El modo de contraste prevalece sobre el color del tema sin borrar la elección.

Funciones y elementos principales

Nombre	Qué hace	Línea
`labelFor`	Encuentra el texto visible correspondiente a un valor de configuración.	11
`Choices`	Construye un grupo de opciones con radios, descripciones y marca de selección.	13
`Preview`	Presenta una ficha de texto de ejemplo que hereda los ajustes actuales.	25
`Accessibility`	Elige el menú a mostrar y compone los controles de configuración.	34
`update`	Entrega cambios a onChange y prepara la confirmación para el perfil activo.	41

Código completo

114 líneas · SHA-256 del original: `4e170d4f623bc21880cc444e25c33fc1863a323af2163d87fb929dba4836a285`

```
1  import { useId, useState } from 'react'
```

```
import { Link, useParams } from 'react-router'
import Icon from '../components/icon/Icon.jsx'
import ReadAloud from '../components/feedback/ReadAloud.jsx'
import { accessibilityMenus, accentColors, contrastModes, elementSizes, textSizes, themes } from '../data/accessibility.js'
import { defaultPreferences } from '../utils/preferenceStore.js'
import { NotFound } from './Information.jsx'
import './Accessibility.css'

const sample = 'Elara Valen, hechicera humana. Una guerrera marcada por la pérdida que busca una segunda oportunidad entre las sombras. Toda historia merece ser contada.'
const labelFor = (options, value) => options.find(option => option.value === value)?.label

function Choices({ name, legend, value, options, onChange }) {
  const id = useId()
  return <fieldset className="settings-choices"><legend>{legend}</legend>
    {options.map(option => <label className={`choice-card${value === option.value ? ' is-selected' : ''}`} key={option.value}>
      <input id={`${id}-${option.value}`} type="radio" name={name} value={option.value} checked={value === option.value} onChange={() => onChange(option.value)} />
      {option.color && <span className="color-swatch" style={{ backgroundColor: option.color }} aria-hidden="true" />}
      <span><strong>{option.label}</strong>{option.description && <small>{option.description}</small>}</span>
      {value === option.value && <Icon name="check" size={18} />}
    </label>)}
  </fieldset>
}

function Preview() {
  return <section className="settings-preview" aria-label="Vista previa de los ajustes">
    <p className="eyebrow">VISTA PREVIA</p><h2>Elara Valen</h2>
    <p className="preview-tags">Humana · Hechicera</p>
    <p>Una guerrera marcada por la pérdida que busca una segunda oportunidad entre las sombras.</p>
    <p>Toda historia merece ser contada.</p>
  </section>
}

export default function Accessibility({ preferences, onChange, error, user }) {
  const { section } = useParams()
  const [message, setMessage] = useState('')
  const profile = user ? user.name : 'Invitado'
  const menu = accessibilityMenus.find(item => item.id === section)
  if (section && !menu) return <NotFound />

  function update(changes) {
    onChange(changes)
    setMessage(`Ajustes aplicados al perfil ${profile}.`)
  }

  const summaries = {
    texto: labelFor(textSizes, preferences.textSize), contraste: labelFor(contrastModes, preferences.contrast),
    lectura: preferences.readAloud ? 'Lectura en voz alta activada' : 'Lector del dispositivo',
    teclado: preferences.enhancedFocus ? 'Foco reforzado' : 'Foco estándar',
    apariencia: `${labelFor(themes, preferences.theme)} · ${labelFor(elementSizes, preferences.elementSize)}`,
  }

```

```
53   return <section className="accessibility-page">
54     {section && <Link className="settings-back" to="/accesibilidad"><Icon name="back" size={18} />Menú de accesibilidad</Link>}
55     <div className="settings-profile"><Icon name="user" size={20} /><p>Ajustes de <strong>{profile}</strong><small>Cada perfil conserva su propia configuración.</small></p></div>
56     {!section ? <>
57       <p className="settings-description">Adapta Umbra a tu forma de leer y navegar. Elige una opción para abrir su menú.</p>
58       <nav aria-label="Menú de accesibilidad" className="settings-menu">
59         {accessibilityMenus.map(item => <Link className="settings-menu-item" key={item.id} to={`/accesibilidad/${item.id}`}>
60           <span className="settings-icon"><Icon name={item.icon} size={25} /></span>
61           <span><strong>{item.label}</strong><small>{item.description}</small><span className="settings-current">{summaries[item.id]}</span></span>
62           <Icon name="chevron" size={18} />
63         </Link>)}
64       </nav>
65       <button className="secondary-button" type="button" onClick={() => update(defaultPreferences)}>Restablecer ajustes de este perfil</button>
66     </> : <>
67       {section === 'texto' && <>
68         <p className="settings-description">Aumenta o reduce el texto. El cambio se aplica en todas las pantallas, incluidas las fichas y los formularios.</p>
69         <label className="text-size-slider">Tamaño de texto: {labelFor(textSizes, preferences.textSize)} <span><span aria-hidden="true">A</span><input type="range" name="textSizeSlider" min="0" max="3" step="1" value={textSizes.findIndex(item => item.value === preferences.textSize)} aria-valuetext={labelFor(textSizes, preferences.textSize)} onChange={event => update({ textSize: textSizes[Number(event.target.value)].value })} /><span aria-hidden="true">A</span></span></label>
70         <Choices name="textSize" legend="Elige un tamaño" value={preferences.textSize} options={textSizes} onChange={textSize => update({ textSize })} />
71         <Preview />
72       </>}
73       {section === 'contraste' && <>
74         <p className="settings-description">Elige cómo distinguir el texto, los bordes y los controles. Las imágenes conservan sus colores.</p>
75         <Choices name="contrast" legend="Modo de contraste" value={preferences.contrast} options={contrastModes} onChange={contrast => update({ contrast })} />
76         <Preview />
77         <p className="form-hint">El contraste normal utiliza el tema y el acento elegidos en Personalización visual.</p>
78       </>}
79       {section === 'lectura' && <>
80         <p className="settings-description">Puedes usar el lector de pantalla de tu dispositivo, como TalkBack, VoiceOver o NVDA. Los controles tienen etiquetas, las imágenes tienen alternativas y los diálogos mantienen el foco.</p>
81         <label className="settings-toggle"><input type="checkbox" name="announceNavigation" checked={preferences.announceNavigation} onChange={event => update({ announceNavigation: event.target.checked })} /><span>Anunciar cambios de pantalla<small>Añade un aviso con el nombre de la pantalla al navegar.</small></span></label>
82         <label className="settings-toggle"><input type="checkbox" name="readAloud" checked={preferences.readAloud} onChange={event => update({ readAloud: event.target.checked })} /><span>Lectura en voz alta<small>Muestra un botón para escuchar el contenido de las fichas.</small></span></label>
83         {preferences.readAloud && <>
84           <label className="form-field">Velocidad de lectura<select name="readingRate" value={preferences.readingRate} onChange={event => update({ readingRate: Number(event.target.value) })}><option value={0.75}>Pausada</option><option value={1}>Normal</option><option value={1.25}>Rápida</option></select></label>
85           <Preview /><ReadAloud text={sample} rate={preferences.readingRate} />
```

```
          <p className="form-hint">La voz depende de las voces disponibles en tu navegador o dispositivo y solo empieza al pulsar el botón.</p>
        </>}
      </>}
      {section === 'teclado' && <>
        <p className="settings-description">Recorre Umbra con el teclado. El foco indica qué control vas a activar.</p>
        <label className="settings-toggle"><input type="checkbox" name="enhancedFocus" checked={preferences.enhancedFocus} onChange={event => update({ enhancedFocus: event.target.checked })} /> <span>Reforzar el indicador de foco<small>Aumenta el grosor y el espacio del contorno al usar el teclado.</small></span></label>
        <table className="keyboard-shortcuts"><caption>Atajos principales</caption><tbody>
          <tr><th scope="row"><kbd>Tab</kbd></th><td>Siguiente control</td></tr>
          <tr><th scope="row"><kbd>Mayús</kbd> + <kbd>Tab</kbd></th><td>Control anterior</td></tr>
          <tr><th scope="row"><kbd>Enter</kbd> / <kbd>Espacio</kbd></th><td>Activar enlaces o botones; marcar opciones</td></tr>
          <tr><th scope="row"><kbd>←</kbd> <kbd>→</kbd></th><td>Cambiar de opción en los grupos; recorrer la barra inferior</td></tr>
          <tr><th scope="row"><kbd>Inicio</kbd> / <kbd>Fin</kbd></th><td>Primer o último enlace de la barra inferior</td></tr>
          <tr><th scope="row"><kbd>Esc</kbd></th><td>Cerrar un menú o diálogo</td></tr>
        </tbody></table>
        <p className="form-hint">Al empezar con Tab aparece «Saltar al contenido». Los lectores de pantalla pueden utilizar también sus propios atajos.</p>
        <Link className="secondary-button" to="/biblioteca">Practicar en la biblioteca</Link>
      </>}
      {section === 'apariencia' && <>
        <Choices name="theme" legend="Tema" value={preferences.theme} options={themes} onChange={theme => update({ theme })} />
        <Choices name="elementSize" legend="Tamaño de elementos" value={preferences.elementSize} options={elementSizes} onChange={elementSize => update({ elementSize })} />
        <Choices name="accent" legend="Color de acento" value={preferences.accent} options={accentColors} onChange={accent => update({ accent })} />
        <label className="settings-toggle"><input type="checkbox" name="reduceMotion" checked={preferences.reduceMotion} onChange={event => update({ reduceMotion: event.target.checked })} /> <span>Reducir movimiento<small>Elimina las transiciones y el movimiento de las tarjetas.</small></span></label>
        {preferences.contrast !== 'normal' && <p className="form-hint">El modo de contraste tiene prioridad sobre los colores. Tu tema y acento se conservan para cuando vuelvas al contraste normal.</p>}
        <Preview />
      </>}
    </>}
    <p className={`feedback${error ? ' error' : ''}`} role="status">{error || message || `Los ajustes de ${profile} se guardan automáticamente en este navegador.`}</p>
  </section>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 11 DE 62

Information.jsx

`src/pages/Information.jsx`

Agrupa Inspiración, Ayuda, Privacidad y la pantalla de ruta inexistente.

Estado. Inspiration mantiene el índice de la idea narrativa y un personaje aleatorio.

- Las ideas narrativas se encuentran en el array prompts. El enlace a crear pasa un borrador mediante el estado de navegación; todavía no lo guarda.
- Ayuda utiliza details y summary para preguntas desplegables. Privacidad explica la persistencia local. NotFound permite volver a Inicio o Ayuda.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Inspiration`	Muestra una idea y una semilla de personaje para comenzar una ficha.	—
`generate`	Escoge una idea distinta de la actual y vuelve a generar el personaje.	—
`Help`	Presenta preguntas frecuentes y accesos a ayuda de accesibilidad.	3
`Privacy`	Explica qué datos se conservan y qué contiene la copia de personajes.	18
`NotFound`	Presenta el estado 404 para rutas o fichas que no se encuentran.	29

Código completo

37 líneas · SHA-256 del original: `34c021139a3f52fbf63a96c592d2a533366ad042c82b425d46c2b275e86e0afb`

```
1  import { Link } from 'react-router'
2
3  export function Help() {
4    return <section className="information-page">
5      <p>Umbra es un espacio para imaginar personajes, guardar sus fichas y dar vida a sus historias.</p>
6      <details><summary>¿Cómo navego por Umbra?</summary><p>Usa el menú superior para acceder a todas las secciones y la barra inferior para Inicio, Biblioteca, Inspiración y Perfil.</p></details>
7      <details><summary>¿Necesito iniciar sesión?</summary><p>Puedes crear y guardar personajes como invitado. Si creas una cuenta local, tendrás una biblioteca independiente en este navegador. Guarda el código de recuperación que aparece al registrarte.</p></details>
8      <details><summary>¿Dónde se guardan mis personajes?</summary><p>En el almacenamiento de este navegador. No se sincronizan entre dispositivos. Desde Perfil puedes exportar una copia JSON y restaurarla en otro navegador o cuenta. Borrar los datos del sitio elimina las cuentas y las bibliotecas locales.</p></details>
```

```
    <details><summary>¿Cómo edito o elimino una ficha?</summary><p>Abre un personaje desde Inicio o Biblioteca. Su ficha tiene una pantalla propia con desplazamiento normal y un enlace para volver. Puedes marcarlo como favorito, editarlo, duplicarlo o eliminarlo. La confirmación para eliminar aparece dentro de la misma pantalla.</p></details>
    <details><summary>¿Qué ocurre si pierdo la conexión?</summary><p>Se muestra un aviso y puedes seguir trabajando mientras la app esté abierta. Para volver a cargarla necesitas conexión al sitio o tener el servidor local en marcha.</p></details>
    <details><summary>¿Cómo recupero mi contraseña?</summary><p>Abre «¿Olvidaste tu contraseña?» e introduce tu correo y el código que guardaste al crear la cuenta. La recuperación es local y no envía correos. Si borraste los datos del navegador, necesitarás crear otra cuenta e importar tu copia de personajes.</p></details>
    <details><summary>¿Cómo mejoro la legibilidad?</summary><p>En Accesibilidad puedes aumentar el contraste y el tamaño del texto. La interfaz también respeta la preferencia del sistema de reducir el movimiento.</p></details>
    <Link className="secondary-button" to="/accesibilidad">Abrir Accesibilidad</Link>
    <Link className="inline-link" to="/inicio">Volver a Inicio</Link>
  </section>
}

export function Privacy() {
  return <section className="information-page">
    <h2>Datos en tu navegador</h2>
    <p>UMBRA guarda personajes, retratos, favoritos, preferencias y cuentas en el almacenamiento local del navegador. No envía estos datos a un servidor ni los sincroniza entre dispositivos.</p>
    <p>Las cuentas locales permiten organizar bibliotecas independientes en este navegador. El código que recibes al registrarte sirve para recuperar el acceso si olvidas tu contraseña. No se envían correos y las copias de seguridad no incluyen las credenciales.</p>
    <h2>Conservar tus historias</h2>
    <p>Exporta una copia desde Perfil antes de borrar los datos del sitio o cambiar de navegador. La copia incluye las fichas y sus retratos, sin cuentas ni contraseñas. Usa solo imágenes que tengas permiso para utilizar.</p>
    <Link className="secondary-button" to="/perfil">Ir a Perfil</Link>
  </section>
}

export function NotFound() {
  return <section className="information-page error-page">
    <p className="error-code" aria-hidden="true">404</p>
    <h2>Este camino no lleva a ninguna historia</h2>
    <p>La dirección no existe o ha cambiado. Puedes volver a Inicio o consultar Ayuda para continuar.</p>
    <Link className="primary-button" to="/inicio">Volver a Inicio</Link>
    <Link className="secondary-button" to="/ayuda">Consultar Ayuda</Link>
  </section>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 12 DE 62

Header.jsx

`src/components/header/Header.jsx`

Cabecera compartida con marca, menú de secciones y acceso a avisos o ayuda.

Entradas / props. onNotify abre los avisos; login adapta la cabecera del acceso; user permite ocultar Iniciar sesión cuando ya hay sesión.

Estado. menuOpen indica si el menú está abierto. Las refs identifican la cabecera y su botón.

- Cierra el menú al elegir una sección, pulsar su botón de cierre, usar Escape o pulsar fuera de la cabecera.
- aria-expanded y aria-controls describen el desplegable. El listener pointerdown se añade solo con el menú abierto y se limpia después.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Header`	Dibuja la cabecera y el menú de navegación.	6
`closeOutside`	Detecta un clic o toque fuera y cierra el menú.	13
`closeMenu`	Cierra el menú y devuelve el foco al botón que lo abre.	20

Código completo

45 líneas · SHA-256 del original: `bc63b3517248772123ab66ef9a73d28aa449b2c50fadd77cbf49182acf312fec`

```
1  import { useEffect, useRef, useState } from 'react'
2  import { Link, NavLink } from 'react-router'
3  import Icon from '../icon/Icon.jsx'
4  import { menuItems } from '../../data/navigation.js'
5
6  export default function Header({ onNotify, login = false, user }) {
7    const [menuOpen, setMenuOpen] = useState(false)
8    const header = useRef(null)
9    const menuButton = useRef(null)
10
11   useEffect(() => {
12     if (!menuOpen) return
13     function closeOutside(event) {
14       if (!header.current.contains(event.target)) setMenuOpen(false)
15     }
16     document.addEventListener('pointerdown', closeOutside)
```

```
    return () => document.removeEventListener('pointerdown', closeOutside)
  }, [menuOpen])

  function closeMenu() {
    setMenuOpen(false)
    menuButton.current.focus()
  }

  return (
    <header ref={header} className="header" onKeyDown={event => {
      if (event.key === 'Escape' && menuOpen) closeMenu()
    }}>
      {login ? <Link className="icon-button" to="/inicio" aria-label="Volver al inicio"><Icon name="back" /></Link> : <button ref={menuButton} className="icon-button" aria-label={menuOpen ? 'Cerrar menú' : 'Abrir menú'}
        aria-expanded={menuOpen} aria-controls="header-menu" onClick={() => setMenuOpen(!menuOpen)}>
        <Icon name="menu" />
      </button>}
      <Link className="brand" to="/inicio" aria-label="Umbra, inicio" onClick={() => setMenuOpen(false)}>
        <Icon name="crest" size={27} /><span>UMBRA</span>
      </Link>
      {login ? <Link className="icon-button" to="/ayuda" aria-label="Ayuda y soporte"><Icon name="help" /></Link> : <button className="icon-button notification-trigger" aria-label="Notificaciones" onClick={onNotify}>
        <Icon name="bell" />
      </button>}
      {!login && menuOpen && <nav id="header-menu" className="header-popover" aria-label="Menú de secciones">
        {menuItems.filter(item => !user || item.path !== '/login').map(item => <NavLink key={item.path} to={item.path} onClick={() => setMenuOpen(false)}>{item.label}</NavLink>)}
        <button onClick={closeMenu}>Cerrar menú</button>
      </nav>}
    </header>
  )
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 13 DE 62

BottomNavigation.jsx

`src/components/bottomnavigation/BottomNavigation.jsx`

Barra inferior para Inicio, Biblioteca, Inspiración y Perfil.

- Lee mainTabs de navigation.js. NavLink identifica la ruta activa.
- Las flechas izquierda/derecha mueven el foco entre enlaces de forma circular; Inicio y Fin van al primero y al último. Enter activa el enlace enfocado.

Funciones y elementos principales

Nombre	Qué hace	Línea
`BottomNavigation`	Construye los cuatro enlaces con icono y texto.	5
`handleKeys`	Calcula el enlace que debe recibir el foco según la tecla pulsada.	6

Código completo

24 líneas · SHA-256 del original: `2e6b222d69ace0e5eab439918671e1652cea516540324a4d8087656a898fc8ae`

```
1  import { NavLink } from 'react-router'
2  import Icon from '../icon/Icon.jsx'
3  import { mainTabs } from '../../data/navigation.js'
4
5  export default function BottomNavigation() {
6    function handleKeys(event) {
7      if (!['ArrowLeft', 'ArrowRight', 'Home', 'End'].includes(event.key)) return
8      const links = [...event.currentTarget.querySelectorAll('a')]
9      const index = links.indexOf(document.activeElement)
10     const next = event.key === 'Home' ? 0 : event.key === 'End' ? links.length - 1
11       : (index + (event.key === 'ArrowRight' ? 1 : -1) + links.length) % links.length
12     event.preventDefault()
13     links[next].focus()
14   }
15
16   return (
17     <nav className="bottom-navigation" aria-label="Navegación principal" onKeyDown={handleKeys}>
18       {mainTabs.map(tab => <NavLink key={tab.path} to={tab.path}
19         className={({ isActive }) => isActive ? 'nav-item active' : 'nav-item'}>
20         <Icon name={tab.icon} /><span className="nav-label">{tab.label}</span>
21       </NavLink>)}
22     </nav>
23   )
24 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 14 DE 62

Hero.jsx

`src/components/hero/Hero.jsx`

Portada de Inicio con paisaje, mensaje de bienvenida y acceso al editor.

- El paisaje tiene un nombre accesible. Crear personaje es un Link a /crear-personaje, de modo que no necesita guardar estado ni recibir callbacks.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Hero`	Renderiza la portada y el acceso a crear una ficha.	3

Código completo

13 líneas · SHA-256 del original: `122bddc95a161c7f53e81d9faac748ef8533d6e2a76129a0705eb8c661a610c1`

```
1  import { Link } from 'react-router'
2
3  export default function Hero() {
4    return <section className="hero" aria-labelledby="hero-title">
5      <div className="hero-landscape" role="img" aria-label="Castillo entre montañas y niebla" />
6      <div className="hero-copy">
7        <p className="welcome">Bienvenido a Umbra</p>
8        <h1 id="hero-title">Que tus historias<br />nunca se apaguen.</h1>
9        <p className="hero-description">Aquí comienza tu leyenda.</p>
10       <Link className="primary-button" to="/crear-personaje"><span aria-hidden="true">+</span> Crear
   personaje</Link>
11     </div>
12   </section>
13 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 15 DE 62

CharacterCard.jsx

`src/components/charactercard/CharacterCard.jsx`

Tarjeta reutilizable para fichas locales y públicas.

Entradas / props. character e interactive (true por defecto).

- Por defecto crea un Link a /biblioteca/:id y guarda la ruta y filtros de retorno en location.state.
- Con interactive=false devuelve un article sin enlace interno, para envolverlo en el enlace al detalle público sin anidar enlaces.
- Solo muestra nivel si level no es null ni undefined. Comparte Portrait y el indicador de favorito.

Funciones y elementos principales

Nombre	Qué hace	Línea
`CharacterCard`	Construye el contenido de tarjeta y decide si incluye enlace.	4

Código completo

41 líneas · SHA-256 del original: `890463fb92fa7923aeeca4cb961a792e842408a2cd61b05e62343d0d13c03e4b`

```
1  import { Link, useLocation } from 'react-router'
2  import Portrait from './Portrait.jsx'
3
4  export default function CharacterCard({ character, interactive = true }) {
5    const { pathname, search } = useLocation()
6
7    const contenido = (
8      <>
9        <Portrait character={character} />
10       <div className="character-info">
11         <h3>{character.name}</h3>
12         <p>
13           {character.type}
14           {character.level != null && ` · Nivel ${character.level}`}
15         </p>
16         {character.favorite && (
17           <span className="favorite-label">★ Favorito</span>
18         )}
19       </div>
20     </>
21   )
22
23   if (!interactive) {
24     return (
```

```
      <article className="character-card">
        {contenido}
      </article>
    )
  }

  return (
    <Link
      className="character-card"
      to={`/biblioteca/${encodeURIComponent(character.id)}`}
      state={{ returnTo: `${pathname}${search}` }}
      aria-label={`Ver ficha de ${character.name}, ${character.type}, nivel ${character.level}`}
    >
      {contenido}
    </Link>
  )
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 16 DE 62

Portrait.jsx

`src/components/charactercard/Portrait.jsx`

Retrato con prioridades de imagen, encuadre configurable e iniciales de respaldo.

Entradas / props. character y atributos adicionales.

Estado. failedImage guarda la imagen que produjo un error.

- Elige character.imageDataUrl, después character.image y después uno de los dos PNG incluidos según portrait.
- failedImage registra una URL que no cargó; si no hay imagen o coincide con la fallida se muestran hasta dos iniciales.
- objectPosition usa character.imagePosition o center. Los atributos adicionales se transmiten al elemento renderizado.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Portrait`	Presenta imagen o iniciales con texto accesible.	3

Código completo

26 líneas · SHA-256 del original: `fb13bc4be4580e567430468de02d95b4aa232ed266d8850e6268ac8e976f0668`

```
import { useState } from 'react'

export default function Portrait({ character, ...props }) {
  const [failedImage, setFailedImage] = useState('')
  const portraits = ['/images/elara-valen.png', '/images/kael-grim.png']
  const image = character.imageDataUrl || character.image || portraits[character.portrait]

  if (!image || failedImage === image) {
    return <div className="character-portrait portrait-placeholder" role="img" aria-label={`Iniciales de ${character.name}`} {...props}>
      <span aria-hidden="true">{character.name.split(' ').map(word => word[0]).slice(0, 2).join('')}</span>
    </div>
  }

  return (
    <img
      className="character-portrait"
      src={image}
      onError={() => setFailedImage(image)}
```

```
      alt={`Retrato de ${character.name}`}
      width="400"
      height="400"
      style={{ objectPosition: character.imagePosition || 'center' }}
      {...props}
    />
  )
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 17 DE 62

Modal.jsx

`src/components/modal/Modal.jsx`

Ventana modal reutilizable basada en el elemento nativo dialog.

Entradas / props. title, onClose y children. useId relaciona el título con aria-labelledby.

- showModal coloca el diálogo en la capa superior y activa el comportamiento modal del navegador.
- Puede cerrarse con el botón, Escape o un clic en el fondo. Al desmontarse cierra el diálogo y devuelve el foco anterior si el elemento sigue existiendo.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Modal`	Abre y presenta un diálogo accesible, ejecuta la limpieza y comunica su cierre.	4

Código completo

29 líneas · SHA-256 del original: `16f7bc8b7d3092477abecf3e47eb5926dec44bd540964f20ec9aea17eac39d94`

```
1  import { useEffect, useId, useRef } from 'react'
2  import Icon from '../icon/Icon.jsx'
3
4  export default function Modal({ title, onClose, children }) {
5    const dialog = useRef(null)
6    const titleId = useId()
7
8    useEffect(() => {
9      const element = dialog.current
10     const previousFocus = document.activeElement
11     element.showModal()
12     return () => {
13       element.close()
14       if (previousFocus?.isConnected) previousFocus.focus({ preventScroll: true })
15     }
16   }, [])
17
18   return (
19     <dialog ref={dialog} className="umbra-dialog" aria-labelledby={titleId}
20       onClose={() => { if (!dialog.current.open) onClose() }}
21       onClick={event => { if (event.target === event.currentTarget) event.currentTarget.close() }}>
22       <div className="dialog-content">
23         <button className="icon-button dialog-close" aria-label="Cerrar diálogo" onClick={() =>
   dialog.current.close()}><Icon name="close" /></button>
24         <h2 id={titleId}>{title}</h2>
```

```
25          {children}
26        </div>
27      </dialog>
28    )
29  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 18 DE 62

AppForm.jsx

`src/components/forms/AppForm.jsx`

Motor de formularios reutilizable para personajes, registro y recuperación.

Entradas / props. fields describe los campos; onSubmit ejecuta el guardado; submitLabel, successMessage, children, cancelTo y notice personalizan el formulario.

Estado. errors, success, dirty, confirmCancel, failure y pending controlan validación, confirmaciones y envío.

- Genera inputs, selects, textareas y casillas según la definición recibida. Asocia etiquetas, ayudas y errores mediante identificadores únicos.
- La validación es propia: noValidate evita mensajes distintos entre navegadores. El primer campo inválido recibe foco después de actualizar React.
- Mientras espera el envío desactiva los campos y el botón. La cancelación confirma el descarte si hubo cambios detectados dentro del formulario.

Funciones y elementos principales

Nombre	Qué hace	Línea
`AppForm`	Genera los controles y las zonas de feedback y cancelación.	6
`handleSubmit`	Valida FormData, impide envíos repetidos, espera onSubmit y gestiona éxito o error.	16

Código completo

81 líneas · SHA-256 del original: `4a8ae6f5bc75be07c4937e00dc05883937ce1be83b04afbe677e730131fef18e`

```
1  import { useId, useState } from 'react'
2  import { useNavigate } from 'react-router'
3  import Modal from '../modal/Modal.jsx'
4  import { validateForm } from '../../utils/validateForm.js'
5
6  export default function AppForm({ fields, submitLabel, successMessage, onSubmit, children, cancelTo =
   '/inicio', notice }) {
7    const id = useId()
8    const navigate = useNavigate()
9    const [errors, setErrors] = useState({})
10   const [success, setSuccess] = useState('')
11   const [dirty, setDirty] = useState(false)
12   const [confirmCancel, setConfirmCancel] = useState(false)
```

```
  const [failure, setFailure] = useState('')
  const [pending, setPending] = useState(false)

  async function handleSubmit(event) {
    event.preventDefault()
    if (pending) return
    const form = event.currentTarget
    const nextErrors = validateForm(fields, new FormData(form))
    setErrors(nextErrors)
    setSuccess('')
    setFailure('')
    if (Object.keys(nextErrors).length) {
      requestAnimationFrame(() => form.querySelector('[aria-invalid="true"]')?.focus())
      return
    }
    setPending(true)
    try {
      await onSubmit(new FormData(form))
      setDirty(false)
      setSuccess(successMessage || '')
    } catch (error) {
      setFailure(error.message || 'No se pudo completar la operación. Vuelve a intentarlo.')
    } finally {
      setPending(false)
    }
  }

  return <>
    {notice && <p className="form-notice" id={`${id}-notice`}>{notice}</p>}
    <form className="character-form" noValidate aria-describedby={notice ? `${id}-notice` : undefined}
onSubmit={handleSubmit}
      onChange={() => { setDirty(true); setSuccess('') }}>
      <p className="form-hint">Los campos marcados con * son obligatorios.</p>
      <fieldset className="form-contents" disabled={pending}>
      {Object.keys(errors).length > 0 && <p className="feedback error" role="alert">Revisa los campos
indicados antes de continuar.</p>}
      {fields.map(field => {
        const inputId = `${id}-${field.name}`
        const props = {
          id: inputId, name: field.name, required: field.required,
          defaultValue: field.type === 'checkbox' ? undefined : field.defaultValue, minLength:
field.minLength, maxLength: field.maxLength,
          'aria-invalid': Boolean(errors[field.name]),
          'aria-describedby': [field.hint && `${inputId}-hint`, errors[field.name] && `${inputId}-
error`].filter(Boolean).join(' ') || undefined,
        }
        return <div className={field.type === 'checkbox' ? 'form-field checkbox-field' : 'form-field'}
key={field.name}>
          {field.type === 'checkbox' ? <label className="checkbox-label"><input {...props}
type="checkbox" defaultChecked={field.defaultValue === true} />{field.label}{field.required && ' *'}
</label> : <>
            <label htmlFor={inputId}>{field.label}{field.required && ' *'}</label>
            {field.options ? <select {...props}>{field.options.map(option => <option key={option}>
{option}</option>)}</select>
              : field.type === 'textarea' ? <textarea {...props} rows="4" placeholder={field.placeholder}
/>
                : <input {...props} type={field.type || 'text'} autoComplete={field.autoComplete} min=
{field.min} max={field.max} placeholder={field.placeholder} />}
          </>}
```

```
          {field.hint && <p id={`${inputId}-hint`} className="form-hint">{field.hint}</p>}
          {errors[field.name] && <p id={`${inputId}-error`} className="field-error">{errors[field.name]}
</p>}
        </div>
      })}
      {children}
      </fieldset>
      {failure && <p className="feedback error" role="alert">{failure}</p>}
      <p className={success ? 'feedback success' : 'visually-hidden'} role="status" aria-live="polite">
{success}</p>
      <button className="primary-button" type="submit" disabled={pending}>{pending ? 'Guardando…' :
submitLabel}</button>
      <button className="secondary-button" type="button" disabled={pending} onClick={() => dirty ?
setConfirmCancel(true) : navigate(cancelTo)}>Cancelar</button>
    </form>
    {confirmCancel && <Modal title="¿Descartar lo escrito?" onClose={() => setConfirmCancel(false)}>
      <p className="page-description">Los campos introducidos se perderán si sales de este formulario.
</p>
      <div className="action-stack">
        <button className="primary-button" onClick={() => setConfirmCancel(false)}>Seguir
editando</button>
        <button className="secondary-button" onClick={() => navigate(cancelTo)}>Descartar y
salir</button>
      </div>
    </Modal>}
  </>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 19 DE 62

ConnectionStatus.jsx

`src/components/feedback/ConnectionStatus.jsx`

Aviso de pérdida y recuperación de conexión del navegador.

Estado. message conserva el aviso que se anuncia mediante una región de estado.

- Escucha los eventos offline y online y elimina ambos listeners al desmontarse.
- navigator.onLine describe el estado que comunica el navegador, no comprueba la disponibilidad de un backend. No implementa caché ni un service worker.

Funciones y elementos principales

Nombre	Qué hace	Línea
`ConnectionStatus`	Inicia y presenta el estado de conexión.	3
`offline`	Prepara el aviso de falta de conexión.	7
`online`	Prepara el aviso de conexión restablecida.	8

Código completo

18 líneas · SHA-256 del original: `661240da7ee93895c744b93593d0078c7983655d346fa477b5c26a7da245c1bf`

```
1  import { useEffect, useState } from 'react'
2
3  export default function ConnectionStatus() {
4    const [message, setMessage] = useState(() => navigator.onLine ? '' : 'Sin conexión. Puedes seguir trabajando mientras la app esté abierta.')
5
6    useEffect(() => {
7      const offline = () => setMessage('Sin conexión. Puedes seguir trabajando mientras la app esté abierta.')
8      const online = () => setMessage('Conexión restablecida.')
9      window.addEventListener('offline', offline)
10     window.addEventListener('online', online)
11     return () => {
12       window.removeEventListener('offline', offline)
13       window.removeEventListener('online', online)
14     }
15   }, [])
16
17   return <div className={message ? 'connection-status feedback' : 'visually-hidden'} role="status" aria-live="polite" aria-atomic="true">{message}</div>
18 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 20 DE 62

ReadAloud.jsx

`src/components/feedback/ReadAloud.jsx`

Botón para leer texto en español mediante las voces del navegador o dispositivo.

Entradas / props. text contiene lo que se leerá; rate define la velocidad, normalmente 0.75, 1 o 1.25.

Estado. reading y error controlan la interfaz; la ref current conserva la locución en curso.

- Comprueba SpeechSynthesis y SpeechSynthesisUtterance antes de ofrecer el botón. Prefiere una voz cuyo idioma empiece por es.
- La lectura comienza solo por acción de la persona. Al detenerla o desmontarse el componente cancela la locución.
- La reproducción depende de las voces disponibles y puede fallar con un mensaje. Esta función es distinta del lector de pantalla del sistema.

Funciones y elementos principales

Nombre	Qué hace	Línea
`ReadAloud`	Presenta el botón o la explicación de que no hay soporte.	4
`toggle`	Inicia una locución con el texto y velocidad actuales o detiene la que se está leyendo.	18

Código completo

48 líneas · SHA-256 del original: `6b8db696e2080055c48acb39dbaf6904f444089ce5c135a8a09fb4c9c3414069`

```
1  import { useEffect, useRef, useState } from 'react'
2  import Icon from '../icon/Icon.jsx'
3
4  export default function ReadAloud({ text, rate = 1 }) {
5    const supported = 'speechSynthesis' in window && 'SpeechSynthesisUtterance' in window
6    const current = useRef(null)
7    const [reading, setReading] = useState(false)
8    const [error, setError] = useState('')
9
10   useEffect(() => () => {
11     if (current.current) {
12       current.current.onend = null
13       current.current.onerror = null
14       window.speechSynthesis.cancel()
15     }
16   }, [])
```

```
17
18    function toggle() {
19      if (reading) {
20        window.speechSynthesis.cancel()
21        current.current = null
22        setReading(false)
23        return
24      }
25      setError('')
26      window.speechSynthesis.cancel()
27      const speech = new SpeechSynthesisUtterance(text)
28      speech.lang = 'es-ES'
29      speech.rate = rate
30      const voice = window.speechSynthesis.getVoices().find(item => item.lang.startsWith('es'))
31      if (voice) speech.voice = voice
32      speech.onend = () => { current.current = null; setReading(false) }
33      speech.onerror = event => {
34        current.current = null
35        setReading(false)
36        if (!['canceled', 'interrupted'].includes(event.error)) setError('No se pudo reproducir la voz.
    Comprueba que el dispositivo tenga una voz en español disponible.')
37      }
38      current.current = speech
39      setReading(true)
40      window.speechSynthesis.speak(speech)
41    }
42
43    if (!supported) return <p className="form-hint">Este navegador no ofrece lectura en voz alta. Puedes
    utilizar el lector de pantalla de tu dispositivo.</p>
44    return <div className="read-aloud">
45      <button className="secondary-button" type="button" onClick={toggle} aria-pressed={reading}><Icon
    name="speaker" />{reading ? 'Detener lectura' : 'Leer en voz alta'}</button>
46      {error && <p className="feedback error" role="alert">{error}</p>}
47    </div>
48  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 21 DE 62

Icon.jsx

`src/components/icon/Icon.jsx`

Catálogo de iconos SVG reutilizables sin archivos externos.

Entradas / props. name elige el dibujo y size fija el tamaño; por defecto es 22.

- paths contiene los trazados de cada icono. aria-hidden evita que un dibujo decorativo repita el nombre que ya aporta su botón o enlace.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Icon`	Genera un svg con el trazado, tamaño y estilo comunes.	36

Código completo

38 líneas · SHA-256 del original: `1977265b8f47114bc14f3043b7319f93a62c2a909805c4022728bcc36f129ec6`

```
1  const paths = {
2    dice: <><rect x="3" y="3" width="18" height="18" rx="4" /><circle cx="8" cy="8" r="1"
   fill="currentColor" /><circle cx="16" cy="8" r="1" fill="currentColor" /><circle cx="12" cy="12" r="1"
   fill="currentColor" /><circle cx="8" cy="16" r="1" fill="currentColor" /><circle cx="16" cy="16" r="1"
   fill="currentColor" /></>,
3    unlock: <><rect x="5" y="10" width="14" height="11" rx="2" /><path d="M8 10V7a4 4 0 0 1 7.5-2M12 14v3"
   /></>,
4    undo: <><path d="M3 10h11a7 7 0 0 1 0 14M3 10l5-5M3 10l5 5" transform="translate(0 -2)" /></>,
5    mail: <><rect x="3" y="5" width="18" height="14" rx="2" /><path d="m3 6 9 7 9-7" /></>,
6    lock: <><rect x="5" y="10" width="14" height="11" rx="2" /><path d="M8 10V7a4 4 0 0 1 8 0v3M12 14v3" />
   </>,
7    'eye-off': <><path d="m3 3 18 18M10.5 5.1 12 5c6 0 10 7 10 7a20 20 0 0 1-3 3.5M6 6.5A22 22 0 0 0 2 12s4
   7 10 7a11 11 0 0 0 5-1.2M10 10a3 3 0 0 0 4 4" /></>,
8    login: <><path d="M14 3h6v18h-6M3 12h13m-5-5 5 5-5 5" /></>,
9    'user-plus': <><circle cx="9" cy="7" r="4" /><path d="M2 22v-3a7 7 0 0 1 14 0v3M19 8v8M15 12h8" /></>,
10   sigil: <><path d="m12 2 10 5-2.5 11-7.5 4-7.5-4L2 7ZM5 7l7 12 7-12M12 2v10M8 11h8M9.5 15h5" /></>,
11   close: <path d="m6 6 12 12M6 18 18 6" />,
12   back: <path d="m14 5-7 7 7 7" />,
13   chevron: <path d="m9 5 7 7-7 7" />,
14   search: <><circle cx="10" cy="10" r="6" /><path d="m15 15 6 6" /></>,
15   edit: <><path d="m15 4 5 5M4 20l5-1L21 7l-5-5L4 14Z" /></>,
16   trash: <><path d="M3 6h18M9 6V3h6v3M6 6l1 15h10l1-15M10 10v7M14 10v7" /></>,
17   bookmark: <path d="M6 3h12v19l-6-4-6 4Z" />,
18   help: <><circle cx="12" cy="12" r="9" /><path d="M9 9a3 3 0 1 1 4 3c-1 0-1 1-1 2M12 17h.01" /></>,
19   logout: <><path d="M9 3H4v18h5M10 12h12m-5-5 5 5-5 5" /></>,
20   palette: <><path d="M12 3a9 9 0 0 1 0 0 18h1a2 2 0 0 0 0-4h-1a2 2 0 0 1 0-4h4a5 5 0 0 0 5-5c0-3-5-5-9-5Z"
   /><path d="M7 9h.01M11 6h.01M6 14h.01M16 7h.01" /></>,
```

```
21   contrast: <><circle cx="12" cy="12" r="9" /><path d="M12 3v18" /><path d="M12 3a9 9 0 0 0 1 0 18Z"
    fill="currentColor" /></>,
22   type: <path d="m3 20 7-16 7 16M6 14h8M17 12h6M20 12v8" />,
23   speaker: <><path d="m11 4-6 5H2v6h3l6 5ZM15 8a6 6 0 0 1 0 8M18 5a10 10 0 0 1 0 14" /></>,
24   keyboard: <><rect x="2" y="5" width="20" height="14" rx="2" /><path d="M6 9h1m3 0h1m3 0h1m3 0h1M6
    12h1m3 0h1m3 0h1m3 0h1M7 16h10" /></>,
25   check: <path d="m4 12 5 5L20 6" />,
26   eye: <><path d="M2 12s4-7 10-7 10 7 10 7-4 7-10 7S2 12 2 12Z" /><circle cx="12" cy="12" r="3" /></>,
27   menu: <><path d="M5 6h14M5 12h14M5 18h14" /></>,
28   bell: <><path d="M18 8a6 6 0 0 0-12 0c0 7-3 7-3 9h18c0-2-3-2-3-9M10 21h4" /></>,
29   home: <><path d="m3 10 9-7 9 7v11h-6v-7H9v7H3Z" /></>,
30   book: <><path d="M12 5v16M3 3l9 2 9-2v16l-9 2-9-2Z" /></>,
31   sparkle: <><path d="m12 3 2.5 6.5L21 12l-6.5 2.5L12 21l-2.5-6.5L3 12l6.5-2.5ZM20 2v4M18 4h4" /></>,
32   user: <><circle cx="12" cy="7" r="4" /><path d="M4 22v-3a8 8 0 0 1 16 0v3" /></>,
33   crest: <><path d="m12 2 8 5-2 10-6 5-6-5L4 7ZM12 2v20M8 6l1 8 3 3 3-3 1-8" /></>,
34 }
35
36 export default function Icon({ name, size = 22 }) {
37   return <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke="currentColor"
    strokeWidth="1.3" strokeLinecap="round" strokeLinejoin="round" aria-hidden="true">{paths[name]}</svg>
38 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 22 DE 62

useCharacters.js

`src/hooks/useCharacters.js`

Mantiene la biblioteca React del perfil y coordina las escrituras persistentes.

Entradas / props. owner es el identificador estable de la cuenta o guest.

Estado. loaded contiene el resultado inicial; characters es la biblioteca visible; storageError informa de fallos.

- Carga una vez al montarse. App vuelve a montar Workspace cuando cambia el perfil.
- Actualiza el estado visible solo después de guardar correctamente. Si el almacenamiento falla conserva la biblioteca anterior y lanza un error para el formulario o diálogo.

Funciones y elementos principales

Nombre	Qué hace	Línea
`useCharacters`	Devuelve personajes, errores y operaciones de biblioteca.	4
`commit`	Valida la nueva lista, la escribe y actualiza el estado; transforma fallos de almacenamiento en un mensaje.	12
`saveCharacter`	Normaliza una ficha, crea identificador cuando hace falta y actualiza updatedAt. Sustituye por id o añade al inicio.	25
`deleteCharacter`	Filtra la ficha con el id indicado y guarda la nueva lista.	34
`toggleFavorite`	Invierte favorite en la ficha indicada y guarda los datos.	35
`replaceCharacters`	Expone commit para sustituir la biblioteca al restaurar una copia.	36

Código completo

38 líneas · SHA-256 del original: `a885b8a5c772dc0c583fbb3ceaf109b45df7b3b28ec9c0074243925f6b04aa14`

```
1  import { useState } from 'react'
2  import { normalizeCharacter, parseCharacters, readCharacters, writeCharacters } from
   '../utils/characterStore.js'
3
4  export function useCharacters(owner) {
5    const [loaded] = useState(() => {
6      try { return { characters: readCharacters(owner), error: '' } }
```

```
    catch { return { characters: [], error: 'No se pudo leer la biblioteca guardada. Puedes importar una copia desde Perfil. Comprueba que el navegador permita el almacenamiento local.' } }
  })
  const [characters, setCharacters] = useState(loaded.characters)
  const [storageError, setStorageError] = useState(loaded.error)

  function commit(next) {
    parseCharacters(next)
    try {
      const checked = writeCharacters(owner, next)
      setCharacters(checked)
      setStorageError('')
    } catch {
      const message = 'No se pudo guardar. El almacenamiento puede estar lleno o bloqueado. Exporta una copia desde Perfil y prueba con un retrato más pequeño.'
      setStorageError(message)
      throw new Error(message)
    }
  }

  function saveCharacter(draft) {
    const character = normalizeCharacter({ ...draft, id: draft.id || crypto.randomUUID(), updatedAt: Date.now() })
    const exists = characters.some(item => item.id === character.id)
    commit(exists ? characters.map(item => item.id === character.id ? character : item) : [character, ...characters])
    return character
  }

  return {
    characters, storageError, saveCharacter,
    deleteCharacter: id => commit(characters.filter(character => character.id !== id)),
    toggleFavorite: id => commit(characters.map(character => character.id === id ? { ...character, favorite: !character.favorite } : character)),
    replaceCharacters: commit,
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 23 DE 62

usePreferences.js

`src/hooks/usePreferences.js`

Carga y aplica los ajustes de accesibilidad del perfil activo.

Entradas / props. owner es el identificador del perfil o guest.

Estado. preferences conserva la configuración; preferenceError informa de lectura o guardado fallidos.

- Si no puede leer los ajustes utiliza los valores predeterminados y muestra un aviso.
- A diferencia de los personajes, un ajuste de accesibilidad se aplica visualmente incluso si no se puede persistir. Se avisa de que solo durará mientras la app siga abierta.

Funciones y elementos principales

Nombre	Qué hace	Línea
`usePreferences`	Recupera ajustes y devuelve su valor, su error y la operación de cambio.	4
`updatePreferences`	Combina cambios parciales, normaliza, aplica al estado y trata de guardar para owner.	12

Código completo

24 líneas · SHA-256 del original: `473006af0a4bb3363458de9ab2d2b34b1a6357d3fcf7c297af1ddc6caa3a5208`

```
1  import { useState } from 'react'
2  import { defaultPreferences, normalizePreferences, readPreferences, writePreferences } from
   '../utils/preferenceStore.js'
3
4  export function usePreferences(owner) {
5    const [loaded] = useState(() => {
6      try { return { preferences: readPreferences(owner), error: '' } }
7      catch { return { preferences: { ...defaultPreferences }, error: 'No se pudieron recuperar los ajustes
   de este perfil. Puedes volver a elegirlos.' } }
8    })
9    const [preferences, setPreferences] = useState(loaded.preferences)
10   const [preferenceError, setPreferenceError] = useState(loaded.error)
11
12   function updatePreferences(changes) {
13     const updated = normalizePreferences({ ...preferences, ...changes })
14     setPreferences(updated)
15     try {
16       writePreferences(owner, updated)
17       setPreferenceError('')
18     } catch {
```

```
      setPreferenceError('El navegador no permite guardar tus preferencias. Se aplicarán mientras la app esté abierta.')
    }
  }

  return { preferences, updatePreferences, preferenceError }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 24 DE 62

characterStore.js

`src/utils/characterStore.js`

Valida fichas, lee y escribe bibliotecas y prepara las copias JSON.

- MAX_IMAGE_BYTES fija 750 KB para la carga de retratos; imagePattern limita el formato de data URL a PNG, JPEG o WebP.
- readCharacters inicia una biblioteca nueva con seis ejemplos si la clave no existe. Una lista guardada vacía se conserva vacía.
- La importación acepta hasta 500 fichas y una versión de copia reconocida; la pantalla Perfil limita también el archivo a 10 MB.

Funciones y elementos principales

Nombre	Qué hace	Línea
`normalizeCharacter`	Construye una ficha con las propiedades admitidas; comprueba nombre, clase, raza, nivel, atributos, rasgos e imagen. Añade un id nuevo si no recibe uno utilizable.	7
`text`	Ayudante interno que recorta y limita campos de texto.	9
`parseCharacters`	Comprueba que recibe una lista, valida cada ficha y rechaza identificadores repetidos.	35
`readCharacters`	Lee la clave del propietario y devuelve su biblioteca o los ejemplos iniciales.	42
`writeCharacters`	Valida la lista y la serializa en localStorage para el propietario.	47
`importCharacters`	Analiza el JSON, verifica app y version y devuelve las fichas validadas.	53
`downloadCharacters`	Serializa app, version y characters, crea un Blob y un enlace temporal y dispara la descarga; después libera la URL.	60

Código completo

68 líneas · SHA-256 del original: `ccf59fe9fab148b04558021ebbf4e148e02f272ec2b83bdaa779840457b1b510`

```
1  import { initialCharacters } from '../data/characters.js'
2  import { characterRaces, characterTypes } from '../data/characterOptions.js'
```

```
3
4  export const MAX_IMAGE_BYTES = 750 * 1024
5  const imagePattern = /^data:image\/(png|jpeg|webp)base64,[A-Za-z0-9+/]+=*$/
6
7  export function normalizeCharacter(value) {
8    if (!value || typeof value !== 'object') throw new Error('La ficha no es válida.')
9    const text = (key, max, required = false) => {
10     const result = typeof value[key] === 'string' ? value[key].trim() : ''
11     const label = { name: 'nombre', realm: 'origen', story: 'historia', imageDataUrl: 'retrato',
   imageName: 'nombre de imagen' }[key]
12     if ((required && result.length < 2) || result.length > max) throw new Error(`Revisa el campo ${label}
   de la ficha.`)
13     return result
14   }
15   const name = text('name', 60, true)
16   if (!characterTypes.includes(value.type) || !characterRaces.includes(value.race)) throw new Error('La
   clase o la raza no son válidas.')
17   const level = Number(value.level)
18   if (!Number.isInteger(level) || level < 1 || level > 20) throw new Error('El nivel debe estar entre 1 y
   20.')
19   if (!Array.isArray(value.stats) || value.stats.length !== 4 || value.stats.some(stat =>
   !Number.isInteger(stat) || stat < 1 || stat > 20)) throw new Error('Los cuatro atributos deben estar
   entre 1 y 20.')
20   if (!Array.isArray(value.traits) || value.traits.length > 6 || value.traits.some(trait => typeof trait
   !== 'string' || trait.length > 40)) throw new Error('Puedes elegir hasta seis rasgos.')
21   const imageDataUrl = text('imageDataUrl', MAX_IMAGE_BYTES * 1.4)
22   if (imageDataUrl && !imagePattern.test(imageDataUrl)) throw new Error('El retrato debe ser una imagen
   PNG, JPG o WebP.')
23   return {
24     id: typeof value.id === 'string' && /^[\w-]{1,80}$/.test(value.id) ? value.id : crypto.randomUUID(),
25     name, type: value.type, race: value.race, level,
26     realm: text('realm', 100), story: text('story', 3000),
27     portrait: Number.isInteger(value.portrait) && value.portrait >= 0 && value.portrait <= 5 ?
   value.portrait : 0,
28     imageDataUrl, imageName: text('imageName', 255),
29     traits: [...new Set(value.traits)], stats: [...value.stats],
30     favorite: value.favorite === true,
31     updatedAt: Number.isFinite(value.updatedAt) ? value.updatedAt : 0,
32   }
33 }
34
35 export function parseCharacters(values) {
36   if (!Array.isArray(values) || values.length > 500) throw new Error('La copia debe contener una lista de
   hasta 500 personajes.')
37   const characters = values.map(normalizeCharacter)
38   if (new Set(characters.map(character => character.id)).size !== characters.length) throw new Error('La
   copia contiene identificadores repetidos.')
39   return characters
40 }
41
42 export function readCharacters(owner, storage = localStorage) {
43   const saved = storage.getItem(`umbra.characters.v1.${owner}`)
44   return saved === null ? initialCharacters.map(normalizeCharacter) : parseCharacters(JSON.parse(saved))
45 }
46
47 export function writeCharacters(owner, characters, storage = localStorage) {
48   const checked = parseCharacters(characters)
49   storage.setItem(`umbra.characters.v1.${owner}`, JSON.stringify(checked))
50   return checked
```

```
}

export function importCharacters(contents) {
  let backup
  try { backup = JSON.parse(contents) } catch { throw new Error('El archivo no contiene JSON válido.') }
  if (backup?.app !== 'UMBRA' || backup.version !== 1) throw new Error('Selecciona una copia de seguridad de UMBRA compatible.')
  return parseCharacters(backup.characters)
}

export function downloadCharacters(characters) {
  const blob = new Blob([JSON.stringify({ app: 'UMBRA', version: 1, characters }, null, 2)], { type: 'application/json' })
  const url = URL.createObjectURL(blob)
  const link = document.createElement('a')
  link.href = url
  link.download = `umbra-personajes-${new Date().toISOString().slice(0, 10)}.json`
  link.click()
  setTimeout(() => URL.revokeObjectURL(url), 1000)
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 25 DE 62

localAccounts.js

`src/utils/localAccounts.js`

Módulo local anterior; la recuperación todavía lo usa, el acceso y registro actuales usan la API.

- readSession, registerAccount, loginAccount y logoutAccount siguen disponibles y probados, pero App no los utiliza para la sesión actual. recoverAccount permanece conectado a /recuperar.
- La cuenta guarda id, nombre, correo normalizado, sal y verificadores. No persiste la contraseña ni el código de recuperación en texto plano.
- PBKDF2-SHA-256 utiliza una sal aleatoria de 16 bytes, 100000 iteraciones y resultado de 256 bits. El código aleatorio de recuperación usa 16 bytes y se verifica con SHA-256.
- Con Recordarme el id de sesión queda en localStorage; sin marcarlo queda en sessionStorage. No son sesiones autenticadas por un servidor.
- Estas cuentas locales no proporcionan autorización de servidor. No se conoce desde este módulo cómo almacena el backend las credenciales; la identidad HTTP actual incluye rol.

Funciones y elementos principales

Nombre	Qué hace	Línea
`bytesToHex`	Convierte un array de bytes en una cadena hexadecimal.	4
`normalizeEmail`	Elimina espacios en los extremos y convierte el correo a minúsculas.	5
`passwordHash`	Deriva el verificador de contraseña con PBKDF2 y la sal recibida.	7
`recoveryHash`	Calcula SHA-256 del código recortado y convertido a mayúsculas.	13
`accounts`	Lee y comprueba el array de cuentas locales.	17
`readSession`	Busca el id de sesión temporal o persistente y devuelve solo id, nombre y correo.	23
`registerAccount`	Rechaza correo duplicado, genera identificadores y verificadores, guarda la cuenta y devuelve el código de recuperación.	29
`loginAccount`	Verifica correo y contraseña, escribe la sesión en el almacén elegido y devuelve la identidad.	40

Nombre	Qué hace	Línea
`logoutAccount`	Borra la referencia de sesión en los dos tipos de almacenamiento.	49
`recoverAccount`	Verifica correo y código, cambia sal y contraseña y retira las referencias de sesión de esa cuenta.	54

Código completo

63 líneas · SHA-256 del original: `097f5b62a324833b5fa35bd1da92483cda3b00eb7fff68c072c2cce9ea0533cd`

```
const accountsKey = 'umbra.accounts.v1'
const sessionKey = 'umbra.session.v1'
const iterations = 100000
const bytesToHex = bytes => Array.from(bytes, byte => byte.toString(16).padStart(2, '0')).join('')
const normalizeEmail = email => email.trim().toLocaleLowerCase('es')

async function passwordHash(password, salt) {
  const key = await crypto.subtle.importKey('raw', new TextEncoder().encode(password), 'PBKDF2', false, ['deriveBits'])
  const bits = await crypto.subtle.deriveBits({ name: 'PBKDF2', salt: new TextEncoder().encode(salt), iterations, hash: 'SHA-256' }, key, 256)
  return bytesToHex(new Uint8Array(bits))
}

async function recoveryHash(code) {
  return bytesToHex(new Uint8Array(await crypto.subtle.digest('SHA-256', new TextEncoder().encode(code.trim().toUpperCase()))))
}

function accounts(storage) {
  const saved = JSON.parse(storage.getItem(accountsKey) || '[]')
  if (!Array.isArray(saved)) throw new Error('No se pueden leer las cuentas de este navegador.')
  return saved
}

export function readSession(storage = localStorage, temporary = sessionStorage) {
  const id = temporary.getItem(sessionKey) || storage.getItem(sessionKey)
  const account = accounts(storage).find(item => item.id === id)
  return account ? { id: account.id, name: account.name, email: account.email } : null
}

export async function registerAccount({ name, email, password }, storage = localStorage) {
  const list = accounts(storage)
  const normalizedEmail = normalizeEmail(email)
  if (list.some(account => account.email === normalizedEmail)) throw new Error('Ya existe una cuenta con este correo en este navegador.')
  const salt = bytesToHex(crypto.getRandomValues(new Uint8Array(16)))
  const recoveryCode = bytesToHex(crypto.getRandomValues(new Uint8Array(16))).toUpperCase()
  const account = { id: crypto.randomUUID(), name: name.trim(), email: normalizedEmail, salt, passwordHash: await passwordHash(password, salt), recoveryHash: await recoveryHash(recoveryCode) }
  storage.setItem(accountsKey, JSON.stringify([...list, account]))
  return recoveryCode
}

```

```
40 export async function loginAccount({ email, password, remember }, storage = localStorage, temporary =
   sessionStorage) {
41   const account = accounts(storage).find(item => item.email === normalizeEmail(email))
42   if (!account || await passwordHash(password, account.salt) !== account.passwordHash) throw new
   Error('El correo o la contraseña no coinciden con una cuenta de este navegador.')
43   const target = remember ? storage : temporary
44   target.setItem(sessionKey, account.id)
45   ;(remember ? temporary : storage).removeItem(sessionKey)
46   return { id: account.id, name: account.name, email: account.email }
47 }
48
49 export function logoutAccount(storage = localStorage, temporary = sessionStorage) {
50   storage.removeItem(sessionKey)
51   temporary.removeItem(sessionKey)
52 }
53
54 export async function recoverAccount({ email, recoveryCode, password }, storage = localStorage, temporary
   = sessionStorage) {
55   const list = accounts(storage)
56   const account = list.find(item => item.email === normalizeEmail(email))
57   if (!account || await recoveryHash(recoveryCode) !== account.recoveryHash) throw new Error('El correo o
   el código de recuperación no son correctos.')
58   const salt = bytesToHex(crypto.getRandomValues(new Uint8Array(16)))
59   const updated = { ...account, salt, passwordHash: await passwordHash(password, salt) }
60   storage.setItem(accountsKey, JSON.stringify(list.map(item => item.id === account.id ? updated : item)))
61   if (storage.getItem(sessionKey) === account.id) storage.removeItem(sessionKey)
62   if (temporary.getItem(sessionKey) === account.id) temporary.removeItem(sessionKey)
63 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 26 DE 62

preferenceStore.js

`src/utils/preferenceStore.js`

Define valores predeterminados y persistencia de accesibilidad separada por perfil.

- defaultPreferences usa texto normal, contraste normal, tema oscuro, controles normales, acento rojo y anuncios de navegación. La voz, el foco reforzado y el movimiento reducido empiezan desactivados.
- La antigua clave global v1 solo se recupera para guest. Las cuentas nuevas empiezan con sus propios valores predeterminados.
- La normalización admite únicamente tamaños, contrastes, temas, acentos y velocidades conocidos.

Funciones y elementos principales

Nombre	Qué hace	Línea
`normalizePreferences`	Construye una configuración válida y sustituye opciones desconocidas por sus valores predeterminados.	8
`choose`	Ayudante interno que acepta un valor solo si aparece en el catálogo de opciones.	10
`readPreferences`	Lee la clave v2 del propietario, considera la migración de Invitado o devuelve defaults.	21
`writePreferences`	Normaliza y guarda la configuración del propietario.	32

Código completo

36 líneas · SHA-256 del original: `b8f0d16a392951b7a68d78af14e91fdd1f28ca3d40889207ca74ebcb701c70e9`

```
1  import { accentColors, contrastModes, elementSizes, textSizes, themes } from '../data/accessibility.js'
2
3  export const defaultPreferences = Object.freeze({
4    textSize: 'normal', contrast: 'normal', theme: 'dark', elementSize: 'normal', accent: 'red',
5    readAloud: false, readingRate: 1, announceNavigation: true, enhancedFocus: false, reduceMotion: false,
6  })
7
8  export function normalizePreferences(saved) {
9    const source = saved && typeof saved === 'object' ? saved : {}
10   const choose = (key, options) => options.some(option => option.value === source[key]) ? source[key] :
   defaultPreferences[key]
11   return {
12     textSize: choose('textSize', textSizes), contrast: choose('contrast', contrastModes),
```

```
    theme: choose('theme', themes), elementSize: choose('elementSize', elementSizes), accent: choose('accent', accentColors),
    readAloud: source.readAloud === true,
    readingRate: [0.75, 1, 1.25].includes(source.readingRate) ? source.readingRate : 1,
    announceNavigation: source.announceNavigation !== false,
    enhancedFocus: source.enhancedFocus === true, reduceMotion: source.reduceMotion === true,
  }
}

export function readPreferences(owner, storage = localStorage) {
  const saved = storage.getItem(`umbra.accessibility.v2.${owner}`)
  if (saved !== null) return normalizePreferences(JSON.parse(saved))
  // The previous app-wide settings belong only to the guest. Accounts start independently.
  if (owner === 'guest') {
    const legacy = JSON.parse(storage.getItem('umbra.accessibility.v1') || 'null')
    if (legacy) return normalizePreferences({ textSize: legacy.largeText ? 'large' : 'normal', contrast: legacy.highContrast ? 'high' : 'normal' })
  }
  return { ...defaultPreferences }
}

export function writePreferences(owner, preferences, storage = localStorage) {
  const checked = normalizePreferences(preferences)
  storage.setItem(`umbra.accessibility.v2.${owner}`, JSON.stringify(checked))
  return checked
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 27 DE 62

validateForm.js

`src/utils/validateForm.js`

Validación declarativa compartida por AppForm y Login.

- Recibe la definición de campos y un FormData. Devuelve un objeto de errores, con el nombre del campo como clave.
- Comprueba obligatoriedad, formato de correo, longitudes, opciones permitidas, carácter especial, enteros en rango y coincidencia de contraseñas.
- Se detiene en el primer error de cada campo. Las comprobaciones del navegador y de la interfaz no sustituyen la validación de un futuro backend.

Funciones y elementos principales

Nombre	Qué hace	Línea
`validateForm`	Recorre las reglas y devuelve mensajes listos para asociarse a los controles.	1

Código completo

25 líneas · SHA-256 del original: `97ab9fdf59e9edaeff458ef55a36ad19a9328d9cd8a61051f1cc0bcc9740a4c4`

```
1  export function validateForm(fields, values) {
2    const errors = {}
3    for (const field of fields) {
4      const raw = values.get(field.name)
5      const value = typeof raw === 'string' ? raw : ''
6      if (field.required && !value.trim()) {
7        errors[field.name] = field.type === 'checkbox' ? 'Debes marcar esta casilla para continuar.' :
   `Completa el campo «${field.label}».`
8      } else if (value && field.type === 'email' && !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)) {
9        errors[field.name] = 'Escribe un correo válido, por ejemplo nombre@dominio.es.'
10     } else if (value && field.minLength && value.trim().length < field.minLength) {
11       errors[field.name] = `Introduce al menos ${field.minLength} caracteres.`
12     } else if (value && field.maxLength && value.length > field.maxLength) {
13       errors[field.name] = `Introduce como máximo ${field.maxLength} caracteres.`
14     } else if (value && field.options && !field.options.includes(value)) {
15       errors[field.name] = 'Selecciona una opción de la lista.'
16     } else if (value && field.requireSpecial && !/[^\p{L}\p{N}\s]/u.test(value)) {
17       errors[field.name] = 'Incluye al menos un carácter especial, como !, @, # o $.'
18     } else if (value && field.type === 'number' && (!Number.isInteger(Number(value)) || Number(value) <
   field.min || Number(value) > field.max)) {
19       errors[field.name] = `Introduce un número entero entre ${field.min} y ${field.max}.`
20     } else if (field.matches && value !== values.get(field.matches)) {
21       errors[field.name] = 'Las contraseñas deben coincidir.'
```

```
    }
  }
  return errors
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 28 DE 62

characters.js

`src/data/characters.js`

Seis personajes de ejemplo para inicializar bibliotecas locales.

- initialCharacters incluye Elara, Kael, Seraphine, Draeg, Inquisidor y Angélica.
- El generador completo se encuentra ahora en src/utils/generateCharacter.js. Este archivo no envía datos al servidor.

Código completo

38 líneas · SHA-256 del original: `b868c716b17d873e9019daf55604b1e2ff5ce4f31ea41899a4425de74eb96193`

```
export const initialCharacters = [
  {
    id: 'elara', name: 'Elara Valen', type: 'Hechicera', race: 'Humana', level: 8,
    realm: 'Reino de Aethoria', portrait: 0,
    story: 'Nacida en las ruinas de Velmora, Elara teje la magia de aquello que otros temen. Busca una verdad enterrada entre las sombras, aunque el precio sea su propia alma.',
    traits: ['Reservada', 'Leal', 'Determinada'], stats: [8, 14, 18, 12],
  },
  {
    id: 'kael', name: 'Kael Grim', type: 'Pícaro', race: 'Humano', level: 6,
    realm: 'Ciudad de Nox', portrait: 1,
    story: 'Entre los callejones de Nox, Kael aprendió que todo secreto tiene un precio. Ahora un antiguo juramento lo obliga a proteger aquello que una vez prometió robar.',
    traits: ['Astuto', 'Independiente', 'Irónico'], stats: [12, 19, 14, 15],
  },
  {
    id: 'seraphine', name: 'Seraphine', type: 'Hechicera', race: 'Elfa', level: 10,
    realm: 'Bosque de Lúmina', portrait: 2,
    story: 'La última guardiana de un bosque sin amanecer escucha las voces de los árboles. Una melodía olvidada podría devolver la luz a su pueblo.',
    traits: ['Serena', 'Sabia', 'Protectora'], stats: [7, 15, 20, 16],
  },
  {
    id: 'draeg', name: 'Draeg', type: 'Guerrero', race: 'Humano', level: 12,
    realm: 'Tierras del Norte', portrait: 3,
    story: 'Su armadura recuerda cada batalla, pero Draeg solo recuerda a quienes no pudo salvar. Camina hacia el norte buscando una paz que ninguna victoria le ha dado.',
    traits: ['Valiente', 'Leal', 'Obstinado'], stats: [20, 12, 10, 13],
  },
  {
    id: 'inquisidor', name: 'Inquisidor', type: 'Guerrero', race: 'Humano', level: 9,
    realm: 'Orden del Eclipse', portrait: 4,
    story: 'Tras la máscara del inquisidor se esconde una duda. Cuando la orden exige quemar una biblioteca, deberá elegir entre su deber y la memoria del mundo.',
    traits: ['Metódico', 'Severo', 'Cauto'], stats: [17, 11, 16, 10],
  },
  {
```

```
33      id: 'angelica', name: 'Angélica Mangus', type: 'Hechicera', race: 'Celestial', level: 15,
34      realm: 'Reino del Cielo', portrait: 5,
35      story: 'Nacida de una pluma del arcángel Miguel, con carácter fuerte, valentía y poca paciencia.
    Descendió para encontrar un nombre perdido entre los mortales.',
36      traits: ['Extrovertida', 'Dominante', 'Audaz'], stats: [8, 20, 18, 19],
37    },
38  ]
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 29 DE 62

characterOptions.js

`src/data/characterOptions.js`

Catálogos de clases, razas, rasgos y nombres de atributos, además de la ficha vacía.

- Fuerza, Destreza, Inteligencia y Carisma determinan el orden del array stats en toda la app.
- blankCharacter prepara los campos iniciales del editor, con nivel 1 y cuatro atributos de valor 10.

Funciones y elementos principales

Nombre	Qué hace	Línea
`blankCharacter`	Devuelve un objeto nuevo de personaje todavía sin nombre ni identificador persistente.	6

Código completo

19 líneas · SHA-256 del original: `638e78935a9815f6be3f8f4f941d8f40c874ae40fff4fdecef8bdc0d1516c7e3`

```
1  export const characterTypes = ['Hechicera', 'Hechicero', 'Guerrera', 'Guerrero', 'Pícara', 'Pícaro',
   'Exploradora', 'Explorador', 'Sanadora', 'Sanador', 'Paladín', 'Bardo']
2  export const characterRaces = ['Humana', 'Humano', 'Elfa', 'Elfo', 'Enana', 'Enano', 'Orca', 'Orco',
   'Celestial', 'Tiefling', 'Otra']
3  export const personalityTraits = ['Leal', 'Valiente', 'Reservada', 'Determinada', 'Astuto',
   'Independiente', 'Serena', 'Sabia', 'Protectora', 'Irónico', 'Audaz', 'Cauto']
4  export const attributeNames = ['Fuerza', 'Destreza', 'Inteligencia', 'Carisma']
5
6  export function blankCharacter() {
7    return {
8      name: '',
9      type: 'Guerrero',
10     race: 'Humana',
11     level: 1,
12     realm: '',
13     portrait: 0,
14     imageDataUrl: '',
15     imageName: '',
16     story: '',
17     traits: [],
18     stats: [10, 10, 10, 10] }
19 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 30 DE 62

navigation.js

`src/data/navigation.js`

Fuente común de enlaces y títulos de pantalla.

- menuItems define las secciones de la cabecera; mainTabs define los cuatro accesos inferiores; pageTitles relaciona rutas con títulos.
- App utiliza estos títulos para el encabezado, la pestaña del navegador y el aviso de cambio de pantalla.

Código completo

24 líneas · SHA-256 del original: `1c2230c0b5cc331334eb1857b21ac7fcebaf74f9ee217d5c879250faa6668386`

```
export const menuItems = [
  { label: 'Inicio', path: '/inicio', icon: 'home' },
  { label: 'Biblioteca', path: '/biblioteca', icon: 'book' },
  { label: 'Crear personaje', path: '/crear-personaje' },
  { label: 'Inspiración', path: '/inspiracion', icon: 'sparkle' },
  { label: 'Perfil', path: '/perfil', icon: 'user' },
  { label: 'Ayuda', path: '/ayuda' },
  { label: 'Accesibilidad', path: '/accesibilidad' },
  { label: 'Iniciar sesión', path: '/login' },
]

export const mainTabs = [
  { label: 'Inicio', path: '/inicio', icon: 'home' },
  { label: 'Biblioteca', path: '/biblioteca', icon: 'book' },
  { label: 'Inspiración', path: '/inspiracion', icon: 'sparkle' },
  { label: 'Perfil', path: '/perfil', icon: 'user' },
]
export const pageTitles = {
  ...Object.fromEntries(menuItems.map(item => [item.path, item.label])),
  '/registro': 'Crear cuenta', '/recuperar': 'Recuperar acceso', '/privacidad': 'Privacidad y uso',
  '/accesibilidad/texto': 'Tamaño de texto', '/accesibilidad/contraste': 'Modo de contraste',
  '/accesibilidad/lectura': 'Lectores de pantalla', '/accesibilidad/teclado': 'Navegación por teclado',
  '/accesibilidad/apariencia': 'Personalización visual',
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 31 DE 62

accessibility.js

`src/data/accessibility.js`

Catálogos de valores y etiquetas de los cinco menús de accesibilidad.

- Define textSizes, contrastModes, themes, elementSizes y accentColors.
- accessibilityMenus contiene id, etiqueta, icono y descripción de cada submenú. La pantalla y la validación leen los mismos catálogos para evitar opciones divergentes.

Código completo

36 líneas · SHA-256 del original: `80a1383f7d244410cc28615ffe6b6f2a566c191656b219ed07995f320a452818`

```
export const textSizes = [
  { value: 'small', label: 'Pequeño' },
  { value: 'normal', label: 'Normal' },
  { value: 'large', label: 'Grande' },
  { value: 'extra-large', label: 'Extra grande' },
]
export const contrastModes = [
  { value: 'normal', label: 'Normal', description: 'Colores del tema elegido.' },
  { value: 'high', label: 'Alto contraste', description: 'Fondo negro, texto blanco y controles amarillos.' },
  { value: 'inverted', label: 'Invertido', description: 'Fondo claro, texto negro y controles azules.' },
]
export const themes = [
  { value: 'dark', label: 'Oscuro' },
  { value: 'light', label: 'Claro' },
  { value: 'sepia', label: 'Sepia' },
]
export const elementSizes = [
  { value: 'normal', label: 'Normal' },
  { value: 'large', label: 'Grande' },
  { value: 'extra-large', label: 'Extra grande' },
]
export const accentColors = [
  { value: 'red', label: 'Rojo', color: '#9e3537' },
  { value: 'purple', label: 'Violeta', color: '#7738a8' },
  { value: 'blue', label: 'Azul', color: '#12679a' },
  { value: 'green', label: 'Verde', color: '#27743e' },
  { value: 'orange', label: 'Naranja', color: '#a04d0b' },
  { value: 'slate', label: 'Gris', color: '#536177' },
]
export const accessibilityMenus = [
  { id: 'texto', label: 'Tamaño de texto', icon: 'type', description: 'Ajusta la lectura y prueba el resultado.' },
  { id: 'contraste', label: 'Modo de contraste', icon: 'contrast', description: 'Normal, alto contraste o invertido.' },
  { id: 'lectura', label: 'Lectores de pantalla', icon: 'speaker', description: 'Anuncios de navegación y lectura en voz alta.' },
```

```
34   { id: 'teclado', label: 'Navegación por teclado', icon: 'keyboard', description: 'Atajos y resaltado
     del foco.' },
35   { id: 'apariencia', label: 'Personalización visual', icon: 'palette', description: 'Tema, tamaño de
     elementos y color de acento.' },
36 ]
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 32 DE 62

index.css

`src/index.css`

Fuentes locales, reinicio de estilos y variables base de color, texto y controles.

- Declara Cormorant Garamond e Inter con font-face y recursos de public/fonts.
- Define los tokens base: foreground, background, surface, accent, text-scale, control-height y otros.
- Establece box-sizing, estilo base de enlaces y controles, foco visible y respeto a prefers-reduced-motion.

Código completo

77 líneas · SHA-256 del original: `c978c61f6a0b248b94abc4091f7f8418e86dceb6738676638b7eb73e1e113af0`

```
@font-face {
  font-family: 'Cormorant Garamond';
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url('/fonts/cormorant-garamond-regular.ttf') format('truetype');
}
@font-face {
  font-family: 'Cormorant Garamond';
  font-style: normal;
  font-weight: 500;
  font-display: swap;
  src: url('/fonts/cormorant-garamond-medium.ttf') format('truetype');
}
@font-face {
  font-family: 'Cormorant Garamond';
  font-style: italic;
  font-weight: 400;
  font-display: swap;
  src: url('/fonts/cormorant-garamond-italic.ttf') format('truetype');
}
@font-face {
  font-family: 'Inter';
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url('/fonts/inter-regular.ttf') format('truetype');
}

:root {
  font-family: 'Inter', sans-serif;
  color: #ded9c7;
  background: #202020;
  color-scheme: dark;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
```

```
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  --font-display: 'Cormorant Garamond', Georgia, serif;
  --background: #090f0f;
  --gold: #a99770;
  --muted: #9b9d91;
  --border: #303833;
  --accent: #9e3537;
  --foreground: #ded9c7;
  --surface: #111a18;
  --surface-raised: #25352c;
  --input-background: #0b1210;
  --feedback-background: #1b221d;
  --error: #ffbba5;
  --error-background: #302020;
  --success: #7d9c70;
  --accent-hover: #b44244;
  --accent-ink: #e09b82;
  --button-text: #fff8ef;
  --text-scale: 1;
  --control-height: 44px;
  --control-padding: 10px;
  --icon-size: 22px;
}

* { box-sizing: border-box; }
body { min-width: 280px; min-height: 100vh; margin: 0; }
button, input, select, textarea { font: inherit; }
button, a { -webkit-tap-highlight-color: transparent; }
button { color: inherit; cursor: pointer; border: 0; }
a { color: inherit; text-decoration: none; }
h1, h2, h3, p { margin: 0; }
button:focus-visible, a:focus-visible, input:focus-visible, select:focus-visible, textarea:focus-visible
{
  outline: 2px solid var(--gold);
  outline-offset: 4px;
}
img { max-width: 100%; }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto !important; transition: none !important; }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 33 DE 62

App.css

`src/App.css`

Composición general, tarjetas, formularios, diálogos y reglas globales de accesibilidad.

- La interfaz tiene un marco máximo de 390 px y se adapta a pantallas estrechas. Grid y flex distribuyen tarjetas, barras y controles.
- Los selectores data-text-size multiplican los tamaños de fuente. data-element-size ajusta alturas y espacios de controles e iconos.
- Los selectores data-theme eligen oscuro, claro o sepia; data-accent cambia el acento. Las reglas de contraste se declaran después y tienen prioridad.
- enhanced-focus refuerza el contorno y reduce-motion elimina transiciones. Se ajustan las columnas para evitar desbordamiento en móvil.

Código completo

1159 líneas · SHA-256 del original: `1a3d9232a1bc2254aa61f8294708aa4b5192ed1a229293c803c7941a09b2f8df`

```
.app-frame {
  width: 100%;
  max-width: 390px;
  min-height: 780px;
  margin: 32px auto;
  border: 1px solid var(--border);
  background: var(--background);
  color: var(--foreground);
  display: flex;
  flex-direction: column;
  position: relative;
}

.header {
  height: 62px;
  flex-shrink: 0;
  padding: 0 17px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1px solid var(--border);
  position: relative;
  z-index: 2;
}

.icon-button {
  display: grid;
  place-items: center;
  width: 44px;
  height: 44px;
```

```
  padding: 0;
  color: var(--gold);
  background: transparent;
}

.brand {
  display: flex;
  align-items: center;
  gap: 8px;
  font-family: var(--font-display);
  font-size: calc(25px * var(--text-scale));
  letter-spacing: 1px;
}

.brand svg {
  color: var(--gold);
}

.notification-trigger {
  position: relative;
}

.notification-dot {
  width: 5px;
  height: 5px;
  border-radius: 50%;
  background: var(--accent-ink);
  position: absolute;
  right: 8px;
  top: 6px;
}

.header-popover {
  position: absolute;
  top: 55px;
  left: 16px;
  right: 16px;
  max-height: calc(100dvh - 85px);
  overflow-y: auto;
  padding: 8px;
  background: var(--surface);
  border: 1px solid var(--border);
  box-shadow: 0 12px 24px #0008;
}

.header-popover a,
.header-popover button {
  display: block;
  width: 100%;
  padding: 12px;
  font-size: calc(13px * var(--text-scale));
  text-align: left;
  background: transparent;
}

.header-popover a:hover,
.header-popover button:hover {
  background: var(--surface-raised);
}
```

```
90
91  .main-content {
92    flex: 1;
93    padding: 24px 23px 0;
94  }
95
96  .hero {
97    position: relative;
98    padding-top: 106px;
99    overflow: hidden;
100 }
101
102 .hero-landscape {
103   position: absolute;
104   inset: 0 0 auto;
105   height: 246px;
106   background: linear-gradient(180deg, #090f0f05 5%, #090f0f40 35%, var(--background) 94%),
    url('/images/umbra-castle.png') center 38% / cover;
107 }
108
109 .hero-copy {
110   position: relative;
111 }
112
113 .welcome {
114   color: var(--gold);
115   font-family: var(--font-display);
116   font-size: calc(17px * var(--text-scale));
117   margin-bottom: 7px;
118 }
119
120 h1 {
121   font-family: var(--font-display);
122   font-weight: 400;
123   font-size: calc(31px * var(--text-scale));
124   line-height: 1.04;
125 }
126
127 .hero h1 {
128   text-transform: uppercase;
129   font-variant-caps: small-caps;
130   letter-spacing: -.2px;
131 }
132
133 .hero-description {
134   margin: 12px 0 27px;
135   color: var(--muted);
136   font-size: calc(12px * var(--text-scale));
137   line-height: 1.65;
138 }
139
140 .primary-button {
141   display: flex;
142   align-items: center;
143   justify-content: center;
144   gap: 9px;
145   width: 100%;
146   min-height: 47px;
147   padding: 9px 12px;
```

```
  border-radius: 2px;
  background: var(--accent);
  color: var(--button-text);
  font-family: var(--font-display);
  font-size: calc(18px * var(--text-scale));
  transition: background .2s;
}

.primary-button:hover {
  background: var(--accent-hover);
}

.primary-button>span {
  font-family: Georgia, serif;
  font-size: calc(23px * var(--text-scale));
  line-height: 1;
}

.ornament {
  display: flex;
  align-items: center;
  gap: 22px;
  margin: 16px 0 17px;
  color: var(--gold);
  font-size: calc(14px * var(--text-scale));
}

.ornament::before,
.ornament::after {
  content: '';
  height: 1px;
  flex: 1;
  background: var(--border);
}

.section-heading {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 8px;
  margin-bottom: 13px;
}

h2 {
  font-family: var(--font-display);
  font-size: calc(22px * var(--text-scale));
  line-height: 1.2;
  font-weight: 500;
}

.text-button {
  flex-shrink: 0;
  min-height: 44px;
  display: inline-flex;
  align-items: center;
  padding: 8px 0;
  color: var(--accent-ink);
  background: transparent;
  font-size: calc(11px * var(--text-scale));
```

```
}

.text-button span {
  margin-left: 4px;
  font-size: calc(15px * var(--text-scale));
}

.character-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}

.character-card {
  width: 100%;
  padding: 0;
  text-align: left;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 4px;
  overflow: hidden;
  transition: border-color .2s, transform .2s;
}

.character-card:hover {
  border-color: var(--gold);
  transform: translateY(-2px);
}

.character-portrait {
  display: block;
  width: 100%;
  height: auto;
  aspect-ratio: 1.18;
  object-fit: cover;
  object-position: center;
}

.portrait-placeholder {
  display: grid;
  place-items: center;
  color: var(--gold);
  background: radial-gradient(ellipse at top, var(--surface-raised), var(--surface) 75%);
  font-family: var(--font-display);
  font-size: calc(42px * var(--text-scale));
}

.character-info {
  padding: 10px 10px 9px;
}

.character-info h3 {
  font-family: var(--font-display);
  font-size: calc(20px * var(--text-scale));
  line-height: 1.1;
  font-weight: 500;
}

.character-info p {
```

```
  margin-top: 6px;
  color: var(--muted);
  font-size: calc(11px * var(--text-scale));
  line-height: 1.5;
}

.closing-quote {
  padding: 20px 0 25px;
  text-align: center;
  color: var(--muted);
  font-family: var(--font-display);
  font-size: calc(14px * var(--text-scale));
  font-style: italic;
}

.bottom-navigation {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 4px;
  padding: 12px 8px 16px;
  border-top: 1px solid var(--border);
  background: var(--surface);
}

.nav-item {
  min-height: 44px;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  gap: 7px;
  color: var(--muted);
}

.nav-label {
  font-size: calc(11px * var(--text-scale));
}

.nav-item.active {
  color: var(--accent-ink);
}

.page-heading {
  padding: 20px 0 28px;
}

.page-heading h1 {
  margin-top: 12px;
}

.eyebrow {
  color: var(--gold);
  font-size: calc(10px * var(--text-scale));
  letter-spacing: 1.5px;
}

.page-description,
.character-story {
  color: var(--muted);
```

```
  font-size: calc(13px * var(--text-scale));
  line-height: 1.8;
  padding: 16px 0;
  white-space: pre-wrap;
  overflow-wrap: anywhere;
}

.search-field {
  display: flex;
  gap: 8px;
  align-items: center;
  margin-bottom: 18px;
  color: var(--gold);
}

.search-field input {
  min-width: 0;
  width: 100%;
}

.inspiration-panel,
.profile-panel {
  display: grid;
  gap: 22px;
  padding: 0 0 32px;
}

.inspiration-panel>svg,
.profile-panel>svg {
  color: var(--gold);
}

.inspiration-panel p,
.profile-panel p {
  color: var(--muted);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.8;
}

.storage-notice {
  color: var(--error);
  padding: 20px 0;
  font-size: calc(12px * var(--text-scale));
  line-height: 1.6;
}

.umbra-dialog {
  width: min(360px, calc(100% - 32px));
  max-height: calc(100dvh - 48px);
  margin: auto;
  padding: 0;
  color: var(--foreground);
  background: var(--surface);
  border: 1px solid var(--border);
}

.umbra-dialog::backdrop {
  background: #000b;
}
```

```
384
385 .dialog-content {
386   padding: 28px 22px;
387   position: relative;
388 }
389
390 .dialog-close {
391   position: absolute;
392   right: 7px;
393   top: 7px;
394 }
395
396 .dialog-content h2 {
397   margin: 0 20px 24px 0;
398   font-size: calc(29px * var(--text-scale));
399 }
400
401 .dialog-content>.character-portrait {
402   aspect-ratio: 1;
403   margin-bottom: 20px;
404   border-radius: 3px;
405 }
406
407 .character-form {
408   display: grid;
409   gap: 16px;
410 }
411
412 .character-form label {
413   display: grid;
414   gap: 7px;
415   font-size: calc(12px * var(--text-scale));
416   color: var(--muted);
417 }
418
419 input,
420 select,
421 textarea {
422   width: 100%;
423   min-width: 0;
424   padding: 10px;
425   border: 1px solid var(--border);
426   border-radius: 3px;
427   background: var(--input-background);
428   color: var(--foreground);
429   font-size: calc(13px * var(--text-scale));
430 }
431
432 textarea {
433   resize: vertical;
434 }
435
436 .form-row {
437   display: grid;
438   grid-template-columns: 1fr 1fr;
439   gap: 12px;
440 }
441
442 @media (max-width: 480px) {
```

```
  .app-frame {
    min-height: 100dvh;
    margin: 0 auto;
    border: 0;
  }

  .bottom-navigation {
    padding-bottom: max(16px, env(safe-area-inset-bottom));
  }
}

@media (max-width: 350px) {
  .header {
    height: 54px;
  }

  .brand {
    font-size: calc(22px * var(--text-scale));
  }

  .main-content {
    padding: 21px 20px 0;
  }

  .hero {
    padding-top: 96px;
  }

  .hero h1 {
    font-size: calc(28px * var(--text-scale));
  }

  .primary-button {
    min-height: 42px;
  }

  .section-heading {
    margin-bottom: 10px;
  }

  .text-button {
    padding-top: 4px;
    padding-bottom: 4px;
  }

  .character-info {
    padding-top: 8px;
    padding-bottom: 7px;
  }

  .character-info p {
    margin-top: 5px;
  }

  .closing-quote {
    padding-top: 16px;
    padding-bottom: 20px;
  }

```

```
  .character-info h3 {
    font-size: calc(18px * var(--text-scale));
  }

  h2 {
    font-size: calc(20px * var(--text-scale));
  }

  .bottom-navigation {
    padding-top: 10px;
    padding-bottom: max(12px, env(safe-area-inset-bottom));
  }

  .nav-item {
    min-height: 44px;
  }
}

.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip-path: inset(50%);
  white-space: nowrap;
  border: 0;
}

.skip-link {
  position: absolute;
  top: 8px;
  left: 8px;
  z-index: 5;
  padding: 12px;
  background: var(--foreground);
  color: var(--background);
  transform: translateY(-200%);
}

.skip-link:focus {
  transform: translateY(0);
}

.main-content:focus {
  outline: none;
}

.guest-note {
  padding: 0 0 22px;
  color: var(--muted);
  font-size: calc(12px * var(--text-scale));
  line-height: 1.7;
}

.guest-note a,
.account-link a,
.inline-link {
```

```
  color: var(--accent-ink);
  text-decoration: underline;
  text-underline-offset: 3px;
}

.inline-link {
  display: inline-flex;
  align-items: center;
  min-height: 44px;
  font-size: calc(13px * var(--text-scale));
}

.form-notice,
.feedback {
  padding: 12px;
  border: 1px solid var(--border);
  border-radius: 3px;
  background: var(--feedback-background);
  color: var(--foreground);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.7;
  overflow-wrap: anywhere;
}

.form-notice {
  margin-bottom: 20px;
}

.connection-status {
  margin: 12px 16px 0;
}

.feedback.error {
  color: var(--error);
  border-color: var(--error);
  background: var(--error-background);
}

.feedback.success {
  border-color: var(--success);
}

.form-field {
  display: grid;
  gap: 7px;
}

.form-field label {
  display: block;
  font-size: calc(13px * var(--text-scale));
}

.form-hint {
  font-size: calc(12px * var(--text-scale));
  color: var(--muted);
  line-height: 1.7;
}

.field-error {
```

```
  color: var(--error);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.6;
}

[aria-invalid="true"] {
  border-color: var(--error);
}

.checkbox-label {
  display: flex !important;
  align-items: flex-start;
  gap: 10px;
  min-height: 44px;
  line-height: 1.7;
}

input[type="checkbox"] {
  flex-shrink: 0;
  width: 20px;
  height: 20px;
  margin: 3px 0 0;
  accent-color: var(--gold);
}

.secondary-button {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 44px;
  width: 100%;
  padding: 10px;
  border: 1px solid var(--border);
  border-radius: 3px;
  background: var(--surface);
  color: var(--foreground);
  font-size: calc(13px * var(--text-scale));
  text-align: center;
  line-height: 1.5;
}

.secondary-button:hover {
  background: var(--surface-raised);
}

.action-stack,
.empty-state {
  display: grid;
  gap: 14px;
}

.auth-page {
  padding-bottom: 24px;
}

.account-link {
  margin-top: 24px;
  color: var(--muted);
  font-size: calc(13px * var(--text-scale));
```

```
  line-height: 1.7;
}

.search-results {
  margin-bottom: 16px;
  color: var(--muted);
  font-size: calc(12px * var(--text-scale));
}

.empty-state {
  padding: 16px 0;
  font-size: calc(13px * var(--text-scale));
  color: var(--muted);
  line-height: 1.8;
}

.information-page {
  display: grid;
  gap: 20px;
  padding-bottom: 30px;
}

.information-page p {
  color: var(--muted);
  font-size: calc(14px * var(--text-scale));
  line-height: 1.8;
}

.information-page .feedback {
  color: var(--foreground);
}

.information-page details {
  border-bottom: 1px solid var(--border);
  padding-bottom: 12px;
}

.information-page summary {
  min-height: 44px;
  cursor: pointer;
  font-size: calc(14px * var(--text-scale));
  line-height: 1.7;
  padding: 10px 0;
}

.information-page details p {
  padding: 8px 0;
}

.preference-option {
  display: flex;
  align-items: center;
  gap: 12px;
  min-height: 44px;
  font-size: calc(14px * var(--text-scale));
  line-height: 1.6;
}

.information-page .error-code {
```

```
  font-family: var(--font-display);
  font-size: calc(72px * var(--text-scale));
  color: var(--gold);
  line-height: 1;
}

.character-info h3,
h1,
h2 {
  overflow-wrap: anywhere;
}

.editor-page {
  padding-bottom: 28px;
}

.form-contents {
  display: contents;
  border: 0;
  margin: 0;
  padding: 0;
  min-width: 0;
}

.traits-field {
  margin: 0;
  padding: 14px;
  border: 1px solid var(--border);
  min-width: 0;
}

.traits-field legend {
  font-size: calc(13px * var(--text-scale));
  line-height: 1.6;
  color: var(--gold);
}

.traits-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 8px;
}

.traits-grid .checkbox-label {
  font-size: calc(12px * var(--text-scale));
  overflow-wrap: anywhere;
}

.portrait-editor {
  display: grid;
  gap: 14px;
}

.portrait-editor>.character-portrait {
  width: 160px;
  max-width: 100%;
  aspect-ratio: 1;
  margin: auto;
}
```

```
797
798 .random-button {
799   margin-bottom: 24px;
800 }
801
802 .library-filters {
803   margin-bottom: 12px;
804 }
805
806 .library-filters label {
807   display: grid;
808   gap: 8px;
809   font-size: calc(12px * var(--text-scale));
810   color: var(--muted);
811 }
812
813 .search-field {
814   margin-top: 18px;
815 }
816
817 .favorite-label {
818   display: block;
819   margin-top: 8px;
820   color: var(--gold);
821   font-size: calc(11px * var(--text-scale));
822 }
823
824 .trait-list {
825   list-style: none;
826   display: flex;
827   flex-wrap: wrap;
828   gap: 8px;
829   padding: 0;
830   margin: 0 0 20px;
831 }
832
833 .trait-list li {
834   padding: 6px 9px;
835   background: var(--surface-raised);
836   border-radius: 3px;
837   font-size: calc(12px * var(--text-scale));
838 }
839
840 .attribute-grid {
841   display: grid;
842   grid-template-columns: repeat(2, minmax(0, 1fr));
843   gap: 10px;
844   margin: 0 0 24px;
845 }
846
847 .attribute-grid div {
848   padding: 12px;
849   border: 1px solid var(--border);
850   text-align: center;
851 }
852
853 .attribute-grid dt {
854   font-size: calc(12px * var(--text-scale));
855   color: var(--muted);
```

```
}

.attribute-grid dd {
  margin: 8px 0 0;
  font-family: var(--font-display);
  font-size: calc(26px * var(--text-scale));
  color: var(--gold);
}

.danger-button {
  color: var(--error);
  justify-content: center;
  width: 100%;
}

.profile-stats {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.profile-stats strong {
  color: var(--gold);
  font-size: calc(22px * var(--text-scale));
}

.inspiration-seed {
  display: grid;
  gap: 12px;
  padding: 16px;
  border: 1px solid var(--border);
}

.storage-notice {
  padding: 16px;
}

button:disabled {
  cursor: wait;
  opacity: .6;
}

@media (max-width: 350px) {
  .section-heading {
    flex-wrap: wrap;
  }
}

.app-frame[data-text-size="small"] {
  --text-scale: .9;
}

.app-frame[data-text-size="large"] {
  --text-scale: 1.2;
}

.app-frame[data-text-size="extra-large"] {
  --text-scale: 1.4;
}
```

```
915
916 .app-frame[data-element-size="large"] {
917   --control-height: 52px;
918   --control-padding: 13px;
919   --icon-size: 26px;
920 }
921
922 .app-frame[data-element-size="extra-large"] {
923   --control-height: 60px;
924   --control-padding: 16px;
925   --icon-size: 30px;
926 }
927
928 .app-frame[data-accent="purple"] {
929   --accent: #7738a8;
930   --accent-hover: #8b45bd;
931   --accent-ink: #d4a6fa;
932 }
933
934 .app-frame[data-accent="blue"] {
935   --accent: #12679a;
936   --accent-hover: #1977ac;
937   --accent-ink: #87d2ff;
938 }
939
940 .app-frame[data-accent="green"] {
941   --accent: #27743e;
942   --accent-hover: #308747;
943   --accent-ink: #9bdaa5;
944 }
945
946 .app-frame[data-accent="orange"] {
947   --accent: #a04d0b;
948   --accent-hover: #b05915;
949   --accent-ink: #ffbf82;
950 }
951
952 .app-frame[data-accent="slate"] {
953   --accent: #536177;
954   --accent-hover: #64748b;
955   --accent-ink: #c5d4eb;
956 }
957
958 .app-frame[data-theme="light"] {
959   color-scheme: light;
960   --background: #f5f5f0;
961   --foreground: #252d28;
962   --muted: #50594f;
963   --gold: #726039;
964   --surface: #fff;
965   --surface-raised: #e4e8df;
966   --input-background: #fff;
967   --border: #818c7d;
968   --feedback-background: #e6eddf;
969   --error: #9c2b28;
970   --error-background: #ffe5df;
971   --success: #34733d;
972   --accent-ink: var(--accent);
973 }
```

```
974
975 .app-frame[data-theme="sepia"] {
976   color-scheme: light;
977   --background: #efe2c8;
978   --foreground: #3e3020;
979   --muted: #61503b;
980   --gold: #77551f;
981   --surface: #f9efd9;
982   --surface-raised: #e1d0b2;
983   --input-background: #fff7e7;
984   --border: #8e7958;
985   --feedback-background: #e5d9bb;
986   --error: #922c24;
987   --error-background: #f7d4bb;
988   --success: #38642d;
989   --accent-ink: var(--accent);
990 }
991
992 .app-frame[data-contrast="high"] {
993   color-scheme: dark;
994   --background: #000;
995   --foreground: #fff;
996   --muted: #fff;
997   --gold: #ffed00;
998   --surface: #000;
999   --surface-raised: #171717;
1000   --input-background: #000;
1001   --border: #fff;
1002   --feedback-background: #000;
1003   --error: #ffbcae;
1004   --error-background: #200000;
1005   --success: #b1ff9f;
1006   --accent: #ffed00;
1007   --accent-hover: #fff68a;
1008   --accent-ink: #ffed00;
1009   --button-text: #000;
1010 }
1011
1012 .app-frame[data-contrast="inverted"] {
1013   color-scheme: light;
1014   --background: #fff;
1015   --foreground: #000;
1016   --muted: #111;
1017   --gold: #004b7a;
1018   --surface: #fff;
1019   --surface-raised: #e0f2ff;
1020   --input-background: #fff;
1021   --border: #000;
1022   --feedback-background: #eaf7ff;
1023   --error: #960000;
1024   --error-background: #ffe8e8;
1025   --success: #125122;
1026   --accent: #005180;
1027   --accent-hover: #003a5c;
1028   --accent-ink: #004b7a;
1029   --button-text: #fff;
1030 }
1031
1032 .app-frame .primary-button,
```

```
.app-frame .secondary-button,
.app-frame .icon-button,
.app-frame .text-button,
.app-frame .nav-item,
.app-frame .inline-link,
.app-frame .header-popover a,
.app-frame .header-popover button,
.app-frame .login-create-account,
.app-frame .login-recovery,
.app-frame .login-remember {
  min-height: var(--control-height);
}

.app-frame .icon-button {
  width: var(--control-height);
  height: var(--control-height);
  flex-shrink: 0;
}

.app-frame .icon-button svg,
.app-frame .nav-item svg {
  width: var(--icon-size);
  height: var(--icon-size);
}

.app-frame .header {
  height: auto;
  min-height: 62px;
  padding-top: 6px;
  padding-bottom: 6px;
}

.app-frame input:not([type="checkbox"]):not([type="radio"]):not([type="range"]),
.app-frame select,
.app-frame textarea {
  min-height: var(--control-height);
  padding: var(--control-padding);
}

.app-frame .login-input input {
  height: auto;
  padding-left: 34px;
  padding-right: 44px;
}

.app-frame .login-password-toggle {
  height: var(--control-height);
  width: var(--control-height);
}

.app-frame input[type="checkbox"],
.app-frame input[type="radio"] {
  accent-color: var(--accent);
}

.app-frame .section-heading {
  flex-wrap: wrap;
}

```

```
.app-frame .nav-item {
  min-width: 0;
}

.app-frame .nav-label {
  text-align: center;
  overflow-wrap: anywhere;
}

.app-frame p,
.app-frame label,
.app-frame small,
.app-frame legend {
  overflow-wrap: anywhere;
}

.app-frame .read-aloud {
  display: grid;
  gap: 12px;
  margin-bottom: 16px;
}

.app-frame .hero-copy {
  color: var(--foreground);
}

.app-frame[data-theme="light"] .hero-landscape,
.app-frame[data-theme="sepia"] .hero-landscape,
.app-frame[data-contrast="inverted"] .hero-landscape {
  opacity: .18;
}

.app-frame[data-contrast="high"] .hero-landscape {
  opacity: .25;
}

.app-frame.enhanced-focus :focus-visible {
  outline: 4px solid var(--gold);
  outline-offset: 5px;
}

.app-frame.reduce-motion *,
.app-frame.reduce-motion *::before,
.app-frame.reduce-motion *::after {
  animation: none !important;
  transition: none !important;
  scroll-behavior: auto !important;
}

.app-frame.reduce-motion .character-card:hover {
  transform: none;
}

@media (prefers-reduced-motion: reduce) {
  .character-card:hover {
    transform: none;
  }
}

```

```
@media (max-width: 350px) {

  .app-frame[data-text-size="large"] .character-grid,
  .app-frame[data-text-size="extra-large"] .character-grid,
  .app-frame[data-text-size="extra-large"] .traits-grid,
  .app-frame[data-element-size="extra-large"] .library-filters {
    grid-template-columns: 1fr;
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 34 DE 62

Login.css

`src/pages/Login.css`

Composición específica de la pantalla de acceso.

- Estiliza el marco de login, el símbolo, los campos con iconos, el botón de visibilidad, las opciones y los separadores.
- Utiliza las variables de color y text-scale para respetar los ajustes del perfil Invitado que se ven antes del acceso.

Código completo

238 líneas · SHA-256 del original: `9171b7bceded35939c7f54794fde857b8e4a5636fff99f6c79b6ff68fd34d886`

```
.app-frame.login-frame {
  max-width: 336px;
  min-height: 728px;
}

.login-frame .header {
  height: 54px;
  padding: 0 8px;
}

.login-frame .brand {
  gap: 7px;
  font-size: calc(20px * var(--text-scale));
  letter-spacing: .5px;
}

.login-frame .brand svg {
  width: 25px;
  height: 25px;
}

.login-frame .header .icon-button svg {
  width: 18px;
  height: 18px;
}

.login-frame .main-content {
  display: flex;
  padding: 0 20px;
}

.login-page {
  display: flex;
  flex-direction: column;
  width: 100%;
  min-width: 0;
```

```
}

.login-intro {
  padding-top: 37px;
  text-align: center;
}

.login-intro>svg {
  color: var(--gold);
  stroke-width: .65;
}

.login-intro h1 {
  margin-top: 0;
  font-size: calc(26px * var(--text-scale));
  text-transform: uppercase;
  line-height: 1.2;
}

.login-intro p {
  margin-top: 11px;
  color: var(--muted);
  font-family: var(--font-display);
  font-size: calc(15px * var(--text-scale));
  line-height: 1.5;
}

.login-divider {
  display: flex;
  align-items: center;
  gap: 19px;
  margin: 28px 0 18px;
  color: var(--gold);
  font-size: calc(13px * var(--text-scale));
}

.login-divider::before,
.login-divider::after {
  content: '';
  flex: 1;
  height: 1px;
  background: var(--border);
}

.login-form {
  display: grid;
  gap: 16px;
}

.login-field {
  display: grid;
  gap: 6px;
}

.login-field>label {
  font-family: var(--font-display);
  font-size: calc(14px * var(--text-scale));
}

```

```
.login-input {
  position: relative;
  display: flex;
  align-items: center;
}

.login-input>svg {
  position: absolute;
  left: 10px;
  color: var(--gold);
  pointer-events: none;
}

.login-input input {
  height: 42px;
  padding: 10px 38px 10px 34px;
  border-color: var(--border);
  border-radius: 2px;
  background: var(--input-background);
  font-size: calc(10px * var(--text-scale));
}

.login-input input::placeholder {
  color: var(--muted);
  opacity: 1;
}

.login-input input[aria-invalid="true"] {
  border-color: var(--error);
}

.login-password-toggle {
  position: absolute;
  right: 0;
  display: grid;
  place-items: center;
  width: 38px;
  height: 42px;
  padding: 0;
  background: transparent;
  color: var(--muted);
}

.login-options {
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: 6px 12px;
  margin-top: -7px;
  font-size: calc(9px * var(--text-scale));
}

.login-remember {
  display: flex;
  align-items: center;
  gap: 7px;
  min-height: 28px;
  color: var(--muted);
```

```
  cursor: pointer;
}

.login-remember input[type="checkbox"] {
  appearance: none;
  width: 12px;
  height: 12px;
  margin: 0;
  padding: 0;
  border: 1px solid var(--gold);
  border-radius: 2px;
  background: transparent;
}

.login-remember input:checked {
  background: var(--gold);
  box-shadow: inset 0 0 0 2px var(--background);
}

.login-options .login-recovery {
  display: inline-flex;
  align-items: center;
  min-height: 28px;
  padding: 0;
  background: transparent;
  color: var(--gold);
  font-size: inherit;
}

.login-form .login-submit {
  min-height: 40px;
  margin-top: -10px;
  gap: 8px;
  font-size: calc(16px * var(--text-scale));
}

.login-account-divider {
  gap: 10px;
  margin: 20px 0 19px;
  color: var(--muted);
  font-family: var(--font-display);
  font-size: calc(13px * var(--text-scale));
}

.login-create-account {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 40px;
  border: 1px solid var(--border);
  border-radius: 2px;
  background: var(--surface);
  font-family: var(--font-display);
  font-size: calc(16px * var(--text-scale));
}

.login-create-account:hover {
  background: var(--surface-raised);
```

```
}

.login-terms {
  margin-top: 20px;
  color: var(--muted);
  text-align: center;
  font-size: calc(8px * var(--text-scale));
  line-height: 1.7;
}

.login-motto {
  margin-top: auto;
  padding: 40px 0 17px;
  color: var(--muted);
  text-align: center;
  font-family: var(--font-display);
  font-size: calc(13px * var(--text-scale));
  font-style: italic;
}

@media (max-width: 480px) {
  .app-frame.login-frame {
    max-width: 390px;
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 35 DE 62

Accessibility.css

`src/pages/Accessibility.css`

Presentación del menú y los controles de accesibilidad.

- Da forma a las filas de navegación, el perfil activo, radios, marcas de selección, muestras de color, deslizador y vista previa.
- La tabla keyboard-shortcuts describe atajos. Todos los estilos heredan las variables del tema, el contraste y el tamaño elegido.

Código completo

294 líneas · SHA-256 del original: `39ea18472a3037f82f55fbd59d189b02c2c15b003882d15e7d22481db5147dfc`

```
.accessibility-page {
  display: grid;
  gap: 22px;
  padding-bottom: 30px;
}

.settings-back {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  min-height: var(--control-height);
  color: var(--accent-ink);
  font-size: calc(13px * var(--text-scale));
}

.settings-profile {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  padding: 15px;
  border: 1px solid var(--border);
  background: var(--surface);
}

.settings-profile>svg {
  flex-shrink: 0;
  color: var(--gold);
}

.settings-profile p {
  font-size: calc(13px * var(--text-scale));
  line-height: 1.7;
}

.settings-profile small {
  display: block;
```

```
  color: var(--muted);
  font-size: calc(11px * var(--text-scale));
}

.settings-description {
  color: var(--muted);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.8;
}

.settings-menu {
  display: grid;
  gap: 10px;
}

.settings-menu-item {
  display: flex;
  gap: 12px;
  align-items: center;
  min-height: var(--control-height);
  padding: var(--control-padding);
  border: 1px solid var(--border);
  border-radius: 4px;
  background: var(--surface);
}

.settings-menu-item:hover {
  background: var(--surface-raised);
}

.settings-menu-item>span:nth-child(2) {
  min-width: 0;
  flex: 1;
}

.settings-menu-item>svg {
  flex-shrink: 0;
  color: var(--gold);
}

.settings-icon {
  display: grid;
  place-items: center;
  width: 38px;
  min-height: 44px;
  color: var(--gold);
  flex-shrink: 0;
}

.settings-menu-item strong {
  display: block;
  font-size: calc(14px * var(--text-scale));
  line-height: 1.5;
  font-weight: 500;
}

.settings-menu-item small {
  display: block;
  margin-top: 5px;
```

```
  font-size: calc(11px * var(--text-scale));
  line-height: 1.6;
  color: var(--muted);
}

.settings-current {
  display: block;
  margin-top: 8px;
  color: var(--accent-ink);
  font-size: calc(11px * var(--text-scale));
  line-height: 1.6;
}

.settings-choices {
  min-width: 0;
  display: grid;
  gap: 10px;
  border: 0;
  padding: 0;
  margin: 0;
}

.settings-choices legend {
  margin-bottom: 14px;
  color: var(--gold);
  font-size: calc(16px * var(--text-scale));
  line-height: 1.5;
}

.choice-card {
  display: flex;
  align-items: center;
  gap: 10px;
  min-height: var(--control-height);
  padding: var(--control-padding);
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 4px;
  cursor: pointer;
}

.choice-card.is-selected {
  border-color: var(--accent-ink);
  box-shadow: inset 3px 0 var(--accent-ink);
  background: var(--surface-raised);
}

.choice-card>input {
  width: 18px;
  height: 18px;
  padding: 0;
  margin: 0;
  flex-shrink: 0;
}

.choice-card>span:not(.color-swatch) {
  flex: 1;
  min-width: 0;
}
```

```
155
156 .choice-card strong {
157   font-size: calc(13px * var(--text-scale));
158   font-weight: 500;
159   line-height: 1.6;
160 }
161
162 .choice-card small {
163   display: block;
164   color: var(--muted);
165   font-size: calc(11px * var(--text-scale));
166   line-height: 1.7;
167   margin-top: 5px;
168 }
169
170 .choice-card>svg {
171   flex-shrink: 0;
172   color: var(--accent-ink);
173 }
174
175 .color-swatch {
176   display: block;
177   width: 22px;
178   height: 22px;
179   flex-shrink: 0;
180   border-radius: 50%;
181   border: 1px solid var(--foreground);
182 }
183
184 .settings-preview {
185   display: grid;
186   gap: 12px;
187   padding: 18px;
188   border: 1px solid var(--border);
189   background: var(--surface);
190   border-radius: 4px;
191 }
192
193 .settings-preview p:not(.eyebrow) {
194   font-size: calc(13px * var(--text-scale));
195   line-height: 1.8;
196 }
197
198 .settings-preview .preview-tags {
199   color: var(--gold);
200 }
201
202 .text-size-slider {
203   display: grid;
204   gap: 16px;
205   color: var(--muted);
206   font-size: calc(13px * var(--text-scale));
207   line-height: 1.7;
208 }
209
210 .text-size-slider>span {
211   display: flex;
212   align-items: center;
213   gap: 14px;
```

```
}

.text-size-slider>span>span:last-child {
  font-size: calc(25px * var(--text-scale));
}

.text-size-slider input {
  flex: 1;
  width: 100%;
  min-height: var(--control-height);
  padding: 0;
  accent-color: var(--accent);
}

.settings-toggle {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  min-height: var(--control-height);
  padding: var(--control-padding);
  border: 1px solid var(--border);
  background: var(--surface);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.8;
  cursor: pointer;
}

.settings-toggle small {
  display: block;
  color: var(--muted);
  font-size: calc(11px * var(--text-scale));
}

.keyboard-shortcuts {
  width: 100%;
  border-collapse: collapse;
  table-layout: fixed;
  font-size: calc(12px * var(--text-scale));
  line-height: 1.8;
}

.keyboard-shortcuts caption {
  text-align: left;
  color: var(--gold);
  font-size: calc(16px * var(--text-scale));
  margin-bottom: 12px;
}

.keyboard-shortcuts th,
.keyboard-shortcuts td {
  padding: 12px 6px;
  border-bottom: 1px solid var(--border);
  text-align: left;
  overflow-wrap: anywhere;
}

.keyboard-shortcuts th {
  width: 45%;
  font-weight: 400;
```

```
}

.keyboard-shortcuts kbd {
  display: inline-block;
  padding: 2px 5px;
  margin: 2px 0;
  border: 1px solid var(--border);
  border-radius: 3px;
  background: var(--surface);
  color: var(--foreground);
  font: inherit;
}

@media (max-width: 350px) {
  .settings-menu-item {
    gap: 8px;
  }

  .settings-icon {
    width: 27px;
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 36 DE 62

package.json

package.json

Identidad del paquete, dependencias directas y comandos de desarrollo.

- React y React DOM componen la interfaz; React Router gestiona las rutas; Vite desarrolla y compila; ESLint comprueba el código.
- Los comandos son dev, build, test, test:browser, lint y preview. Las versiones con ^ son rangos declarados; package-lock.json concreta la resolución instalada.

Código completo

30 líneas · SHA-256 del original: `9bd88f61dd2f00d8cd60f4a9885a90e18f8772c2aa0751bca8a863a7d448f81b`

```
{
  "name": "proyecto-interdisciplinario---umbra",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "node --test",
    "test:browser": "node scripts/browser-smoke.mjs",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.3.1",
    "react-dom": "^19.3.1",
    "react-router": "^7.14.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "@vitejs/plugin-react": "^6.1.2",
    "eslint": "^10.13.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.7",
    "globals": "^17.13.0",
    "vite": "^8.3.4"
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 37 DE 62

vite.config.js

`vite.config.js`

Plugin React, proxy de API de desarrollo y exclusión de fuentes del observador.

- server.proxy reenvía /api a http://localhost:8080 con changeOrigin: true y sin rewrite.
- server.watch.ignored excluye **/public/fonts/** para evitar bloqueos de vigilancia en Windows.
- El proxy no se incluye en dist. Vite hereda server.proxy para la vista previa local; VITE_API_URL no se lee en api.js.

Funciones y elementos principales

Nombre	Qué hace	Línea
`defineConfig`	Define la configuración exportada de Vite.	—

Código completo

17 líneas · SHA-256 del original: `3fa45147c9e6f1cb1a625f1ad3b19adc095d764c02a955611bee0dccdd3caa83`

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
      },
    },
    watch: {
      ignored: ['**/public/fonts/**'],
    },
  },
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 38 DE 62

eslint.config.js

`eslint.config.js`

Reglas de análisis estático para JavaScript y JSX.

- Ignora dist y combina recomendaciones de JavaScript, hooks de React y React Refresh.
- Declara globals de navegador y soporte para JSX. No ejecuta la app ni sustituye las pruebas de comportamiento.

Funciones y elementos principales

Nombre	Qué hace	Línea
`defineConfig`	Compone el array de reglas de ESLint.	—
`globalIgnores`	Indica los directorios que se excluyen del análisis.	—

Código completo

21 líneas · SHA-256 del original: `9f9b44b8e55768eaf86e417b96919510b12055d87806e0e2f0703c5a0eb64817`

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
])
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 39 DE 62

characterStore.test.js

`tests/characterStore.test.js`

Pruebas del almacenamiento y las copias de personajes.

- Comprueba aislamiento de bibliotecas, persistencia de listas vacías, importación válida e inválida, ids repetidos y que un fallo de escritura conserve datos anteriores.

Funciones y elementos principales

Nombre	Qué hace	Línea
`memoryStorage`	Crea un almacenamiento de prueba basado en Map para no usar datos reales de navegador.	6

Código completo

43 líneas · SHA-256 del original: `66e4796c990f66f5da9eeda449a4a953c822d30de4ffee8d866dc8e2d95f12cd`

```
1  import assert from 'node:assert/strict'
2  import test from 'node:test'
3  import { initialCharacters } from '../src/data/characters.js'
4  import { importCharacters, normalizeCharacter, readCharacters, writeCharacters } from
   '../src/utils/characterStore.js'
5
6  function memoryStorage() {
7    const data = new Map()
8    return { getItem: key => data.has(key) ? data.get(key) : null, setItem: (key, value) => data.set(key,
   value) }
9  }
10
11 test('una biblioteca vacía persiste y las cuentas conservan bibliotecas independientes', () => {
12   const storage = memoryStorage()
13   assert.equal(readCharacters('guest', storage).length, 6)
14   writeCharacters('guest', [], storage)
15   assert.deepEqual(readCharacters('guest', storage), [])
16   assert.equal(readCharacters('account', storage).length, 6)
17   const character = normalizeCharacter({ ...initialCharacters[0], name: 'Nueva leyenda', favorite: true
   })
18   writeCharacters('account', [character], storage)
19   assert.deepEqual(readCharacters('account', storage), [character])
20   assert.deepEqual(readCharacters('guest', storage), [])
21 })
22
23 test('una copia restaura los datos y rechaza identificadores repetidos o contenido inválido', () => {
24   const character = normalizeCharacter(initialCharacters[0])
25   const backup = characters => JSON.stringify({ app: 'UMBRA', version: 1, characters })
26   assert.deepEqual(importCharacters(backup([character])), [character])
```

```
  assert.deepEqual(importCharacters(backup([])), [])
  assert.throws(() => importCharacters('no es JSON'))
  assert.throws(() => importCharacters(JSON.stringify({ version: 2, characters: [] })))
  assert.throws(() => importCharacters(backup([character, character])), /repetidos/)
  for (const changes of [{ level: 21 }, { stats: [1, 2, 3, 99] }, { traits: ['a', 'b', 'c', 'd', 'e', 'f', 'g'] }, { type: 'Inventada' }, { imageDataUrl: 'javascript:alert(1)' }]) {
    assert.throws(() => importCharacters(backup([{ ...character, ...changes }])))
  }
})

test('un fallo de almacenamiento no destruye la copia anterior', () => {
  const storage = memoryStorage()
  writeCharacters('guest', initialCharacters, storage)
  const before = readCharacters('guest', storage)
  const blocked = { ...storage, setItem: () => { throw new Error('QuotaExceeded') } }
  assert.throws(() => writeCharacters('guest', [], blocked))
  assert.deepEqual(readCharacters('guest', storage), before)
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 40 DE 62

localAccounts.test.js

`tests/localAccounts.test.js`

Pruebas del módulo de cuentas locales anterior; no prueban la autenticación de Spring Boot.

- Comprueba correos duplicados, contraseñas erróneas, ausencia de secretos en texto plano, sesión temporal, Recordarme y cierre de sesión.
- Verifica que el código es necesario para recuperar y que cambiar la contraseña conserva el id de la cuenta.

Funciones y elementos principales

Nombre	Qué hace	Línea
`memoryStorage`	Simula localStorage y sessionStorage mediante Map.	5

Código completo

46 líneas · SHA-256 del original: `f9c74b3be3d6b928482ef505aff66a27eebf49e46cd6689ac47108c804688b8c`

```
1  import assert from 'node:assert/strict'
2  import test from 'node:test'
3  import { loginAccount, logoutAccount, readSession, recoverAccount, registerAccount } from
   '../src/utils/localAccounts.js'
4
5  function memoryStorage() {
6    const data = new Map()
7    return { getItem: key => data.get(key) ?? null, setItem: (key, value) => data.set(key, value),
   removeItem: key => data.delete(key) }
8  }
9
10 test('registro, verificación de contraseña, sesiones temporales y Recordarme', async () => {
11   const storage = memoryStorage()
12   const temporary = memoryStorage()
13   const data = { name: 'Elara', email: 'Elara@Umbra.es', password: 'Historia1!' }
14   const code = await registerAccount(data, storage)
15   assert.equal(code.length, 32)
16   const serialized = storage.getItem('umbra.accounts.v1')
17   assert.ok(!serialized.includes(data.password))
18   assert.ok(!serialized.includes(code))
19   await assert.rejects(registerAccount({ ...data, email: 'elara@umbra.es' }, storage), /Ya existe/)
20   await assert.rejects(loginAccount({ ...data, password: 'incorrecta' }, storage, temporary), /no
   coinciden/)
21   const user = await loginAccount(data, storage, temporary)
22   assert.equal(user.name, 'Elara')
```

```
  assert.deepEqual(readSession(storage, temporary), user)
  assert.equal(readSession(storage, memoryStorage()), null)
  await loginAccount({ ...data, remember: true }, storage, temporary)
  assert.deepEqual(readSession(storage, memoryStorage()), user)
  assert.equal(temporary.getItem('umbra.session.v1'), null)
  await loginAccount({ ...data, remember: false }, storage, temporary)
  assert.equal(storage.getItem('umbra.session.v1'), null)
  logoutAccount(storage, temporary)
  assert.equal(readSession(storage, temporary), null)
})

test('recuperación requiere el código y cambia la contraseña conservando la identidad', async () => {
  const storage = memoryStorage()
  const temporary = memoryStorage()
  const data = { name: 'Kael', email: 'kael@umbra.es', password: 'Antigua1!' }
  const code = await registerAccount(data, storage)
  const original = await loginAccount({ ...data, remember: true }, storage, temporary)
  await assert.rejects(recoverAccount({ ...data, recoveryCode: 'incorrecto' }, storage, temporary), /no
son correctos/)
  const changed = { ...data, password: 'OtraClave2!' }
  await recoverAccount({ ...changed, recoveryCode: code.toLowerCase() }, storage, temporary)
  assert.equal(readSession(storage, temporary), null)
  await assert.rejects(loginAccount(data, storage, temporary))
  assert.equal((await loginAccount(changed, storage, temporary)).id, original.id)
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 41 DE 62

preferenceStore.test.js

`tests/preferenceStore.test.js`

Pruebas de accesibilidad independiente por perfil.

- Comprueba dos cuentas e Invitado, restablecimiento aislado, migración de la antigua configuración global y rechazo de valores desconocidos.
- También verifica que un fallo de escritura no borre la configuración persistida anterior.

Funciones y elementos principales

Nombre	Qué hace	Línea
`memoryStorage`	Proporciona un almacén aislado para cada escenario de prueba.	5

Código completo

42 líneas · SHA-256 del original: `44f18b3026203a6c61dcb97cf28c841a10937f78486391c25a6e6e77810d4a53`

```
1  import assert from 'node:assert/strict'
2  import test from 'node:test'
3  import { defaultPreferences, normalizePreferences, readPreferences, writePreferences } from
   '../src/utils/preferenceStore.js'
4
5  function memoryStorage() {
6    const values = new Map()
7    return { getItem: key => values.get(key) ?? null, setItem: (key, value) => values.set(key, value) }
8  }
9
10 test('cada perfil y el invitado conservan todos sus ajustes independientes', () => {
11   const storage = memoryStorage()
12   const first = { ...defaultPreferences, theme: 'sepia', textSize: 'extra-large', contrast: 'high',
   accent: 'purple', elementSize: 'large', readAloud: true, readingRate: 0.75, enhancedFocus: true,
   announceNavigation: false, reduceMotion: true }
13   writePreferences('account-a', first, storage)
14   writePreferences('account-b', { ...defaultPreferences, theme: 'light', accent: 'blue' }, storage)
15   writePreferences('guest', { ...defaultPreferences, textSize: 'small' }, storage)
16   assert.deepEqual(readPreferences('account-a', storage), first)
17   assert.equal(readPreferences('account-b', storage).theme, 'light')
18   assert.equal(readPreferences('guest', storage).textSize, 'small')
19   assert.deepEqual(readPreferences('new-account', storage), defaultPreferences)
20   writePreferences('account-a', defaultPreferences, storage)
21   assert.equal(readPreferences('account-b', storage).theme, 'light')
22   assert.equal(readPreferences('guest', storage).textSize, 'small')
23 })
24
25 test('los ajustes globales antiguos se recuperan solo para el invitado', () => {
```

```
  const storage = memoryStorage()
  storage.setItem('umbra.accessibility.v1', JSON.stringify({ highContrast: true, largeText: true }))
  assert.equal(readPreferences('guest', storage).contrast, 'high')
  assert.equal(readPreferences('guest', storage).textSize, 'large')
  assert.deepEqual(readPreferences('account-a', storage), defaultPreferences)
  writePreferences('guest', defaultPreferences, storage)
  assert.deepEqual(readPreferences('guest', storage), defaultPreferences)
})

test('los valores desconocidos no se aplican y el fallo de guardado no sobrescribe ajustes anteriores', () => {
  const storage = memoryStorage()
  assert.deepEqual(normalizePreferences({ textSize: 'huge', theme: 'unknown', accent: 'invalid', readingRate: 99 }), defaultPreferences)
  assert.deepEqual(normalizePreferences(null), defaultPreferences)
  writePreferences('account-a', defaultPreferences, storage)
  assert.throws(() => writePreferences('account-a', { ...defaultPreferences, theme: 'sepia' }, { ...storage, setItem: () => { throw new Error('Storage blocked') } }))
  assert.deepEqual(readPreferences('account-a', storage), defaultPreferences)
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 42 DE 62

validateForm.test.js

`tests/validateForm.test.js`

Pruebas de las reglas compartidas de formularios.

- Incluye campos vacíos, correos, longitud y carácter especial de contraseña, confirmación, nivel entero entre 1 y 20 y casilla obligatoria.

Funciones y elementos principales

Nombre	Qué hace	Línea
`values`	Construye FormData a partir de un objeto sencillo para cada caso de prueba.	5

Código completo

50 líneas · SHA-256 del original: `f86dec842e30062c94bbfc7924821830f5054fcd34d816c21bdc79b58d2a24ee`

```
import assert from 'node:assert/strict'
import test from 'node:test'
import { validateForm } from '../src/utils/validateForm.js'

function values(entries) {
  const data = new FormData()
  for (const [name, value] of Object.entries(entries)) data.set(name, value)
  return data
}

test('un nombre vacío o compuesto de espacios requiere una corrección', () => {
  const fields = [{ name: 'name', label: 'Nombre', required: true }]
  assert.ok(validateForm(fields, values({ name: '   ' })).name)
  assert.deepEqual(validateForm(fields, values({ name: 'Elara Valen' })), {})
})

test('el correo exige usuario, arroba y dominio', () => {
  const fields = [{ name: 'email', type: 'email', required: true }]
  for (const email of ['elara', 'elara@', '@umbra.es', 'elara@umbra', 'elara @umbra.es']) {
    assert.ok(validateForm(fields, values({ email })).email, email)
  }
  assert.deepEqual(validateForm(fields, values({ email: 'elara@umbra.es' })), {})
})

test('el registro exige longitud suficiente, un carácter especial y contraseñas iguales', () => {
  const fields = [
    { name: 'password', required: true, minLength: 8, requireSpecial: true },
    { name: 'confirmPassword', required: true, matches: 'password' },
  ]
```

```
30   assert.ok(validateForm(fields, values({ password: 'corta!', confirmPassword: 'corta!' })).password)
31   for (const password of ['prueba123', 'prueba 123', 'contraseña123']) {
32     assert.ok(validateForm(fields, values({ password, confirmPassword: password })).password, password)
33   }
34   assert.ok(validateForm(fields, values({ password: 'prueba1!', confirmPassword: 'diferente'
})).confirmPassword)
35   for (const password of ['prueba1!', 'prueba1@', 'prueba1#', 'prueba1$']) {
36     assert.deepEqual(validateForm(fields, values({ password, confirmPassword: password })), {})
37   }
38 })
39
40 test('el nivel debe ser entero y estar entre 1 y 20', () => {
41   const fields = [{ name: 'level', type: 'number', required: true, min: 1, max: 20 }]
42   for (const level of ['', '0', '21', '2.5', 'letras']) assert.ok(validateForm(fields, values({ level
})).level)
43   for (const level of ['1', '8', '20']) assert.deepEqual(validateForm(fields, values({ level })), {})
44 })
45
46 test('una casilla obligatoria debe estar marcada', () => {
47   const fields = [{ name: 'consent', type: 'checkbox', required: true }]
48   assert.ok(validateForm(fields, values({})).consent)
49   assert.deepEqual(validateForm(fields, values({ consent: 'on' })), {})
50 })
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 43 DE 62

browser-smoke.mjs

`scripts/browser-smoke.mjs`

Recorridos de integración en Chrome o Edge mediante Chrome DevTools Protocol, sin dependencia de Playwright.

- Pendiente de adaptar: el recorrido de cuentas espera el registro local anterior, el código de recuperación y etiquetas antiguas. No demuestra compatibilidad con la API ni prueba los personajes públicos de extremo a extremo.
- Necesita la app ejecutándose y un navegador con CDP en el puerto 9222. Crea un contexto aislado y lo elimina al terminar; las limpiezas de almacenamiento ocurren dentro de ese contexto.
- Prueba personajes, filtros, retratos, favoritos, duplicado, eliminación, fallos de guardado, copias, cuentas y recuperación.
- Comprueba los cinco menús, los cambios de texto y contraste, el árbol de accesibilidad, el foco, los tamaños, el cambio entre perfiles y el diseño de 320/390 px.
- La voz se sustituye por una simulación en algunos recorridos para comprobar texto, velocidad y cancelación; esos casos no verifican la reproducción de sonido real ni un lector de pantalla externo.
- UMBRA_URL elige la dirección de la app. UMBRA_SCREENSHOT permite guardar una captura. Los fallos producen un código de salida distinto de cero.

Funciones y elementos principales

Nombre	Qué hace	Línea
`send`	Envía una orden CDP con id y sesión y relaciona su respuesta con una promesa.	27
`evaluate`	Ejecuta JavaScript en la página y devuelve el valor o su error.	34
`wait`	Consulta una condición de la página hasta cumplirla o agotar el plazo.	39
`click`	Enfoca y pulsa un elemento encontrado mediante selector.	46
`clickText`	Encuentra un botón o enlace por texto, lo enfoca y lo pulsa.	51
`fill`	Rellena un control identificado por name.	57
`fillSelector`	Utiliza el setter nativo y eventos para actualizar inputs o textareas de React.	60

Nombre	Qué hace	Línea
`select`	Cambia un select y dispara su evento change.	63
`importFile`	Simula la selección de una copia JSON con File y DataTransfer.	66
`go`	Carga una ruta y espera a que aparezca el contenido principal.	69
`reload`	Recarga la página y espera a que React vuelva a montarse.	73

Código completo

515 líneas · SHA-256 del original: `667cf05d3460b4a5b4e8047f05bb6856d9f1bb0da3120f6d79cd745de215d0f2`

```
// Requires Chrome/Edge with CDP on port 9222. Uses an isolated, disposable context.
import assert from 'node:assert/strict'
import { setTimeout as delay } from 'node:timers/promises'
import { writeFile } from 'node:fs/promises'
import { Buffer } from 'node:buffer'
import process from 'node:process'

const origin = process.env.UMBRA_URL || 'http://127.0.0.1:5173'
const browser = await (await fetch('http://127.0.0.1:9222/json/version')).json()
const socket = new WebSocket(browser.webSocketDebuggerUrl)
await new Promise((resolve, reject) => { socket.onopen = resolve; socket.onerror = reject })
let sequence = 0
let sessionId
let browserContextId
const pending = new Map()
const exceptions = []
socket.onmessage = event => {
  const message = JSON.parse(event.data)
  if (message.method === 'Runtime.exceptionThrown') exceptions.push(message.params.exceptionDetails)
  if (pending.has(message.id)) {
    const { resolve, reject } = pending.get(message.id)
    pending.delete(message.id)
    if (message.error) reject(new Error(message.error.message))
    else resolve(message.result)
  }
}
function send(method, params = {}, session = sessionId) {
  return new Promise((resolve, reject) => {
    const id = ++sequence
    pending.set(id, { resolve, reject })
    socket.send(JSON.stringify({ id, method, params, ...(session ? { sessionId: session } : {}) }))
  })
}
async function evaluate(expression) {
  const result = await send('Runtime.evaluate', { expression, returnByValue: true, awaitPromise: true })
  if (result.exceptionDetails) throw new Error(JSON.stringify(result.exceptionDetails))
  return result.result.value
}
async function wait(expression) {
  for (let index = 0; index < 100; index++) {
```

```
    if (await evaluate(expression)) return
    await delay(100)
  }
  throw new Error(`Timed out: ${expression}\n${await evaluate('document.body.innerText')}`)
}
async function click(selector) {
  await wait(`Boolean(document.querySelector(${JSON.stringify(selector)}))`)
  await evaluate(`(() => { const el = document.querySelector(${JSON.stringify(selector)}); el.focus(); el.click(); })()`)
  await delay(80)
}
async function clickText(text) {
  const expression = `[...document.querySelectorAll('button,a')].find(el => el.textContent.trim() === ${JSON.stringify(text)})`
  await wait(`Boolean(${expression})`)
  await evaluate(`(() => { const el = ${expression}; el.focus(); el.click(); })()`)
  await delay(80)
}
async function fill(name, value) {
  await fillSelector(`[name="${name}"]`, value)
}
async function fillSelector(selector, value) {
  await evaluate(`(() => { const el = document.querySelector(${JSON.stringify(selector)}); const proto = el.tagName === 'TEXTAREA' ? HTMLTextAreaElement.prototype : HTMLInputElement.prototype; Object.getOwnPropertyDescriptor(proto, 'value').set.call(el, ${JSON.stringify(value)}); el.dispatchEvent(new Event('input', { bubbles: true })); el.dispatchEvent(new Event('change', { bubbles: true })); })()`)
}
async function select(selector, value) {
  await evaluate(`(() => { const el = document.querySelector(${JSON.stringify(selector)}); el.value = ${JSON.stringify(value)}; el.dispatchEvent(new Event('change', {bubbles:true})); })()`)
}
async function importFile(contents) {
  await evaluate(`(() => { const transfer = new DataTransfer(); transfer.items.add(new File([${JSON.stringify(contents)}], 'backup.json', {type:'application/json'})); const input = document.querySelector('input[type=file]'); input.files = transfer.files; input.dispatchEvent(new Event('change', {bubbles:true})); })()`)
}
async function go(path) {
  await send('Page.navigate', { url: origin + path })
  await wait(`location.pathname === ${JSON.stringify(path)} && Boolean(document.querySelector('.main-content h1'))`)
}
async function reload() {
  await send('Page.reload')
  await delay(300)
  await wait("Boolean(document.querySelector('.main-content h1'))")
}
async function inspirationDraft() {
  return evaluate(`(() => {
    const field = key => document.querySelector('.inspiration-field[data-field=' + key + ']');
    const input = name => document.querySelector('.inspiration-page [name=' + name + ']').value;
    return {
      name: input('name'), type: input('type'), race: input('race'),
      level: Number(input('level')), realm: input('realm'), portrait: Number(input('portrait')),
      story: input('story'),
      traits: [...document.querySelectorAll('.inspiration-traits li')].map(item => item.textContent),
      stats: [0, 1, 2, 3].map(index => Number(input('stat' + index))),
    };
```

```
  })()`)
}

try {
  ;({ browserContextId } = await send('Target.createBrowserContext', { disposeOnDetach: true }, null))
  const { targetId } = await send('Target.createTarget', { url: 'about:blank', browserContextId }, null)
  ;({ sessionId } = await send('Target.attachToTarget', { targetId, flatten: true }, null))
  await send('Runtime.enable')
  await send('Page.enable')
  await send('Page.bringToFront')
  await send('Emulation.setDeviceMetricsOverride', { width: 390, height: 844, deviceScaleFactor: 1, mobile: true })
  await send('Emulation.setTouchEmulationEnabled', { enabled: true })
  await go('/inicio')
  await evaluate('localStorage.clear(); sessionStorage.clear()')
  await reload()
  assert.equal(await evaluate("document.querySelectorAll('.character-card').length"), 2)
  await click('.character-card[aria-label*="Elara"]')
  await wait("location.pathname === '/biblioteca/elara' && Boolean(document.querySelector('.character-detail'))")
  assert.equal(await evaluate("document.querySelectorAll('dialog').length"), 0)
  await clickText('Volver a Inicio')
  await wait("location.pathname === '/inicio'")
  await click('.bottom-navigation a[href="/biblioteca"]')
  await wait("document.querySelectorAll('.character-card').length === 6")
  await fillSelector('input[type=search]', 'angelica')
  await wait("document.querySelectorAll('.character-card').length === 1")
  await fillSelector('input[type=search]', 'nada coincide')
  await wait("document.querySelectorAll('.character-card').length === 0")
  await clickText('Limpiar filtros')
  await wait("document.querySelectorAll('.character-card').length === 6")
  await clickText('+ Crear personaje')
  await clickText('Guardar personaje')
  await wait("Boolean(document.querySelector('[aria-invalid=true]'))")
  await fill('name', 'Ayla de Prueba')
  await fill('realm', 'Bosque de Niebla')
  const longStory = 'Una historia que debe persistir.\n' + 'Ayla camina entre las ruinas y recuerda una leyenda perdida.\n'.repeat(40)
  await fill('story', longStory)
  await click('input[name="traits"][value="Leal"]')
  await evaluate("(() => { const transfer = new DataTransfer(); transfer.items.add(new File(['invalid'], 'portrait.svg', {type:'image/svg+xml'})); const input = document.querySelector('input[type=file]'); input.files = transfer.files; input.dispatchEvent(new Event('change', {bubbles:true})); })()")
  await wait("document.body.innerText.includes('Selecciona una imagen PNG')")
  await evaluate("(async () => { const canvas = document.createElement('canvas'); canvas.width = 4; canvas.height = 4; canvas.getContext('2d').fillRect(0,0,4,4); const blob = await new Promise(resolve => canvas.toBlob(resolve)); const transfer = new DataTransfer(); transfer.items.add(new File([blob], 'portrait.png', {type:'image/png'})); const input = document.querySelector('input[type=file]'); input.files = transfer.files; input.dispatchEvent(new Event('change', {bubbles:true})); })()")
  await wait("Boolean(document.querySelector('.portrait-editor img[src^=\"data:image/png\"]'))")
  await clickText('Guardar personaje')
  await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length === 7")
  await click('.character-card[aria-label*="Ayla"]')
  await wait("location.pathname.startsWith('/biblioteca/') && Boolean(document.querySelector('.character-detail'))")
  const detailPath = await evaluate('location.pathname')
  assert.equal(await evaluate("document.querySelectorAll('dialog').length"), 0)
```

```
136   assert.ok((await evaluate('document.querySelector(".character-detail").innerText')).includes('Una
    historia que debe persistir.'))
137   assert.ok(await evaluate("document.querySelector('.character-detail
    img').src.startsWith('data:image/png')"))
138   assert.ok(await evaluate("document.querySelector('.bottom-navigation
    a[href=\"/biblioteca\"]').getAttribute('aria-current') === 'page'"))
139   assert.ok((await evaluate('document.title')).includes('Ayla de Prueba'))
140   await send('Input.dispatchMouseEvent', { type: 'mouseWheel', x: 180, y: 600, deltaX: 0, deltaY: 600 })
141   await wait('window.scrollY > 100')
142   assert.ok(await evaluate("document.querySelector('.character-detail').scrollHeight > innerHeight"))
143   await reload()
144   await wait("Boolean(document.querySelector('.character-detail'))")
145   assert.equal(await evaluate('location.pathname'), detailPath)
146   await clickText('Guardar en favoritos')
147   await wait("document.querySelector('.character-detail [aria-pressed=true]') !== null")
148   await clickText('Volver a Biblioteca')
149   await wait("location.pathname === '/biblioteca'")
150   await click('.preference-option input')
151   await wait("document.querySelectorAll('.character-card').length === 1")
152   await fillSelector('input[type=search]', 'Ayla')
153   await select('.library-filters select', 'Guerrero')
154   await select('.library-filters label:nth-child(2) select', 'level')
155   const filteredLibrary = await evaluate('location.pathname + location.search')
156   await click('.character-card[aria-label*="Ayla"]')
157   await wait("Boolean(document.querySelector('.character-detail'))")
158   const history = await send('Page.getNavigationHistory')
159   await send('Page.navigateToHistoryEntry', { entryId: history.entries[history.currentIndex - 1].id })
160   await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length
    === 1")
161   assert.equal(await evaluate('location.pathname + location.search'), filteredLibrary)
162   await click('.character-card[aria-label*="Ayla"]')
163   await wait("Boolean(document.querySelector('.character-detail'))")
164   await clickText('Volver a Biblioteca')
165   await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length
    === 1")
166   assert.equal(await evaluate('location.pathname + location.search'), filteredLibrary)
167   await click('.preference-option input')
168   await fillSelector('input[type=search]', '')
169   await select('.library-filters select', '')
170   await select('.library-filters label:nth-child(2) select', 'recent')
171   await click('.character-card[aria-label*="Ayla"]')
172   await clickText('Editar personaje')
173   await wait("location.pathname.includes('/editar')")
174   await fill('name', 'Ayla Editada')
175   await clickText('Guardar cambios')
176   await wait("location.pathname === '/biblioteca' && document.body.innerText.includes('Ayla Editada')")
177   await reload()
178   assert.ok((await evaluate('document.body.innerText')).includes('Ayla Editada'))
179   await click('.character-card[aria-label*="Ayla"]')
180   await clickText('Duplicar personaje')
181   await wait("document.querySelectorAll('.character-card').length === 8")
182   await click('.character-card[aria-label*="copia"]')
183   await clickText('Eliminar personaje')
184   await wait("Boolean(document.querySelector('.character-delete-confirmation'))")
185   assert.equal(await evaluate("document.querySelectorAll('dialog').length"), 0)
186   await send('Input.dispatchKeyEvent', { type: 'rawKeyDown', key: 'Escape', code: 'Escape',
    windowsVirtualKeyCode: 27, nativeVirtualKeyCode: 27 })
187   await send('Input.dispatchKeyEvent', { type: 'keyUp', key: 'Escape', code: 'Escape',
    windowsVirtualKeyCode: 27, nativeVirtualKeyCode: 27 })
```

```
  await wait("!document.querySelector('.character-delete-confirmation')")
  await clickText('Eliminar personaje')
  await clickText('Conservar personaje')
  await clickText('Eliminar personaje')
  await evaluate("window.umbraSetItem = Storage.prototype.setItem; Storage.prototype.setItem = () => { throw new DOMException('Quota exceeded', 'QuotaExceededError') }")
  await clickText('Eliminar definitivamente')
  await wait("document.querySelector('.character-detail').innerText.includes('No se pudo guardar')")
  assert.ok((await evaluate('location.pathname')).startsWith('/biblioteca/'))
  await evaluate('Storage.prototype.setItem = window.umbraSetItem')
  await clickText('Eliminar definitivamente')
  await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length === 7")
  await go('/crear-personaje')
  await fill('name', 'No debe guardarse')
  await evaluate("window.umbraSetItem = Storage.prototype.setItem; Storage.prototype.setItem = function() { throw new DOMException('Quota exceeded', 'QuotaExceededError') }")
  await clickText('Guardar personaje')
  await wait("document.body.innerText.includes('No se pudo guardar')")
  assert.equal(await evaluate('location.pathname'), '/crear-personaje')
  await evaluate('Storage.prototype.setItem = window.umbraSetItem')
  await clickText('Cancelar')
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Descartar y salir')
  await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length === 7")
  await go('/inspiracion')
  const libraryBeforeInspiration = await evaluate("localStorage.getItem('umbra.characters.v1.guest')")
  assert.equal(await evaluate("document.querySelectorAll('.inspiration-field').length"), 9)
  const firstIdea = await inspirationDraft()
  await clickText('Otra inspiración')
  const nextIdea = await inspirationDraft()
  for (const key of Object.keys(firstIdea)) assert.notDeepEqual(nextIdea[key], firstIdea[key], key)
  await click('button[aria-label="Recuperar idea anterior"]')
  assert.deepEqual(await inspirationDraft(), firstIdea)
  await click('button[aria-label="Cambiar atributos"]')
  const rerolledStats = await inspirationDraft()
  assert.notDeepEqual(rerolledStats.stats, firstIdea.stats)
  for (const key of Object.keys(firstIdea).filter(key => key !== 'stats')) assert.deepEqual(rerolledStats[key], firstIdea[key], key)
  await click('button[aria-label="Fijar nombre"]')
  await click('button[aria-label="Fijar atributos"]')
  await clickText('Otra inspiración')
  const lockedIdea = await inspirationDraft()
  assert.equal(lockedIdea.name, rerolledStats.name)
  assert.deepEqual(lockedIdea.stats, rerolledStats.stats)
  assert.notEqual(lockedIdea.story, rerolledStats.story)
  assert.ok(await evaluate("document.querySelector('button[aria-label=\"Cambiar atributos\"]').disabled"))
  await evaluate("[...document.querySelectorAll('button[aria-label^=\"Fijar \"]')].forEach(button => button.click())")
  await wait("document.querySelector('.inspiration-toolbar .primary-button').disabled")
  await clickText('Quitar bloqueos (9)')
  await wait("!document.querySelector('.inspiration-toolbar .primary-button').disabled")
  await clickText('Otra inspiración')
  await fill('name', '')
  await fill('level', '0')
  await fill('stat0', '21')
  await fill('story', 'x'.repeat(3001))
```

```
  await clickText('Guardar personaje')
  await wait("document.body.innerText.includes('Revisa los campos indicados antes de guardar')")
  assert.equal(await evaluate('location.pathname'), '/inspiracion')
  assert.equal(await evaluate('document.activeElement.name'), 'name')
  for (const name of ['name', 'level', 'stat0', 'story']) assert.equal(await evaluate(`document.querySelector('[name=${name}]').getAttribute('aria-invalid')`), 'true')
  await fill('name', 'Ayla de Tinta')
  await select('.inspiration-page [name=type]', 'Exploradora')
  await select('.inspiration-page [name=race]', 'Tiefling')
  await fill('level', '14')
  await fill('realm', 'Bosque Inventado')
  await fill('story', 'Esta historia la escribí directamente en Inspiración y debe conservarse al guardar.')
  for (const [index, value] of [18, 11, 16, 9].entries()) await fill(`stat${index}`, String(value))
  const oldTraits = (await inspirationDraft()).traits
  for (const trait of oldTraits) await click(`button[aria-label=${JSON.stringify(`Quitar rasgo ${trait}`)}]`)
  await click('.inspiration-trait-picker summary')
  await click('.inspiration-page input[name=traits][value="Leal"]')
  for (const trait of ['Curiosa', 'Observadora', 'Tenaz', 'Paciente', 'Soñadora']) {
    await fill('customTrait', trait)
    await clickText('Añadir')
  }
  assert.equal((await inspirationDraft()).traits.length, 6)
  assert.ok(await evaluate("document.querySelector('.inspiration-page input[name=traits][value=Valiente]').disabled"))
  await fill('customTrait', 'Séptimo rasgo')
  assert.ok(await evaluate("[...document.querySelectorAll('button')].find(button=>button.textContent==='Añadir').disabled"))
  await fill('customTrait', '')
  for (const trait of ['Tenaz', 'Paciente', 'Soñadora']) await click(`button[aria-label=${JSON.stringify(`Quitar rasgo ${trait}`)}]`)
  await select('.inspiration-page select[name=portrait]', '2')
  await evaluate("(async () => { const canvas = document.createElement('canvas'); canvas.width = 4; canvas.height = 4; canvas.getContext('2d').fillRect(0,0,4,4); const blob = await new Promise(resolve => canvas.toBlob(resolve)); const transfer = new DataTransfer(); transfer.items.add(new File([blob], 'retrato-manual.png', {type:'image/png'})); const input = document.querySelector('.inspiration-page input[type=file]'); input.files = transfer.files; input.dispatchEvent(new Event('change', {bubbles:true})); })()")
  await wait("Boolean(document.querySelector('.inspiration-portrait-card img[src^=\"data:image/png\"]')) && !document.querySelector('.inspiration-page fieldset').disabled")
  const customPortrait = await evaluate("document.querySelector('.inspiration-portrait-card img').src")
  const manuallyEdited = await inspirationDraft()
  assert.ok(await evaluate("document.querySelector('.inspiration-toolbar .primary-button').disabled"))
  await fill('name', 'Otro nombre temporal')
  assert.equal((await inspirationDraft()).name, 'Otro nombre temporal')
  await click('button[aria-label="Recuperar idea anterior"]')
  assert.deepEqual(await inspirationDraft(), manuallyEdited)
  await click('button[aria-label="Desbloquear clase"]')
  await clickText('Otra inspiración')
  const selectedIdea = await inspirationDraft()
  assert.notEqual(selectedIdea.type, manuallyEdited.type)
  for (const key of Object.keys(selectedIdea).filter(key => key !== 'type')) assert.deepEqual(selectedIdea[key], manuallyEdited[key], key)
  assert.equal(await evaluate("document.querySelector('.inspiration-portrait-card img').src"), customPortrait)
  assert.equal(await evaluate("localStorage.getItem('umbra.characters.v1.guest')"), libraryBeforeInspiration)
```

```
283   await evaluate("window.umbraSetItem = Storage.prototype.setItem; Storage.prototype.setItem = () => {
    throw new DOMException('Storage blocked', 'SecurityError') }")
284   await clickText('Guardar personaje')
285   await wait("document.querySelector('.inspiration-create').innerText.includes('No se pudo guardar')")
286   assert.equal(await evaluate('location.pathname'), '/inspiracion')
287   assert.deepEqual(await inspirationDraft(), selectedIdea)
288   assert.equal(await evaluate("document.querySelector('.inspiration-portrait-card img').src"),
    customPortrait)
289   assert.equal(await evaluate("localStorage.getItem('umbra.characters.v1.guest')"),
    libraryBeforeInspiration)
290   await evaluate('Storage.prototype.setItem = window.umbraSetItem')
291   await clickText('Guardar personaje')
292   await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length
    === 8")
293   assert.ok((await evaluate('document.body.innerText')).includes('personaje guardado desde Inspiración'))
294   await reload()
295   assert.equal(await evaluate("document.querySelectorAll('.character-card').length"), 8)
296   const savedIdea = await
    evaluate(`JSON.parse(localStorage.getItem('umbra.characters.v1.guest')).find(character => character.name
    === ${JSON.stringify(selectedIdea.name)})`)
297   for (const key of ['name', 'type', 'race', 'level', 'realm', 'portrait', 'story', 'stats'])
    assert.deepEqual(savedIdea[key], selectedIdea[key], key)
298   assert.equal(savedIdea.imageDataUrl, customPortrait)
299   assert.equal(savedIdea.imageName, 'retrato-manual.png')
300   assert.deepEqual([...savedIdea.traits].sort(), [...selectedIdea.traits].sort())
301   await click(`.character-card[href="/biblioteca/${savedIdea.id}"]`)
302   await clickText('Eliminar personaje')
303   await clickText('Eliminar definitivamente')
304   await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length
    === 7")
305   assert.equal(await evaluate("localStorage.getItem('umbra.characters.v1.guest')"),
    libraryBeforeInspiration)
306   await go('/registro')
307   await fill('name', 'Cuenta de prueba')
308   await fill('email', 'prueba@umbra.es')
309   await fill('password', 'Historia1!')
310   await fill('confirmPassword', 'Historia1!')
311   await click('input[type="checkbox"]')
312   await clickText('Crear cuenta local')
313   await wait("Boolean(document.querySelector('input[readonly]'))")
314   const recoveryCode = await evaluate("document.querySelector('input[readonly]').value")
315   assert.equal(recoveryCode.length, 32)
316   await clickText('Ya he guardado el código. Iniciar sesión')
317   await fill('email', 'prueba@umbra.es')
318   await fill('password', 'Historia1!')
319   await click('input[name="remember"]')
320   await click('.login-submit')
321   await wait("location.pathname === '/perfil' && document.body.innerText.includes('Cuenta de prueba')")
322   assert.ok((await evaluate('document.body.innerText')).includes('6 personajes'))
323   await reload()
324   assert.ok((await evaluate('document.body.innerText')).includes('Cuenta de prueba'))
325   await clickText('Cerrar sesión')
326   await wait("document.body.innerText.includes('Exploras como invitado')")
327   assert.ok((await evaluate('document.body.innerText')).includes('7 personajes'))
328   await evaluate("window.umbraBackup =
    JSON.stringify({app:'UMBRA',version:1,characters:JSON.parse(localStorage.getItem('umbra.characters.v1.gue
    st'))}); window.umbraCreateObjectURL = URL.createObjectURL; URL.createObjectURL = blob => {
    window.umbraDownload = blob.text(); return window.umbraCreateObjectURL(blob) }")
329   await clickText('Exportar biblioteca (JSON)')
```

```
  const downloaded = await evaluate('window.umbraDownload')
  assert.equal(JSON.parse(downloaded).characters.length, 7)
  await importFile('archivo inválido')
  await wait("document.body.innerText.includes('JSON válido')")
  await importFile(JSON.stringify({ app: 'UMBRA', version: 1, characters: [] }))
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Cancelar')
  assert.ok((await evaluate('document.body.innerText')).includes('7 personajes'))
  await importFile(JSON.stringify({ app: 'UMBRA', version: 1, characters: [] }))
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Sustituir y restaurar')
  await wait("document.body.innerText.includes('0 personajes')")
  await go('/biblioteca')
  await reload()
  assert.equal(await evaluate("document.querySelectorAll('.character-card').length"), 0)
  await go('/perfil')
  await importFile(downloaded)
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Sustituir y restaurar')
  await wait("document.body.innerText.includes('7 personajes')")
  await go('/recuperar')
  await fill('email', 'prueba@umbra.es')
  await fill('recoveryCode', recoveryCode)
  await fill('password', 'NuevaClave2!')
  await fill('confirmPassword', 'NuevaClave2!')
  await clickText('Cambiar contraseña')
  await wait("document.body.innerText.includes('Contraseña actualizada')")
  await clickText('Iniciar sesión')
  await fill('email', 'prueba@umbra.es')
  await fill('password', 'NuevaClave2!')
  await click('.login-submit')
  await wait("location.pathname === '/perfil' && document.body.innerText.includes('Cuenta de prueba')")
  await click('button[aria-label="Abrir menú"]')
  await click('.header-popover a[href="/accesibilidad"]')
  await wait("location.pathname === '/accesibilidad'")
  assert.equal(await evaluate("document.querySelectorAll('.settings-menu-item').length"), 5)
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
  await click('.settings-menu a[href="/accesibilidad/texto"]')
  await click('input[name="textSize"][value="large"]')
  const enlarged = await evaluate("parseFloat(getComputedStyle(document.querySelector('.settings-preview
h2')).fontSize)")
  await fill('textSizeSlider', '3')
  await wait("document.querySelector('.app-frame').dataset.textSize === 'extra-large'")
  assert.ok(await evaluate("parseFloat(getComputedStyle(document.querySelector('.settings-preview
h2')).fontSize)") > enlarged)
  await click('input[name="textSize"][value="large"]')
  const ax = await send('Accessibility.getFullAXTree')
  assert.ok(ax.nodes.some(node => node.role?.value === 'radio' && node.name?.value.includes('Grande') &&
node.properties?.some(property => property.name === 'checked' && property.value.value === 'true')))
  await go('/accesibilidad/contraste')
  await click('input[name="contrast"][value="high"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"),
'rgb(0, 0, 0)')
  await click('input[name="contrast"][value="inverted"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"),
'rgb(255, 255, 255)')
  await click('input[name="contrast"][value="high"]')
  await go('/accesibilidad/apariencia')
  await click('input[name="theme"][value="sepia"]')
```

```
  await click('input[name="elementSize"][value="extra-large"]')
  assert.ok(await evaluate("document.querySelector('.choice-card').getBoundingClientRect().height >= 60"))
  await click('input[name="accent"][value="blue"]')
  await click('input[name="reduceMotion"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(0, 0, 0)')
  await go('/accesibilidad/contraste')
  await click('input[name="contrast"][value="normal"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(239, 226, 200)')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).getPropertyValue('--accent').trim()"), '#12679a')
  await go('/accesibilidad/lectura')
  await click('input[name="readAloud"]')
  await click('input[name="announceNavigation"]')
  await select('select[name="readingRate"]', '0.75')
  assert.ok(!await evaluate("Boolean(document.querySelector('.app-frame > p.visually-hidden[role=status]'))"))
  await evaluate("window.umbraSpoken = []; window.umbraCanceled = 0; speechSynthesis.speak = speech => window.umbraSpoken.push({text:speech.text,rate:speech.rate}); speechSynthesis.cancel = () => window.umbraCanceled++")
  assert.equal(await evaluate('window.umbraSpoken.length'), 0)
  await clickText('Leer en voz alta')
  await wait("document.body.innerText.includes('Detener lectura')")
  assert.equal(await evaluate('window.umbraSpoken[0].rate'), 0.75)
  await clickText('Detener lectura')
  await go('/biblioteca')
  await evaluate("window.umbraSpoken = []; window.umbraCanceled = 0; speechSynthesis.speak = speech => window.umbraSpoken.push({text:speech.text,rate:speech.rate}); speechSynthesis.cancel = () => window.umbraCanceled++")
  await click('.character-card')
  await clickText('Leer en voz alta')
  await wait("document.body.innerText.includes('Detener lectura')")
  assert.ok((await evaluate('window.umbraSpoken[0].text')).includes('Elara Valen'))
  const cancellations = await evaluate('window.umbraCanceled')
  await clickText('Volver a Biblioteca')
  await wait("location.pathname === '/biblioteca'")
  assert.ok(await evaluate('window.umbraCanceled') > cancellations)
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.character-card')).transitionDuration"), '0s')
  await go('/accesibilidad/teclado')
  await click('input[name="enhancedFocus"]')
  await evaluate("document.querySelector('input[name=enhancedFocus]').focus()")
  await send('Input.dispatchKeyEvent', { type: 'rawKeyDown', key: 'Tab', code: 'Tab', windowsVirtualKeyCode: 9, nativeVirtualKeyCode: 9 })
  await send('Input.dispatchKeyEvent', { type: 'keyUp', key: 'Tab', code: 'Tab', windowsVirtualKeyCode: 9, nativeVirtualKeyCode: 9 })
  assert.equal(await evaluate('getComputedStyle(document.activeElement).outlineWidth'), '4px')
  await reload()
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'large')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'sepia')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.accent"), 'blue')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.elementSize"), 'extra-large')
  assert.ok(await evaluate("document.querySelector('.app-frame').classList.contains('enhanced-focus')"))
  await go('/perfil')
  await clickText('Cerrar sesión')
  await wait("document.body.innerText.includes('Exploras como invitado')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
```

```
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'normal')
  await go('/accesibilidad/apariencia')
  await click('input[name="theme"][value="light"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(245, 245, 240)')
  await go('/registro')
  await fill('name', 'Segundo perfil')
  await fill('email', 'segundo@umbra.es')
  await fill('password', 'Historia2!')
  await fill('confirmPassword', 'Historia2!')
  await click('input[type=checkbox]')
  await clickText('Crear cuenta local')
  await wait("Boolean(document.querySelector('input[readonly]'))")
  await clickText('Ya he guardado el código. Iniciar sesión')
  await fill('email', 'segundo@umbra.es')
  await fill('password', 'Historia2!')
  await click('.login-submit')
  await wait("location.pathname === '/perfil' && document.body.innerText.includes('Segundo perfil')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
  await go('/accesibilidad/apariencia')
  await click('input[name="theme"][value="light"]')
  await click('input[name="accent"][value="green"]')
  await go('/accesibilidad')
  await clickText('Restablecer ajustes de este perfil')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
  await go('/perfil')
  await clickText('Cerrar sesión')
  await wait("document.body.innerText.includes('Exploras como invitado')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'light')
  await go('/login')
  await fill('email', 'prueba@umbra.es')
  await fill('password', 'NuevaClave2!')
  await click('.login-submit')
  await wait("location.pathname === '/perfil' && document.body.innerText.includes('Cuenta de prueba')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'sepia')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'large')
  await go('/accesibilidad/contraste')
  await click('input[name="contrast"][value="normal"]')
  await send('Input.dispatchKeyEvent', { type: 'rawKeyDown', key: 'ArrowDown', code: 'ArrowDown', windowsVirtualKeyCode: 40, nativeVirtualKeyCode: 40 })
  await send('Input.dispatchKeyEvent', { type: 'keyUp', key: 'ArrowDown', code: 'ArrowDown', windowsVirtualKeyCode: 40, nativeVirtualKeyCode: 40 })
  await wait("document.querySelector('.app-frame').dataset.contrast === 'high'")
  await click('input[name="contrast"][value="normal"]')
  await go('/accesibilidad/texto')
  await evaluate("window.umbraSetItem = Storage.prototype.setItem; Storage.prototype.setItem = () => { throw new DOMException('Storage blocked', 'SecurityError') }")
  await click('input[name="textSize"][value="extra-large"]')
  await wait("document.body.innerText.includes('no permite guardar tus preferencias')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'extra-large')
  await evaluate('Storage.prototype.setItem = window.umbraSetItem')
  await reload()
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'large')
  await go('/biblioteca')
  assert.ok(await evaluate('document.documentElement.scrollWidth <= innerWidth'))
  await send('Emulation.setDeviceMetricsOverride', { width: 320, height: 740, deviceScaleFactor: 1, mobile: true })
  assert.ok(await evaluate('document.documentElement.scrollWidth <= innerWidth'))
  await go('/accesibilidad/texto')
```

```
  await click('input[name="textSize"][value="extra-large"]')
  for (const path of ['/inicio', '/biblioteca', '/biblioteca/elara', '/crear-personaje', '/perfil', '/inspiracion', '/ayuda', '/accesibilidad', '/accesibilidad/texto', '/accesibilidad/contraste', '/accesibilidad/lectura', '/accesibilidad/teclado', '/accesibilidad/apariencia']) {
    await go(path)
    assert.ok(await evaluate('document.documentElement.scrollWidth <= innerWidth'), `Overflow at ${path}`)
  }
  await go('/biblioteca/personaje-inexistente')
  assert.ok((await evaluate('document.body.innerText')).includes('404'))
  assert.equal(await evaluate("document.querySelectorAll('dialog').length"), 0)
  await go('/ruta-que-no-existe')
  assert.ok((await evaluate('document.body.innerText')).includes('404'))
  await evaluate("window.dispatchEvent(new Event('offline'))")
  await wait("document.querySelector('.connection-status').textContent.includes('Sin conexión')")
  await evaluate("window.dispatchEvent(new Event('online'))")
  await wait("document.querySelector('.connection-status').textContent.includes('restablecida')")
  await send('Emulation.setDeviceMetricsOverride', { width: 390, height: 844, deviceScaleFactor: 1, mobile: true })
  await go('/accesibilidad')
  if (process.env.UMBRA_SCREENSHOT) {
    const shot = await send('Page.captureScreenshot', { format: 'png', captureBeyondViewport: true })
    await writeFile(process.env.UMBRA_SCREENSHOT, Buffer.from(shot.data, 'base64'))
  }
  assert.deepEqual(exceptions, [])
  console.log('Browser OK: fichas en página, scroll normal, rutas directas, vuelta con filtros, CRUD, cuentas, copias, accesibilidad por perfil, voz, foco, móvil, conexión y 404.')
} catch (error) {
  console.error(error)
  console.error('URL:', await evaluate('location.href').catch(() => 'unknown'))
  console.error('Page:', await evaluate('document.body.innerText').catch(() => 'unavailable'))
  process.exitCode = 1
} finally {
  if (browserContextId) await send('Target.disposeBrowserContext', { browserContextId }, null)
  socket.close()
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 44 DE 62

favicon.svg

`public/favicon.svg`

Icono vectorial de la pestaña del navegador.

- Lo referencia index.html. Se incluye su código SVG completo; los PNG y las fuentes se guardan como archivos binarios en el ZIP.

Código completo

4 líneas · SHA-256 del original: `9f9b30bbf7aa3dca61c8b928754abfa4ad370249b9b4d857bdcf1a28825ec5dc`

```
1 <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 32 32">
2   <rect width="32" height="32" rx="6" fill="#090f0f"/>
3   <path d="m16 4 10 6-3 14-7 5-7-5-3-14Zm0 0v25m-5-18 1 9 4 4 4-4 1-9" fill="none" stroke="#a99770"
  stroke-linejoin="round"/>
4 </svg>
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 45 DE 62

ASSETS.md

`public/images/ASSETS.md`

Descripción y procedencia de los recursos gráficos incluidos.

- Documenta los prompts utilizados para los tres PNG. No genera imágenes al ejecutar la app.

Código completo

15 líneas · SHA-256 del original: `b47a7657182ce862ccd9e88a9b70ae4ae33ea49660a9d52875a18fdea6997172`

```
# Imágenes de UMBRA

Generadas con la herramienta integrada `image_gen` para esta pantalla. Las imágenes son interpretaciones del estilo de la referencia, no los archivos originales de la captura.

## umbra-castle.png

Prompt: Use case: stylized-concept. Create a landscape 1536x1024 dark fantasy illustration for a website hero background. A towering, intricate gothic castle with many narrow spires on a steep rocky hill, storm clouds, faint muted copper light on the horizon, atmospheric haze. Castle on the right half, cloudy sky and negative space left. Realistic painterly game concept art, restrained charcoal, desaturated green-black, aged gold, no bright colors. Similar mood to a dark medieval role-playing character library. No words, no text, no borders, no interface. This image will be saved as umbra-castle.png in a local React project.

## elara-valen.png

Prompt: Use case: stylized-concept. Square dark fantasy character portrait for a small character card. Elara Valen, pale adult sorceress with long wavy black hair, dark fitted medieval gown, silver pendant, calm serious expression, centered waist-up front view. Ancient gothic library with carved stone arches and candles behind her. Realistic painterly role-playing game illustration, charcoal black, muted aged gold, desaturated olive green. Soft dramatic light on face; dark atmosphere but facial features readable. No text, no borders, no UI. Save intended asset elara-valen.png.

## kael-grim.png

Prompt: Use case: stylized-concept. Square dark fantasy character portrait for a small character card. Kael Grim, rugged adult male rogue with short dark beard, dark hood, worn brown leather armor and belt, centered waist-up front view. Ancient gothic stone hall with tall arches in background. Realistic painterly role-playing game illustration, charcoal black, muted aged gold, desaturated olive green. Soft dramatic light on face; dark atmosphere but facial features readable. No text, no borders, no UI. Save intended asset kael-grim.png.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 46 DE 62

.gitignore

`.gitignore`

Patrones que Git ignora para evitar añadir dependencias, compilados y archivos temporales.

- Incluye node_modules, dist, archivos de log y datos locales de herramientas. No elimina esos archivos del equipo.

Código completo

24 líneas · SHA-256 del original: `d50ab07e11fa4bf3b1c7c3312e1d7baeab56cca756832ccf40476823ad2a38b7`

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
dist
dist-ssr
*.local

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 47 DE 62

README.md

`README.md`

Guía bilingüe del frontend, configuración de API y límites de la integración actual.

- Complementa este manual. El listado de código reproduce el archivo tal como estaba al generar el documento.

Código completo

457 líneas · SHA-256 del original: `7ad1fd72d23f8bffce9e75d275dbd75de7ee0db3eb68a1f661f757f6bb8ca445`

````
# UMBRA

[Español](#espanol) · [English](#english)

<a id="espanol"></a>

## Español

Aplicación para crear personajes de fantasía, escribir sus historias y organizar sus fichas. Desarrollada con **React, Vite y React Router**, con temas personalizables y accesibilidad independiente para cada perfil.

### Estado actual

Documentación revisada el **09/11/2026** a partir del código del repositorio.

La app combina **servicios HTTP de Spring Boot** con **almacenamiento local del navegador**. El registro, el inicio y cierre de sesión y la comprobación de la sesión ya llaman al backend. Biblioteca muestra además personajes públicos del servidor, con una pantalla de detalle propia.

La creación, edición, duplicado, eliminación, favoritos y copias de la biblioteca personal siguen siendo locales, al igual que los ajustes de accesibilidad por perfil. Iniciar sesión no sube esas fichas al servidor ni sincroniza bibliotecas entre dispositivos.

La interfaz está diseñada principalmente para móvil y tiene un ancho máximo general de **390 px**. Incluye ajustes para pantallas estrechas y texto ampliado; la distribución específica para tablet y PC queda como una futura mejora.

### Empezar

Necesitas Node.js y npm instalados. Desde la carpeta del proyecto:

```bash
npm install
npm run dev
```

Abre la dirección que indique Vite. En PowerShell, si la ejecución de `npm.ps1` está bloqueada, utiliza `npm.cmd`:
````

```powershell
npm.cmd install
npm.cmd run dev
```

Con `package-lock.json` disponible también puedes utilizar `npm ci` para instalar las dependencias según el lock.

Configuración de la API en desarrollo

Arranca por separado el backend Spring Boot en **`http://localhost:8080`**, con su configuración de base de datos. Este repositorio contiene el frontend; no incluye el servidor ni la configuración de MariaDB.

vite.config.js reenvía las solicitudes que comienzan por `/api` a `http://localhost:8080`, con `changeOrigin: true` y sin reescribir la ruta. Por ejemplo, `/api/auth/me` llega al backend como `http://localhost:8080/api/auth/me`. Conserva también la exclusión de `public/fonts/**` del observador de archivos.

.env.development declara:

```dotenv
VITE_API_URL=http://localhost:8080/api
```

Actualmente [api.js](src/services/api.js) utiliza rutas relativas `/api/...` y **no lee `VITE_API_URL`**. Cambiar únicamente esa variable no cambia el destino: la dirección efectiva de desarrollo es el `target` del proxy de Vite. Reinicia Vite después de modificar su configuración.

Primera entrada

Puedes crear y guardar personajes como Invitado. Para registrarte en el servidor, entra en **Iniciar sesión → Crear cuenta**, introduce nombre de 2 a 50 caracteres, correo de hasta 100 y contraseña de 8 a 72 caracteres con un carácter especial, y confirma la contraseña. Tras el registro se muestra un enlace para iniciar sesión; no se genera un código de recuperación local.

El inicio de sesión envía correo y contraseña a la API. Al abrir la app se consulta `/api/auth/me`; si devuelve 401 se continúa como Invitado. Si falla la comprobación se muestra un aviso y también se permite continuar como Invitado.

Usa el mismo navegador y la misma dirección para conservar bibliotecas y preferencias locales: `localhost:5173` y `127.0.0.1:5173` tienen almacenamientos diferentes. El frontend no configura credenciales precreadas.

Funciones

- **Inicio:** portada, acceso al editor y los dos personajes más recientes.
- **Biblioteca:** búsqueda por nombre, clase, raza u origen; filtro de clase y favoritos; ordenación por actualización, nombre o nivel.
- **Fichas:** crear, consultar, editar, duplicar y eliminar personajes. Cada ficha se abre en su propia pantalla (`/biblioteca/:id`), con scroll de página, enlace para volver y confirmación de eliminación dentro de la ficha. Los filtros de biblioteca se conservan en la dirección al volver.
- **Personalización:** nombre, clase, raza, nivel, origen, historia, hasta seis rasgos y cuatro atributos: Fuerza, Destreza, Inteligencia y Carisma.
- **Retratos:** imágenes incluidas, iniciales o un archivo propio PNG, JPG o WebP de hasta 750 KB, con vista previa.

- **Inspiración:** ficha visual con todos los campos aleatorios y editables directamente: nombre, clase, raza, nivel, origen, retrato, personalidad, historia y atributos. Puedes subir un retrato, añadir rasgos propios, regenerar un campo y recuperar la idea anterior. Los cambios manuales se fijan automáticamente para conservarlos al generar otra combinación. **Guardar personaje** valida y guarda directamente en la biblioteca del perfil activo. El editor utiliza el mismo generador completo.
- **Personajes públicos:** listado independiente al final de Biblioteca, con estados de carga, error y lista vacía. Cada tarjeta abre `/personajes-publicos/:id` para consultar historia, personalidad, visibilidad y atributos del servidor. Los filtros locales no filtran este listado.
- **Cuentas del servidor:** registro, acceso, comprobación inicial y cierre de sesión mediante la API; consulta las limitaciones de Recordarme y recuperación más abajo.
- **Perfil:** identidad, recuento de personajes y favoritos, copias de seguridad y acceso a los ajustes.
- **Ayuda y feedback:** preguntas frecuentes, privacidad, avisos de conexión, mensajes de validación y pantalla 404.

Accesibilidad por perfil

Se abre desde **Menú → Accesibilidad** o desde el enlace del Perfil. Está organizada en cinco submenús:

Menú	Opciones
Tamaño de texto	Pequeño, normal, grande y extra grande; selección mediante radios o deslizador.
Modo de contraste	Normal, alto contraste e invertido.
Lectores de pantalla	Anuncios de cambio de pantalla, lectura voluntaria en voz alta y velocidad pausada, normal o rápida.
Navegación por teclado	Foco reforzado y guía de Tab, Mayús + Tab, Enter, Espacio, flechas, Inicio, Fin y Escape.
Personalización visual	Temas oscuro, claro y sepia; tres tamaños de controles; seis colores de acento; movimiento reducido.

Cada cuenta e Invitado conservan sus propios ajustes. Al cambiar de perfil se recupera su configuración; **Restablecer ajustes** afecta únicamente al perfil activo.

El contraste tiene prioridad sobre los colores del tema y del acento, pero conserva esas elecciones para cuando vuelvas al contraste normal. La interfaz respeta también la preferencia del sistema de reducir el movimiento.

La navegación utiliza controles semánticos, etiquetas, textos alternativos y diálogos con gestión del foco. La lectura en voz alta usa las voces disponibles en el navegador o dispositivo y se detiene al cerrar la ficha o salir de su pantalla. Es una función adicional al lector de pantalla del sistema.

Datos, sesión y cuentas

Las bibliotecas personales se guardan en `localStorage`. Cada perfil empieza con seis personajes de ejemplo; una biblioteca guardada vacía permanece vacía. Los datos de Invitado se conservan al iniciar o cerrar sesión. La clave del perfil autenticado usa el `idUsuario` devuelto por el servidor, convertido a cadena.

Dato	Ubicación y comportamiento
`umbra.characters.v1.<owner>`	Biblioteca local; `owner` es el identificador del usuario del servidor o `guest`.
`umbra.accessibility.v2.<owner>`	Ajustes locales de accesibilidad y apariencia de ese perfil.
Sesión actual	React conserva `id`, `name`, `email` y `rol` a partir de la respuesta del backend. Al recargar se vuelve a consultar `/api/auth/me`.
`umbra.accounts.v1` / `umbra.session.v1`	Claves del módulo local anterior. El registro y acceso actuales no las utilizan; la recuperación todavía depende de las cuentas locales antiguas.

Las antiguas preferencias globales de `umbra.accessibility.v1` se recuperan únicamente para Invitado. Borrar los datos del sitio elimina las bibliotecas y ajustes de ese navegador; no elimina la cuenta del backend.

El login, la comprobación de sesión y el logout utilizan `credentials: 'same-origin'`. El cliente espera que el backend gestione la sesión mediante cookies y no almacena un token de autenticación. La duración y los atributos de las cookies dependen del servidor; no se pueden determinar desde este repositorio.

Integración todavía parcial

- **Recordarme:** la casilla permanece en el formulario, pero el servicio solo envía `email` y `password`. Marcarla no configura una duración de sesión diferente en la implementación actual.
- **Recuperación:** `/recuperar` sigue llamando a `recoverAccount` de [localAccounts.js](src/utils/localAccounts.js). Solo modifica cuentas locales antiguas; no restablece la contraseña de las cuentas creadas por la API y no hay envío de correos implementado en el frontend.
- **Roles:** la respuesta de sesión aporta `rol`, pero no existen pantallas ni controles de permisos por rol en este frontend. Los permisos efectivos corresponden al backend.
- **Textos de la interfaz:** algunas etiquetas, ayuda y privacidad aún hablan de «cuenta local». El flujo de registro e inicio de sesión vigente es el de la API descrito aquí.

El módulo local anterior conserva PBKDF2-SHA-256 con sal y 100 000 iteraciones para contraseñas y SHA-256 para códigos. Esto describe las cuentas locales antiguas, no el almacenamiento de contraseñas del backend. Esa recuperación requiere Web Crypto en localhost o HTTPS.

Si no se puede guardar un cambio en la biblioteca, se muestra un error y se conserva el estado anterior. Los ajustes de accesibilidad sí se aplican temporalmente cuando el almacenamiento está bloqueado, con un aviso de que no han podido persistirse.

Con la app ya abierta puedes seguir trabajando con la biblioteca local sin conexión. El registro, el acceso y las fichas públicas necesitan el servidor; recargar requiere acceso al sitio o al servidor local. No hay un service worker para cargar la app sin conexión.

Copias de seguridad de personajes

En **Perfil → Copia de seguridad**:

1. **Exportar biblioteca (JSON)** descarga los personajes, favoritos y retratos del perfil activo.
2. **Importar copia de seguridad** permite seleccionar un JSON de UMBRA de hasta 10 MB.
3. La app valida el contenido y pide confirmación antes de sustituir toda la biblioteca actual. Admite hasta 500 personajes y rechaza identificadores repetidos.

Las copias de personajes no incluyen cuentas, contraseñas ni ajustes de accesibilidad. Puedes utilizarlas para trasladar una biblioteca a otro navegador o perfil. La implementación está en [characterStore.js](src/utils/characterStore.js) y los controles están en [Profile.jsx](src/pages/Profile.jsx).

Comandos y comprobaciones

Comando	Función
`npm run dev`	Inicia el servidor de desarrollo Vite.
`npm test`	Ejecuta las pruebas unitarias de validación, bibliotecas, cuentas y preferencias.
`npm run lint`	Comprueba las reglas de JavaScript, JSX y hooks de React.
`npm run build`	Genera la aplicación compilada en `dist/`.
`npm run preview`	Sirve la compilación para revisarla localmente.
`npm run test:browser`	Ejecuta recorridos automáticos en Chrome o Edge.

Pruebas en navegador

````
143 [browser-smoke.mjs](scripts/browser-smoke.mjs) contiene recorridos de la versión local anterior:
    personajes, filtros, retratos, favoritos, cuentas, recuperación, copias JSON, accesibilidad, separación
    de perfiles, teclado, fallos de almacenamiento y pantallas de 320 y 390 px.
144
145 **Pendiente de adaptar a la API:** el recorrido todavía espera el antiguo registro local, su código de
    recuperación y sus etiquetas. No valida de extremo a extremo el nuevo flujo de sesión ni los personajes
    públicos; no debe darse por compatible con la integración actual. Las pruebas unitarias de cuentas
    locales tampoco prueban la autenticación de Spring Boot.
146
147 Es una herramienta de desarrollo. Utiliza un contexto temporal aislado y lo elimina al terminar; las
    cuentas y bibliotecas de prueba pertenecen a ese contexto.
148
149 Para ejecutarla:
150
151 1. Arranca la app con `npm run dev`.
152 2. Abre Chrome o Edge con depuración remota en el puerto **9222** y un perfil de navegador dedicado a las
    pruebas. Por ejemplo, en otra terminal PowerShell con Chrome instalado en su ruta habitual:
153
154    ```powershell
155    & "C:\Program Files\Google\Chrome\Application\chrome.exe" --headless=new --remote-debugging-port=9222
    "--user-data-dir=$env:TEMP\umbra-browser-smoke"
156    ```
157
158 3. Ejecuta `npm run test:browser` en otra terminal. En PowerShell puedes usar `npm.cmd run test:browser`.
159
160 El script utiliza `http://127.0.0.1:5173` por defecto. Si la app está en otra dirección o quieres una
    captura, configura estas variables antes de ejecutarlo:
161
162 ```powershell
163 $env:UMBRA_URL = "http://127.0.0.1:4173"
164 $env:UMBRA_SCREENSHOT = Join-Path $env:TEMP "umbra-captura.png"
165 npm.cmd run test:browser
166 ```
167
168 `UMBRA_URL` debe apuntar al servidor que tengas activo; por ejemplo, el de `npm run preview`. Si todo
    pasa, el script muestra `Browser OK`. Si falla una comprobación, muestra el error, la dirección y el
    contenido de la pantalla.
169
170 Algunas comprobaciones de voz utilizan una simulación: verifican los controles, el texto y la
    cancelación, pero no la salida de audio real ni un lector de pantalla externo.
171
172 ### Estructura del proyecto
173
174 ```text
175 src/
176   App.jsx             Sesión, rutas, espacio del perfil y diálogos
177   main.jsx            Montaje de React y BrowserRouter
178   App.css             Composición, componentes y variantes de accesibilidad
179   index.css           Fuentes y estilos base
180   pages/              Pantallas de cada ruta y sus estilos específicos
181   components/         Navegación, tarjetas, retratos, formularios y avisos
182   hooks/              Estado y persistencia de bibliotecas y preferencias
183   services/           Peticiones HTTP de personajes públicos y autenticación
184   utils/              Validación, cuentas locales antiguas, almacenamiento y copias JSON
185   data/               Personajes iniciales, catálogos y navegación
186 tests/                Pruebas unitarias
187 scripts/              Recorridos automáticos de navegador
188 public/               Imágenes, icono, fuentes y licencias
189 documentacion/        Manual, código empaquetado y herramientas de generación
````

```
```

Las rutas y los títulos se coordinan en [App.jsx](src/App.jsx) y [navigation.js](src/data/navigation.js). Las imágenes se documentan en [ASSETS.md](public/images/ASSETS.md); las licencias de las fuentes están en `public/fonts/` y la licencia del proyecto en LICENSE.

Documentación para consultar y extraer

- [Manual completo en PDF](documentacion/UMBRA-manual-completo.pdf): funciones, arquitectura, explicación de componentes y código con números de línea.
- [Manual en HTML](documentacion/UMBRA-manual-completo.html): versión para abrir y buscar en el navegador.
- [Documentación y código en ZIP](documentacion/UMBRA-documentacion-y-codigo.zip): manual y carpeta `codigo/` editable con los recursos del proyecto.
- [Guía de documentación](documentacion/LEEME.md): contenido, extracción y herramientas para regenerar el manual.

El manual y el ZIP son una instantánea del proyecto en el momento de su generación. No se actualizan automáticamente al editar el código o este README. El ZIP de código tampoco incluye los datos guardados por las personas en sus navegadores.

API de Spring Boot integrada

Las solicitudes están concentradas en src/services/api.js. Estas son las rutas que consume actualmente el frontend:

Método y ruta	Uso y respuesta esperada
`GET /api/personajes/publicos`	Lista JSON de personajes públicos; las tarjetas usan `idPersonaje` y `nombre`.
`GET /api/personajes/{id}`	Detalle JSON con `nombre`, `visibilidad`, `historia`, `personalidad`, `fuerza`, `destreza`, `inteligencia` y `carisma`. El servicio codifica el identificador en la URL.
`POST /api/auth/register`	Envía JSON `{ nombreUsuario, email, password }`; espera JSON en caso de éxito. Traduce 409 a correo registrado y 400 a datos incorrectos.
`POST /api/auth/login`	Envía JSON `{ email, password }`; espera `{ idUsuario, nombreUsuario, email, rol }`. Traduce 401 a credenciales incorrectas.
`GET /api/auth/me`	Recupera esa misma identidad; 401 se trata como ausencia de sesión. Envía `X-Requested-By: XMLHttpRequest`.
`POST /api/auth/logout`	Solicita cerrar la sesión; después del éxito, la app vuelve a Invitado. No necesita cuerpo JSON de respuesta.

Los personajes públicos se adaptan al componente `CharacterCard` con `interactive={false}`, dentro de un enlace al detalle público. Usan un id visual `public-<idPersonaje>`, muestran «Personaje público» y omiten el nivel. El adaptador actual no asigna retrato, por lo que muestra las iniciales. El detalle público es de consulta; no ofrece edición ni favoritos locales.

`Portrait` elige `imageDataUrl`, después `image` y después un retrato incluido; si no hay imagen o falla la carga, muestra iniciales. Permite ajustar el encuadre con `imagePosition` (por defecto `center`).

Quedan pendientes el CRUD remoto de la biblioteca personal, la sincronización de ajustes, la recuperación de contraseña del servidor y la conexión de Recordarme. El frontend no accede directamente a MariaDB ni al ORM; esas capas pertenecen al backend.

Compilar y publicar

```bash
npm run build
npm run preview
```

Publica el contenido generado en `dist/`. Como se utiliza `BrowserRouter`, configura el alojamiento para devolver `index.html` en las rutas internas de la aplicación. En producción configura también un proxy inverso para `/api` hacia Spring Boot en el mismo origen: el proxy definido en `server.proxy` es de desarrollo y no acompaña a los archivos compilados. `npm run preview` hereda el proxy de desarrollo de Vite; sigue necesitando el backend activo. Cambiar `VITE_API_URL` no modifica las rutas relativas actuales. Utiliza HTTPS fuera del entorno local y revisa la sesión con la configuración real del servidor.

English

An application for creating fantasy characters, writing their stories, and organizing their character sheets. Built with **React, Vite, and React Router**, with customizable themes and independent accessibility settings for each profile.

Current status

Documentation reviewed on **9 November 2026** against the repository source.

The app combines **Spring Boot HTTP services** with **local browser storage**. Registration, sign-in, sign-out, and session checks already call the backend. Library also displays public characters from the server, with a dedicated detail screen.

Personal character creation, editing, duplication, deletion, favorites, and backups remain local, as do accessibility settings per profile. Signing in does not upload these characters or synchronize libraries between devices.

The interface is designed primarily for mobile and has a general maximum width of **390 px**. It includes adjustments for narrow screens and enlarged text; dedicated tablet and desktop layouts remain a future improvement.

Getting started

You need Node.js and npm installed. From the project folder:

```bash
npm install
npm run dev
```

Open the address displayed by Vite. In PowerShell, if running `npm.ps1` is blocked, use `npm.cmd`:

```powershell
npm.cmd install
npm.cmd run dev
```

With `package-lock.json` available, you can also use `npm ci` to install dependencies according to the lockfile.

Development API configuration

Run the Spring Boot backend separately at **`http://localhost:8080`**, with its database configured. This repository contains the frontend; it does not include the backend or MariaDB configuration.

```
[vite.config.js](vite.config.js) proxies requests starting with `/api` to `http://localhost:8080`, with `changeOrigin: true` and no path rewrite. For example, `/api/auth/me` reaches `http://localhost:8080/api/auth/me`. It also excludes `public/fonts/**` from file watching.

[.env.development](.env.development) declares:

```dotenv
VITE_API_URL=http://localhost:8080/api
```

[api.js](src/services/api.js) currently uses relative `/api/...` paths and **does not read `VITE_API_URL`**. Changing that variable alone does not change the destination; Vite's proxy `target` determines the development destination. Restart Vite after changing its configuration.

First visit

You can create and save characters as a Guest. To register on the server, open **Iniciar sesión → Crear cuenta** (Sign in → Create account). Enter a name of 2–50 characters, an email address of up to 100 characters, and a password of 8–72 characters including a special character, then confirm the password. Successful registration displays a sign-in link; it does not issue a local recovery code.

Sign-in sends email and password to the API. On startup, the app checks `/api/auth/me`; a 401 response continues as Guest. If the session check fails, a notice is displayed and Guest access remains available.

Use the same browser and address to retain local libraries and preferences: `localhost:5173` and `127.0.0.1:5173` have separate storage. The frontend does not configure precreated credentials.

Features

- **Home:** landing screen, access to the editor, and the two most recently updated characters.
- **Library:** search by name, class, race, or origin; filter by class and favorites; sort by update time, name, or level.
- **Character sheets:** create, view, edit, duplicate, and delete characters. Each sheet opens on its own screen (`/biblioteca/:id`), with normal page scrolling, a back link, and inline deletion confirmation. Library filters are kept in the URL when returning.
- **Customization:** name, class, race, level, origin, story, up to six traits, and four attributes: Strength, Dexterity, Intelligence, and Charisma.
- **Portraits:** included images, initials, or your own PNG, JPG, or WebP file of up to 750 KB, with a preview.
- **Inspiration:** a visual draft with every field randomized and directly editable: name, class, race, level, origin, portrait, personality, story, and attributes. Upload a portrait, add custom traits, reroll a field, or restore the previous idea. Manual changes are automatically locked to preserve them when generating another combination. **Guardar personaje** (Save character) validates and saves directly to the active profile's library. The editor uses the same complete generator.
- **Public characters:** an independent list below the personal Library, with loading, error, and empty states. Each card opens `/personajes-publicos/:id` to read server-provided story, personality, visibility, and attributes. Local filters do not filter this list.
- **Server accounts:** registration, sign-in, startup session checks, and sign-out through the API; see the Remember me and recovery limitations below.
- **Profile:** identity, character and favorite counts, backups, and access to settings.
- **Help and feedback:** frequently asked questions, privacy information, connection notices, validation messages, and a 404 screen.

Accessibility per profile

Open **Menú → Accesibilidad** (Menu → Accessibility) or use the link on the Profile screen. Settings are organized into five submenus:

| Menu | Options |
```

```
| --- | --- |
| Text size | Small, normal, large, and extra large; selected using radio buttons or a slider. |
| Contrast mode | Normal, high contrast, and inverted. |
| Screen readers | Screen change announcements, optional read-aloud controls, and slow, normal, or fast reading speed. |
| Keyboard navigation | Enhanced focus indicator and a guide to Tab, Shift + Tab, Enter, Space, arrow keys, Home, End, and Escape. |
| Visual customization | Dark, light, and sepia themes; three control sizes; six accent colors; reduced motion. |

Each account and the Guest profile keep their own settings. Switching profiles restores the corresponding configuration; **Restablecer ajustes** (Reset settings) affects only the active profile.

Contrast takes priority over theme and accent colors while preserving those choices for when you return to normal contrast. The interface also respects the system preference for reduced motion.

Navigation uses semantic controls, labels, alternative text, and dialogs with focus management. Read-aloud uses the voices available in the browser or device and stops when the character sheet is closed or its screen is left. It is an additional feature alongside the system screen reader.

### Data, sessions, and accounts

Personal libraries are saved in `localStorage`. Each profile starts with six example characters; a saved empty library stays empty. Guest data is preserved when signing in or out. An authenticated profile uses the server's `idUsuario`, converted to a string, as its storage owner.

| Data | Location and behavior |
| --- | --- |
| `umbra.characters.v1.<owner>` | Local library; `owner` is the server user identifier or `guest`. |
| `umbra.accessibility.v2.<owner>` | Local accessibility and appearance settings for that profile. |
| Current session | React holds `id`, `name`, `email`, and `rol` from the backend response. Reloading checks `/api/auth/me` again. |
| `umbra.accounts.v1` / `umbra.session.v1` | Legacy local module keys. Current registration and sign-in do not use them; recovery still depends on old local accounts. |

Legacy global preferences from `umbra.accessibility.v1` are restored only for Guest. Clearing site data removes that browser's libraries and preferences; it does not delete the backend account.

Login, session checks, and logout use `credentials: 'same-origin'`. The client expects the backend to manage cookie sessions and does not store an authentication token. Cookie lifetime and attributes depend on the server and cannot be established from this repository.

#### Integration limitations

- **Remember me:** the checkbox remains in the form, but the service only sends `email` and `password`. Selecting it does not configure a different session lifetime in the current implementation.
- **Recovery:** `/recuperar` still calls `recoverAccount` in [localAccounts.js](src/utils/localAccounts.js). It only updates legacy local accounts; it does not reset passwords for API-created accounts. The frontend implements no email delivery.
- **Roles:** the session response supplies `rol`, but this frontend has no role-specific screens or permission checks. Effective authorization belongs to the backend.
- **Interface text:** some labels, help, and privacy text still refer to a “local account”. Current registration and sign-in use the API described here.

The legacy local module retains PBKDF2-SHA-256 with a salt and 100,000 iterations for passwords and SHA-256 for recovery codes. This describes old local accounts, not backend password storage. That recovery requires Web Crypto on localhost or HTTPS.
```

If saving a library change fails, an error is shown and the previous state is preserved. Accessibility settings still apply temporarily when storage is blocked, with a notice that they could not be persisted.

With the app already open, the local library can still be used offline. Registration, sign-in, and public characters require the server. Reloading requires access to the website or local server; there is no offline service worker.

Character backups

Under **Perfil → Copia de seguridad** (Profile → Backup):

1. **Exportar biblioteca (JSON)** downloads the active profile's characters, favorites, and portraits.
2. **Importar copia de seguridad** lets you select a UMBRA JSON file of up to 10 MB.
3. The app validates its contents and requests confirmation before replacing the entire current library. It accepts up to 500 characters and rejects duplicate identifiers.

Character backups do not include accounts, passwords, or accessibility settings. You can use them to transfer a library to another browser or profile. The implementation is in [characterStore.js](src/utils/characterStore.js), and the controls are in [Profile.jsx](src/pages/Profile.jsx).

Commands and checks

Command	Purpose
`npm run dev`	Starts the Vite development server.
`npm test`	Runs unit tests for validation, libraries, accounts, and preferences.
`npm run lint`	Checks JavaScript, JSX, and React Hooks rules.
`npm run build`	Generates the compiled application in `dist/`.
`npm run preview`	Serves the build for local review.
`npm run test:browser`	Runs automated workflows in Chrome or Edge.

Browser tests

[browser-smoke.mjs](scripts/browser-smoke.mjs) contains workflows for the previous local version: characters, filters, portraits, favorites, accounts, recovery, JSON backups, accessibility, profile isolation, keyboard navigation, storage failures, and screens 320 and 390 px wide.

API adaptation pending: the workflow still expects legacy local registration, its recovery code, and its labels. It does not validate the new session flow or public characters end to end and should not be assumed compatible with the current integration. Local account unit tests do not test Spring Boot authentication either.

It is a development tool. It uses an isolated temporary browser context and disposes of it when finished; test accounts and libraries belong to that context.

To run it:

1. Start the app with `npm run dev`.
2. Open Chrome or Edge with remote debugging on port **9222** and a browser profile dedicated to testing. For example, in another PowerShell terminal with Chrome installed at its usual location:

   ```powershell
   & "C:\Program Files\Google\Chrome\Application\chrome.exe" --headless=new --remote-debugging-port=9222 "--user-data-dir=$env:TEMP\umbra-browser-smoke"
   ```

3. Run `npm run test:browser` in another terminal. In PowerShell, you can use `npm.cmd run test:browser`.

````
The script uses `http://127.0.0.1:5173` by default. If the app is running at another address or you want a screenshot, set these variables before running it:

```powershell
$env:UMBRA_URL = "http://127.0.0.1:4173"
$env:UMBRA_SCREENSHOT = Join-Path $env:TEMP "umbra-captura.png"
npm.cmd run test:browser
```

`UMBRA_URL` must point to your active server, for example the one started by `npm run preview`. If all checks pass, the script displays `Browser OK`. If a check fails, it displays the error, address, and screen content.

Some voice checks use a simulation: they verify the controls, text, and cancellation, but not actual audio output or an external screen reader.

### Project structure

```text
src/
  App.jsx             Session, routes, profile workspace, and dialogs
  main.jsx            React and BrowserRouter setup
  App.css             Layout, components, and accessibility variants
  index.css           Fonts and base styles
  pages/              Route screens and their specific styles
  components/         Navigation, cards, portraits, forms, and notices
  hooks/              State and persistence for libraries and preferences
  services/           HTTP requests for public characters and authentication
  utils/              Validation, legacy local accounts, storage, and JSON backups
  data/               Initial characters, option catalogs, and navigation
tests/                Unit tests
scripts/              Automated browser workflows
public/               Images, favicon, fonts, and licenses
documentacion/        Manual, packaged source code, and generation tools
```

Routes and titles are coordinated in [App.jsx](src/App.jsx) and [navigation.js](src/data/navigation.js). Images are documented in [ASSETS.md](public/images/ASSETS.md); font licenses are in `public/fonts/`, and the project license is in [LICENSE](LICENSE).

### Documentation to read and extract

- [Complete PDF manual](documentacion/UMBRA-manual-completo.pdf): features, architecture, component explanations, and source code with line numbers.
- [HTML manual](documentacion/UMBRA-manual-completo.html): a version to open and search in a browser.
- [Documentation and source ZIP](documentacion/UMBRA-documentacion-y-codigo.zip): the manual and an editable `codigo/` folder containing project resources.
- [Documentation guide](documentacion/LEEME.md): contents, extraction, and tools for regenerating the manual.

The manual and ZIP are snapshots of the project at the time they were generated. They do not update automatically when the code or this README changes. The source ZIP also excludes data saved by people in their browsers.

### Integrated Spring Boot API

Requests are centralized in [src/services/api.js](src/services/api.js). The frontend currently consumes these routes:
````

````
| Method and route | Purpose and expected response |
| --- | --- |
| `GET /api/personajes/publicos` | JSON public character list; cards use `idPersonaje` and `nombre`. |
| `GET /api/personajes/{id}` | JSON detail with `nombre`, `visibilidad`, `historia`, `personalidad`, `fuerza`, `destreza`, `inteligencia`, and `carisma`. The service URL-encodes the identifier. |
| `POST /api/auth/register` | Sends JSON `{ nombreUsuario, email, password }`; expects JSON on success. Maps 409 to an existing email and 400 to invalid input. |
| `POST /api/auth/login` | Sends JSON `{ email, password }`; expects `{ idUsuario, nombreUsuario, email, rol }`. Maps 401 to invalid credentials. |
| `GET /api/auth/me` | Restores the same identity; 401 means no active session. Sends `X-Requested-By: XMLHttpRequest`. |
| `POST /api/auth/logout` | Requests session termination; successful logout returns the app to Guest. No JSON response body is required. |

Public characters use `CharacterCard` with `interactive={false}`, wrapped in a link to public detail. Their display id is `public-<idPersonaje>`; cards show “Personaje público” and omit the level. The current adapter assigns no portrait, so initials are displayed. Public detail is read-only and provides no editing or local favorite controls.

`Portrait` chooses `imageDataUrl`, then `image`, then an included portrait. Missing or failed images fall back to initials. `imagePosition` adjusts the crop, defaulting to `center`.

Remote personal-library CRUD, preference synchronization, server password recovery, and Remember me integration remain pending. The frontend does not access MariaDB or the ORM directly; those layers belong to the backend.

### Build and deployment

```bash
npm run build
npm run preview
```

Deploy the generated contents of `dist/`. Because the app uses `BrowserRouter`, configure hosting to return `index.html` for internal application routes. In production, also configure a reverse proxy for `/api` to Spring Boot on the same origin: `server.proxy` is a development setting and is not shipped with the compiled files. `npm run preview` inherits Vite’s development proxy and still requires the backend to be running. Changing `VITE_API_URL` does not change the current relative paths. Use HTTPS outside local development and verify sessions against the actual server configuration.
````

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 48 DE 62

LICENSE

LICENSE

Licencia del proyecto incluida en el repositorio.

- Se reproduce íntegra en la documentación y se conserva también en el ZIP. Las fuentes tienen sus propias licencias en public/fonts.

Código completo

21 líneas · SHA-256 del original: `58e32531aacd2fa91b72e5c1cb037fe7b520989bc7ffdc66c40aa6537b184202`

```
MIT License

Copyright (c) 2026 Ixf

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 49 DE 62

Inter-OFL.txt

`public/fonts/Inter-OFL.txt`

Licencia de la fuente incluida en public/fonts.

- Se conserva íntegra junto con el recurso tipográfico correspondiente.

Código completo

93 líneas · SHA-256 del original: `f14f2b95a38f4f20cad4d27f7710593f37534c046641be0348da7c28365f4e39`

```
Copyright 2020 The Inter Project Authors (https://github.com/rsms/inter)

This Font Software is licensed under the SIL Open Font License, Version 1.1.
This license is copied below, and is also available with a FAQ at:
https://scripts.sil.org/OFL


-----------------------------------------------------------
SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007
-----------------------------------------------------------

PREAMBLE
The goals of the Open Font License (OFL) are to stimulate worldwide
development of collaborative font projects, to support the font creation
efforts of academic and linguistic communities, and to provide a free and
open framework in which fonts may be shared and improved in partnership
with others.

The OFL allows the licensed fonts to be used, studied, modified and
redistributed freely as long as they are not sold by themselves. The
fonts, including any derivative works, can be bundled, embedded,
redistributed and/or sold with any software provided that any reserved
names are not used by derivative works. The fonts and derivatives,
however, cannot be released under any other type of license. The
requirement for fonts to remain under this license does not apply
to any document created using the fonts or their derivatives.

DEFINITIONS
"Font Software" refers to the set of files released by the Copyright
Holder(s) under this license and clearly marked as such. This may
include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the
copyright statement(s).

"Original Version" refers to the collection of Font Software components as
distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting,
or substituting -- in part or in whole -- any of the components of the
```

```
Original Version, by changing formats or by porting the Font Software to a
new environment.

"Author" refers to any designer, engineer, programmer, technical
writer or other person who contributed to the Font Software.

PERMISSION & CONDITIONS
Permission is hereby granted, free of charge, to any person obtaining
a copy of the Font Software, to use, study, copy, merge, embed, modify,
redistribute, and sell modified and unmodified copies of the Font
Software, subject to the following conditions:

1) Neither the Font Software nor any of its individual components,
in Original or Modified Versions, may be sold by itself.

2) Original or Modified Versions of the Font Software may be bundled,
redistributed and/or sold with any software, provided that each copy
contains the above copyright notice and this license. These can be
included either as stand-alone text files, human-readable headers or
in the appropriate machine-readable metadata fields within text or
binary files as long as those fields can be easily viewed by the user.

3) No Modified Version of the Font Software may use the Reserved Font
Name(s) unless explicit written permission is granted by the corresponding
Copyright Holder. This restriction only applies to the primary font name as
presented to the users.

4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font
Software shall not be used to promote, endorse or advertise any
Modified Version, except to acknowledge the contribution(s) of the
Copyright Holder(s) and the Author(s) or with their explicit written
permission.

5) The Font Software, modified or unmodified, in part or in whole,
must be distributed entirely under this license, and must not be
distributed under any other license. The requirement for fonts to
remain under this license does not apply to any document created
using the Font Software.

TERMINATION
This license becomes null and void if any of the above conditions are
not met.

DISCLAIMER
THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT
OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE
COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL
DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM
OTHER DEALINGS IN THE FONT SOFTWARE.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 50 DE 62

Cormorant-Garamond-OFL.txt

`public/fonts/Cormorant-Garamond-OFL.txt`

Licencia de la fuente incluida en public/fonts.

- Se conserva íntegra junto con el recurso tipográfico correspondiente.

Código completo

93 líneas · SHA-256 del original: `b2306305139223cbc000c2b9feffaa741687056066c227ba34de31c98eda0814`

```
Copyright 2015 the Cormorant Project Authors (github.com/CatharsisFonts/Cormorant)

This Font Software is licensed under the SIL Open Font License, Version 1.1.
This license is copied below, and is also available with a FAQ at:
https://scripts.sil.org/OFL


-----------------------------------------------------------
SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007
-----------------------------------------------------------

PREAMBLE
The goals of the Open Font License (OFL) are to stimulate worldwide
development of collaborative font projects, to support the font creation
efforts of academic and linguistic communities, and to provide a free and
open framework in which fonts may be shared and improved in partnership
with others.

The OFL allows the licensed fonts to be used, studied, modified and
redistributed freely as long as they are not sold by themselves. The
fonts, including any derivative works, can be bundled, embedded,
redistributed and/or sold with any software provided that any reserved
names are not used by derivative works. The fonts and derivatives,
however, cannot be released under any other type of license. The
requirement for fonts to remain under this license does not apply
to any document created using the fonts or their derivatives.

DEFINITIONS
"Font Software" refers to the set of files released by the Copyright
Holder(s) under this license and clearly marked as such. This may
include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the
copyright statement(s).

"Original Version" refers to the collection of Font Software components as
distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting,
or substituting -- in part or in whole -- any of the components of the
```

```
Original Version, by changing formats or by porting the Font Software to a
new environment.

"Author" refers to any designer, engineer, programmer, technical
writer or other person who contributed to the Font Software.

PERMISSION & CONDITIONS
Permission is hereby granted, free of charge, to any person obtaining
a copy of the Font Software, to use, study, copy, merge, embed, modify,
redistribute, and sell modified and unmodified copies of the Font
Software, subject to the following conditions:

1) Neither the Font Software nor any of its individual components,
in Original or Modified Versions, may be sold by itself.

2) Original or Modified Versions of the Font Software may be bundled,
redistributed and/or sold with any software, provided that each copy
contains the above copyright notice and this license. These can be
included either as stand-alone text files, human-readable headers or
in the appropriate machine-readable metadata fields within text or
binary files as long as those fields can be easily viewed by the user.

3) No Modified Version of the Font Software may use the Reserved Font
Name(s) unless explicit written permission is granted by the corresponding
Copyright Holder. This restriction only applies to the primary font name as
presented to the users.

4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font
Software shall not be used to promote, endorse or advertise any
Modified Version, except to acknowledge the contribution(s) of the
Copyright Holder(s) and the Author(s) or with their explicit written
permission.

5) The Font Software, modified or unmodified, in part or in whole,
must be distributed entirely under this license, and must not be
distributed under any other license. The requirement for fonts to
remain under this license does not apply to any document created
using the Font Software.

TERMINATION
This license becomes null and void if any of the above conditions are
not met.

DISCLAIMER
THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT
OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE
COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL
DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM
OTHER DEALINGS IN THE FONT SOFTWARE.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 51 DE 62

inspiration.js

`src/data/inspiration.js`

Catálogos narrativos y campos del generador.

- Incluye nombres, apellidos, orígenes, trasfondos, objetivos, secretos y etiquetas de inspirationFields.

Código completo

48 líneas · SHA-256 del original: `a28167686bf45f1d2ce219beab92baee9e07c13a266724193aaf14d7f5483d36`

```
export const inspirationFields = [
  { key: 'portrait', label: 'Retrato' },
  { key: 'name', label: 'Nombre' },
  { key: 'type', label: 'Clase' },
  { key: 'race', label: 'Raza' },
  { key: 'level', label: 'Nivel' },
  { key: 'realm', label: 'Origen' },
  { key: 'traits', label: 'Personalidad' },
  { key: 'stats', label: 'Atributos' },
  { key: 'story', label: 'Historia' },
]

export const firstNames = ['Lyra', 'Aren', 'Nerea', 'Valen', 'Eira', 'Darian', 'Nyx', 'Ayla', 'Orin', 'Liora', 'Thane', 'Selene', 'Riven', 'Maelis', 'Iriel', 'Caelan']
export const lastNames = ['Sombraluna', 'del Alba', 'Noctis', 'de Ceniza', 'del Umbral', 'Valcor', 'de la Bruma', 'Cuervo Gris', 'de las Mareas', 'Aster', 'del Ocaso', 'Velo de Plata']
export const realms = ['Aethoria', 'Nox', 'Lúmina', 'Velmora', 'Valdren', 'Nerath', 'Puerto Ceniza', 'Cumbres de Vhal', 'Islas del Velo', 'Eldervan', 'Mirath', 'Valle del Eclipse']

export const backgrounds = [
  'Aprendió a reconocer los presagios que otros confundían con simples coincidencias.',
  'La desaparición de su familia dejó una llave sin cerradura y demasiadas preguntas.',
  'Durante años protegió una biblioteca cuyas páginas cambiaban al caer la noche.',
  'Un juramento olvidado volvió a reclamar su nombre durante el último eclipse.',
  'Encontró entre las ruinas un mapa que solo se puede leer en sueños.',
  'Una antigua deuda convirtió cada ciudad en un refugio provisional.',
  'Sobrevivió a una tormenta que borró de los mapas el lugar de su infancia.',
  'Un encuentro con su propia sombra le reveló que su destino todavía podía cambiar.',
]

export const goals = [
  'busca a la única persona capaz de descifrar el símbolo de su amuleto',
  'persigue una melodía que promete devolver la memoria a su pueblo',
  'debe entregar una carta antes de que su destinatario deje de existir',
  'intenta romper un pacto que su linaje firmó hace siglos',
  'sigue las huellas de una criatura que conoce su verdadero nombre',
  'quiere encontrar el santuario donde se guardan los recuerdos perdidos',
  'reúne fragmentos de una corona que nunca debió volver a forjarse',
  'busca una ciudad que aparece una sola noche cada cien años',
]
```

```
38
39 export const secrets = [
40   'El precio de la verdad podría ser el recuerdo que más aprecia.',
41   'Cada paso acerca también a alguien que juró no volver a encontrar.',
42   'Solo conserva una certeza: la persona que parece su enemiga guarda parte de la respuesta.',
43   'Pero su reflejo conoce un final distinto y empieza a actuar por su cuenta.',
44   'En su equipaje viaja un objeto que ninguna sombra se atreve a tocar.',
45   'Lo que todavía ignora es que su llegada ya estaba anunciada en las ruinas.',
46   'Para avanzar tendrá que elegir entre cumplir su promesa y proteger a quienes le siguen.',
47   'Una voz en sus sueños insiste en que esta aventura ya ocurrió una vez.',
48 ]
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 52 DE 62

CharacterDetail.css

`src/pages/CharacterDetail.css`

Estilos de la ficha local y su confirmación de borrado.

- Distribuye retrato, historia, rasgos, atributos y acciones en una pantalla con desplazamiento de página.

Código completo

57 líneas · SHA-256 del original: `4a3d70292297796b801fe2d6286fc9de4af502e187461a19e1fc36cbae915d19`

```
.character-detail {
  display: grid;
  gap: 20px;
  padding-bottom: 28px;
  min-width: 0;
}

.character-back {
  gap: 8px;
}

.character-detail > .character-portrait {
  aspect-ratio: 1;
  border: 1px solid var(--border);
  border-radius: 4px;
}

.character-detail-summary {
  color: var(--gold);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.8;
}

.character-detail-origin {
  color: var(--muted);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.8;
}

.character-detail-section {
  display: grid;
  gap: 14px;
  min-width: 0;
}

.character-detail .character-story {
  padding: 0;
  font-size: calc(14px * var(--text-scale));
}
```

```
40
41 .character-detail .trait-list,
42 .character-detail .attribute-grid {
43   margin: 0;
44 }
45
46 .character-delete-confirmation {
47   padding: 16px;
48   border: 1px solid var(--error);
49   border-radius: 4px;
50   background: var(--surface);
51 }
52
53 .character-delete-confirmation p {
54   color: var(--muted);
55   font-size: calc(13px * var(--text-scale));
56   line-height: 1.8;
57 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 53 DE 62

CharacterDetail.jsx

`src/pages/CharacterDetail.jsx`

Pantalla propia de cada ficha local, con acciones y confirmación de borrado.

Entradas / props. characters, onSave, onDelete, onToggleFavorite y preferences.

- Busca el id en characters; si no existe muestra NotFound.
- Conserva una ruta de retorno admitida desde Inicio o Biblioteca y sus filtros.
- Incluye lectura opcional, favoritos, edición, duplicado y eliminación con confirmación en la página.

Funciones y elementos principales

Nombre	Qué hace	Línea
`CharacterDetail`	Compone la ficha local.	10
`act`	Captura errores de las acciones.	29
`cancelDelete`	Cancela la confirmación y devuelve el foco.	34
`duplicate`	Guarda una copia con nuevo id y vuelve a Biblioteca.	39
`remove`	Elimina la ficha y vuelve a Biblioteca.	44

Código completo

77 líneas · SHA-256 del original: `62900754d529ee32dd7e3b4a7472c1e72f976a7766519e60282a06bed6bed394`

```
1  import { useEffect, useId, useRef, useState } from 'react'
2  import { Link, useLocation, useNavigate, useParams } from 'react-router'
3  import Portrait from '../components/charactercard/Portrait.jsx'
4  import ReadAloud from '../components/feedback/ReadAloud.jsx'
5  import Icon from '../components/icon/Icon.jsx'
6  import { attributeNames } from '../data/characterOptions.js'
7  import { NotFound } from './Information.jsx'
8  import './CharacterDetail.css'
9
10 export default function CharacterDetail({ characters, onSave, onDelete, onToggleFavorite, preferences })
   {
11   const { id } = useParams()
12   const { state } = useLocation()
13   const navigate = useNavigate()
14   const [error, setError] = useState('')
```

```
  const [confirmDelete, setConfirmDelete] = useState(false)
  const cancelButton = useRef(null)
  const deleteButton = useRef(null)
  const confirmationTitle = useId()
  const character = characters.find(item => item.id === id)
  const returnTo = typeof state?.returnTo === 'string' && (state.returnTo === '/inicio' || /^\/biblioteca(?:\?|$)/.test(state.returnTo)) ? state.returnTo : '/biblioteca'
  const libraryReturn = returnTo.startsWith('/biblioteca') ? returnTo : '/biblioteca'

  useEffect(() => {
    if (confirmDelete) cancelButton.current?.focus()
  }, [confirmDelete])

  if (!character) return <NotFound />

  function act(action) {
    try { action(); setError('') }
    catch (failure) { setError(failure.message) }
  }

  function cancelDelete() {
    setConfirmDelete(false)
    requestAnimationFrame(() => deleteButton.current?.focus())
  }

  function duplicate() {
    const copy = onSave({ ...character, id: undefined, name: `${character.name.slice(0, 52)} (copia)`, favorite: false })
    navigate(libraryReturn, { state: { message: `${copy.name}: personaje duplicado.` } })
  }

  function remove() {
    onDelete(character.id)
    navigate(libraryReturn, { replace: true, state: { message: `${character.name}: personaje eliminado.` } })
  }

  const readingText = [character.name, `${character.race}. ${character.type}. Nivel ${character.level}`, character.realm, character.story, `Rasgos: ${character.traits.join(', ')}`, ...attributeNames.map((name, index) => `${name}: ${character.stats[index]}`)].filter(Boolean).join('. ')

  return <article className="character-detail" aria-label={`Ficha de ${character.name}`}>
    <Link className="secondary-button character-back" to={returnTo}><Icon name="back" size={18} /> {returnTo === '/inicio' ? 'Volver a Inicio' : 'Volver a Biblioteca'}</Link>
    <Portrait character={character} />
    <p className="character-detail-summary">{character.race} · {character.type} · Nivel {character.level}</p>
    {character.realm && <p className="character-detail-origin">{character.realm}</p>}
    <section className="character-detail-section" aria-labelledby="character-story-title">
      <h2 id="character-story-title">Historia</h2>
      <p className="character-story">{character.story || 'Esta historia está por escribir.'}</p>
    </section>
    {preferences.readAloud && <ReadAloud text={readingText} rate={preferences.readingRate} />}
    {character.traits.length > 0 && <section className="character-detail-section" aria-labelledby="character-traits-title"><h2 id="character-traits-title">Rasgos</h2><ul className="trait-list">{character.traits.map(trait => <li key={trait}>{trait}</li>)}</ul></section>}
    <section className="character-detail-section" aria-labelledby="character-stats-title"><h2 id="character-stats-title">Atributos</h2><dl className="attribute-grid">{attributeNames.map((name, index) => <div key={name}><dt>{name}</dt><dd>{character.stats[index]}</dd></div>)}</dl></section>
```

```
63      <div className="action-stack">
64        <button className="secondary-button" type="button" aria-pressed={character.favorite} onClick={() =>
    act(() => onToggleFavorite(character.id))}>{character.favorite ? 'Quitar de favoritos' : 'Guardar en
    favoritos'}</button>
65        <Link className="primary-button" to=
    {`/personaje/${encodeURIComponent(character.id)}/editar`}>Editar personaje</Link>
66        <button className="secondary-button" type="button" onClick={() => act(duplicate)}>Duplicar
    personaje</button>
67        {!confirmDelete && <button ref={deleteButton} className="text-button danger-button" type="button"
    onClick={() => { setError(''); setConfirmDelete(true) }}>Eliminar personaje</button>}
68      </div>
69      {confirmDelete && <section className="character-delete-confirmation action-stack" aria-labelledby=
    {confirmationTitle} onKeyDown={event => { if (event.key === 'Escape') { event.stopPropagation();
    cancelDelete() } }}>
70        <h2 id={confirmationTitle}>¿Eliminar a {character.name}?</h2>
71        <p>La ficha se eliminará de esta biblioteca. Esta acción no se puede deshacer.</p>
72        <button ref={cancelButton} className="secondary-button" type="button" onClick=
    {cancelDelete}>Conservar personaje</button>
73        <button className="primary-button" type="button" onClick={() => act(remove)}>Eliminar
    definitivamente</button>
74      </section>}
75      {error && <p className="feedback error" role="alert">{error}</p>}
76    </article>
77  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 54 DE 62

Inspiration.css

`src/pages/Inspiration.css`

Estilos de la ficha editable de Inspiración.

- Presenta controles de fijación, campos, vista previa y mensajes según el tema y la accesibilidad.

Código completo

446 líneas · SHA-256 del original: `8c1a03ffbfdf43c93f0f132c7ab1cee43253e10f1dd9c2d3ce9891cb7b2400e4`

```
.inspiration-page {
  display: grid;
  gap: 16px;
  padding-bottom: 30px;
  min-width: 0;
}

.inspiration-intro {
  position: relative;
  display: grid;
  gap: 12px;
  padding: 24px 20px;
  border: 1px solid var(--border);
  border-radius: 6px;
  background: radial-gradient(ellipse at top right, var(--surface-raised), transparent 75%), var(--surface);
}

.inspiration-emblem {
  position: absolute;
  right: 16px;
  top: 16px;
  color: var(--gold);
  opacity: .65;
}

.inspiration-intro .eyebrow {
  padding-right: 35px;
  font-size: calc(9px * var(--text-scale));
  line-height: 1.7;
}

.inspiration-intro h2 {
  font-size: calc(29px * var(--text-scale));
}

.inspiration-intro > p:last-child,
.inspiration-guidance p,
.inspiration-create p {
  color: var(--muted);
```

```
  font-size: calc(12px * var(--text-scale));
  line-height: 1.8;
}

.inspiration-toolbar {
  position: sticky;
  top: 0;
  z-index: 1;
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  align-items: stretch;
  gap: 8px;
  padding: 10px 0;
  border-bottom: 1px solid var(--border);
  background: var(--background);
}

.inspiration-toolbar .primary-button {
  min-width: 0;
  font-size: calc(18px * var(--text-scale));
  line-height: 1.4;
}

.inspiration-toolbar svg,
.inspiration-create svg {
  flex-shrink: 0;
}

.inspiration-undo {
  width: var(--control-height);
  min-width: 44px;
  padding: 0;
}

.inspiration-page button:disabled {
  cursor: default;
}

.inspiration-guidance {
  display: grid;
  gap: 3px;
}

.inspiration-guidance .text-button {
  justify-self: start;
}

.inspiration-draft {
  display: grid;
  gap: 14px;
  min-width: 0;
}

.inspiration-draft-heading {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
```

```
  padding: 6px 0;
}

.inspiration-draft-heading .eyebrow {
  font-size: calc(9px * var(--text-scale));
  line-height: 1.8;
}

.inspiration-draft-badge {
  padding: 4px 9px;
  border: 1px solid var(--border);
  border-radius: 20px;
  color: var(--muted);
  font-size: calc(10px * var(--text-scale));
  line-height: 1.5;
}

.inspiration-field {
  min-width: 0;
  padding: 10px 14px 16px;
  border: 1px solid var(--border);
  border-radius: 5px;
  background: var(--surface);
  scroll-margin-top: 95px;
}

.inspiration-field[data-locked="true"] {
  border-color: var(--accent-ink);
}

.inspiration-field-heading {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: 4px;
  margin-bottom: 10px;
}

.inspiration-field-heading h3 {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 6px;
  color: var(--muted);
  font-size: calc(10px * var(--text-scale));
  line-height: 1.6;
  font-weight: 400;
  text-transform: uppercase;
  letter-spacing: .7px;
  overflow-wrap: anywhere;
}

.inspiration-fixed {
  color: var(--accent-ink);
  font-size: calc(9px * var(--text-scale));
  text-transform: none;
  letter-spacing: 0;
}
```

```
158
159 .inspiration-field-actions {
160   display: flex;
161   flex-shrink: 0;
162 }
163
164 .inspiration-field-actions .icon-button[aria-pressed="true"] {
165   color: var(--accent-ink);
166   background: var(--surface-raised);
167   border-radius: 3px;
168 }
169
170 .inspiration-portrait-card {
171   border: 1px solid var(--border);
172   border-radius: 6px;
173   overflow: hidden;
174 }
175
176 .inspiration-portrait-card .inspiration-field {
177   border-radius: 0;
178   border: 0;
179 }
180
181 .inspiration-portrait-card .inspiration-field[data-locked="true"] {
182   box-shadow: inset 3px 0 var(--accent-ink);
183 }
184
185 .inspiration-portrait-card .character-portrait {
186   width: 100%;
187   aspect-ratio: 1.3;
188   border-radius: 3px;
189   object-position: center 30%;
190 }
191
192 .inspiration-portrait-card .portrait-placeholder {
193   font-size: calc(64px * var(--text-scale));
194 }
195
196 .inspiration-portrait-caption {
197   padding-top: 8px;
198   color: var(--muted);
199   font-size: calc(10px * var(--text-scale));
200   line-height: 1.6;
201 }
202
203 .inspiration-name {
204   color: var(--foreground);
205   font-family: var(--font-display);
206   font-size: calc(29px * var(--text-scale));
207   line-height: 1.2;
208 }
209
210 .inspiration-fields {
211   display: grid;
212   grid-template-columns: repeat(2, minmax(0, 1fr));
213   gap: 12px;
214 }
215
216 .inspiration-fields .inspiration-field {
```

```
  padding: 8px 10px 16px;
}

.inspiration-value {
  color: var(--foreground);
  font-family: var(--font-display);
  font-size: calc(23px * var(--text-scale));
  line-height: 1.3;
}

.inspiration-level {
  color: var(--muted);
  font-size: calc(12px * var(--text-scale));
}

.inspiration-level strong {
  color: var(--gold);
  font-family: var(--font-display);
  font-size: calc(34px * var(--text-scale));
  font-weight: 400;
}

.inspiration-traits {
  margin: 0;
}

.inspiration-input-label {
  display: block;
  margin: 14px 0 7px;
  color: var(--muted);
  font-size: calc(12px * var(--text-scale));
  line-height: 1.7;
}

.inspiration-page input,
.inspiration-page select,
.inspiration-page textarea {
  display: block;
  width: 100%;
  min-width: 0;
}

.inspiration-page input.inspiration-name {
  font-family: var(--font-display);
  font-size: calc(26px * var(--text-scale));
}

.inspiration-page .inspiration-fields .inspiration-value {
  font-size: calc(18px * var(--text-scale));
}

.inspiration-page textarea.inspiration-story {
  font-size: calc(16px * var(--text-scale));
  line-height: 1.8;
  min-height: 210px;
}

.inspiration-number {
  display: flex;
```

```
  align-items: center;
  gap: 6px;
  min-width: 0;
}

.inspiration-number input[type="number"] {
  flex: 1;
  width: 0;
  min-width: 0;
  color: var(--gold);
  font-family: var(--font-display);
  font-size: calc(26px * var(--text-scale));
  line-height: 1.2;
  appearance: textfield;
}

.inspiration-number input::-webkit-inner-spin-button,
.inspiration-number input::-webkit-outer-spin-button {
  -webkit-appearance: none;
  margin: 0;
}

.inspiration-number > span {
  flex-shrink: 0;
  color: var(--muted);
  font-family: 'Inter', sans-serif;
  font-size: calc(11px * var(--text-scale));
}

.inspiration-traits li {
  padding: 0;
}

.inspiration-traits button {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  min-height: var(--control-height);
  padding: 6px 10px;
  background: transparent;
  color: var(--foreground);
  font-size: calc(12px * var(--text-scale));
}

.inspiration-traits button svg {
  flex-shrink: 0;
}

.inspiration-trait-picker {
  margin-top: 14px;
  border-top: 1px solid var(--border);
}

.inspiration-trait-picker summary {
  min-height: var(--control-height);
  padding: 12px 0;
  color: var(--accent-ink);
  font-size: calc(13px * var(--text-scale));
  line-height: 1.7;
```

```
  cursor: pointer;
}

.inspiration-trait-options {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
  padding-bottom: 12px;
}

.inspiration-trait-options .checkbox-label {
  color: var(--foreground);
  font-size: calc(12px * var(--text-scale));
}

.inspiration-trait-options input[type="checkbox"] {
  width: 20px;
}

.inspiration-add-trait {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 8px;
  margin-bottom: 8px;
}

.inspiration-field > .field-error {
  margin-top: 8px;
}

.inspiration-stats {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
  margin: 0;
}

.inspiration-stat {
  min-width: 0;
  padding: 12px;
  border: 1px solid var(--border);
  border-radius: 4px;
  background: var(--background);
}

.inspiration-stat dt {
  color: var(--muted);
  font-size: calc(11px * var(--text-scale));
  line-height: 1.6;
  overflow-wrap: anywhere;
}

.inspiration-stat dd {
  margin: 8px 0;
  color: var(--gold);
  font-family: var(--font-display);
  font-size: calc(32px * var(--text-scale));
  line-height: 1.2;
}
```

```
394
395 .inspiration-stat dd span {
396   font-family: 'Inter', sans-serif;
397   color: var(--muted);
398   font-size: calc(10px * var(--text-scale));
399 }
400
401 .inspiration-stat-track {
402   height: 4px;
403   border-radius: 3px;
404   overflow: hidden;
405   background: var(--border);
406 }
407
408 .inspiration-stat-track > span {
409   display: block;
410   height: 100%;
411   border-radius: inherit;
412   background: var(--gold);
413 }
414
415 .inspiration-story {
416   color: var(--muted);
417   font-size: calc(13px * var(--text-scale));
418   line-height: 1.9;
419 }
420
421 .inspiration-create {
422   display: grid;
423   gap: 12px;
424   padding-top: 8px;
425 }
426
427 .inspiration-create .primary-button {
428   line-height: 1.4;
429   text-align: center;
430 }
431
432 .app-frame[data-element-size="large"] .inspiration-fields,
433 .app-frame[data-element-size="extra-large"] .inspiration-fields,
434 .app-frame[data-element-size="large"] .inspiration-stats,
435 .app-frame[data-element-size="extra-large"] .inspiration-stats,
436 .app-frame[data-text-size="extra-large"] .inspiration-trait-options {
437   grid-template-columns: 1fr;
438 }
439
440 @media (max-width: 350px) {
441   .inspiration-fields,
442   .inspiration-stats,
443   .inspiration-trait-options {
444     grid-template-columns: 1fr;
445   }
446 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 55 DE 62

Inspiration.jsx

`src/pages/Inspiration.jsx`

Generador visual editable que guarda directamente en la biblioteca local.

Entradas / props. preferences y onSave.

- Genera todos los campos, permite regenerarlos por separado y conserva los campos fijados.
- Una edición manual fija el campo automáticamente; permite subir retrato, añadir rasgos y recuperar ideas anteriores.
- Valida con validateForm y guarda mediante onSave sin pasar obligatoriamente por el editor.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Field`	Sección editable con bloqueo y regeneración.	23
`Inspiration`	Gestiona borrador, historial, validación y guardado.	38

Código completo

238 líneas · SHA-256 del original: `26b8caab32f88d84c1a740a519140ee25c543c226935e71d1ea9fdd46fcba606`

```
1  import { useRef, useState } from 'react'
2  import { useNavigate } from 'react-router'
3  import Icon from '../components/icon/Icon.jsx'
4  import Portrait from '../components/charactercard/Portrait.jsx'
5  import ReadAloud from '../components/feedback/ReadAloud.jsx'
6  import { attributeNames, characterRaces, characterTypes, personalityTraits } from
   '../data/characterOptions.js'
7  import { inspirationFields } from '../data/inspiration.js'
8  import { randomCharacter } from '../utils/generateCharacter.js'
9  import { readPortrait } from '../utils/readPortrait.js'
10 import { validateForm } from '../utils/validateForm.js'
11 import './Inspiration.css'
12
13 const fields = [
14   { name: 'name', label: 'Nombre', required: true, minLength: 2, maxLength: 60 },
15   { name: 'type', label: 'Clase', required: true, options: characterTypes },
16   { name: 'race', label: 'Raza', required: true, options: characterRaces },
17   { name: 'level', label: 'Nivel', type: 'number', required: true, min: 1, max: 20 },
18   { name: 'realm', label: 'Origen', maxLength: 100 },
19   { name: 'story', label: 'Historia', maxLength: 3000 },
20   ...attributeNames.map((label, index) => ({ name: `stat${index}`, label, type: 'number', required: true,
   min: 1, max: 20 })),
```

```
]

function Field({ field, locked, onLock, onChange, onEditEnd, error, children, className = '' }) {
  const label = inspirationFields.find(item => item.key === field).label
  return <section className={`inspiration-field ${className}`} data-field={field} data-locked={locked} aria-labelledby={`inspiration-${field}-title`} onBlur={event => { if (!event.currentTarget.contains(event.relatedTarget)) onEditEnd(field) }}>
    <div className="inspiration-field-heading">
      <h3 id={`inspiration-${field}-title`}>{label}{locked && <span className="inspiration-fixed">Fijado</span>}</h3>
      <div className="inspiration-field-actions">
        <button className="icon-button" type="button" aria-label={`${locked ? 'Desbloquear' : 'Fijar'} ${label.toLocaleLowerCase('es')}`} aria-pressed={locked} title={locked ? `Desbloquear ${label.toLocaleLowerCase('es')}` : `Conservar ${label.toLocaleLowerCase('es')} al generar otra idea`} onClick={() => onLock(field)}><Icon name={locked ? 'lock' : 'unlock'} size={17} /></button>
        <button className="icon-button" type="button" aria-label={`Cambiar ${label.toLocaleLowerCase('es')}`} title={locked ? 'Desbloquea este campo para cambiarlo' : `Generar ${label.toLocaleLowerCase('es')}`} disabled={locked} onClick={() => onChange(field)}><Icon name="dice" size={19} /></button>
      </div>
    </div>
    {children}
    {error && <p className="field-error" id={`inspiration-${field}-error`}>{error}</p>}
  </section>
}

export default function Inspiration({ preferences, onSave }) {
  const navigate = useNavigate()
  const form = useRef(null)
  const saveInProgress = useRef(false)
  const [editingField, setEditingField] = useState(null)
  const [character, setCharacter] = useState(() => randomCharacter())
  const [lockedFields, setLockedFields] = useState([])
  const [history, setHistory] = useState([])
  const [message, setMessage] = useState('')
  const [submitted, setSubmitted] = useState(false)
  const [customTrait, setCustomTrait] = useState('')
  const [traitError, setTraitError] = useState('')
  const [imageError, setImageError] = useState('')
  const [readingImage, setReadingImage] = useState(false)
  const [saving, setSaving] = useState(false)
  const [saveError, setSaveError] = useState('')
  const allLocked = lockedFields.length === inspirationFields.length
  const values = new Map(fields.map(field => [field.name, String(field.name.startsWith('stat') ? character.stats[Number(field.name.slice(4))] : character[field.name])]))
  const errors = validateForm(fields, values)

  function remember() {
    setHistory(previous => [...previous.slice(-9), character])
  }

  function edit(field, changes) {
    if (editingField !== field) {
      remember()
      setEditingField(field)
    }
    setCharacter(previous => ({ ...previous, ...changes }))
    setLockedFields(previous => previous.includes(field) ? previous : [...previous, field])
    setSubmitted(false)
```

```
    if (field === 'portrait') setImageError('')
  }

  function finishEditing(field) {
    if (editingField !== field) return
    setEditingField(null)
    const label = inspirationFields.find(item => item.key === field).label
    setMessage(`${label} editado y fijado. Puedes desbloquearlo para volver a generarlo.`)
  }

  function generate(field) {
    const preserve = field ? inspirationFields.filter(item => item.key !== field).map(item => item.key) : lockedFields
    const next = randomCharacter(character, preserve)
    remember()
    setEditingField(null)
    setCharacter(next)
    setSubmitted(false)
    const label = inspirationFields.find(item => item.key === field)?.label
    setMessage(field ? `${label}: nueva propuesta generada.` : `Nueva inspiración: ${next.name}. ${lockedFields.length ? 'Se han conservado los campos fijados.' : 'Todos los campos tienen una nueva combinación.'}`)
  }

  function toggleLock(field) {
    const label = inspirationFields.find(item => item.key === field).label
    setLockedFields(previous => previous.includes(field) ? previous.filter(item => item !== field) : [...previous, field])
    setMessage(`${label}: ${lockedFields.includes(field) ? 'desbloqueado' : 'se conservará al generar otra idea'}.`)
  }

  function undo() {
    const previous = history.at(-1)
    if (!previous) return
    setCharacter(previous)
    setEditingField(null)
    setSubmitted(false)
    setImageError('')
    setHistory(items => items.slice(0, -1))
    setMessage(`Idea anterior recuperada: ${previous.name}.`)
  }

  async function upload(event) {
    const input = event.currentTarget
    const file = input.files[0]
    if (!file) return
    setImageError('')
    setReadingImage(true)
    try {
      edit('portrait', await readPortrait(file))
      setMessage('Retrato actualizado y fijado.')
    } catch (failure) { setImageError(failure.message) }
    finally { input.value = ''; setReadingImage(false) }
  }

  function toggleTrait(trait) {
    const selected = character.traits.includes(trait)
    if (!selected && character.traits.length >= 6) return
```

```
124     edit('traits', { traits: selected ? character.traits.filter(item => item !== trait) :
    [...character.traits, trait] })
125     setTraitError('')
126   }
127
128   function addTrait() {
129     const trait = customTrait.trim()
130     if (!trait) return
131     if (character.traits.some(item => item.toLocaleLowerCase('es') === trait.toLocaleLowerCase('es'))) {
132       setTraitError('Ese rasgo ya está seleccionado.')
133       return
134     }
135     if (character.traits.length >= 6) { setTraitError('Puedes elegir hasta seis rasgos.'); return }
136     edit('traits', { traits: [...character.traits, trait] })
137     setCustomTrait('')
138     setTraitError('')
139   }
140
141   async function handleSave(event) {
142     event.preventDefault()
143     if (readingImage || saveInProgress.current) return
144     setSaveError('')
145     if (Object.keys(errors).length) {
146       setSubmitted(true)
147       requestAnimationFrame(() => {
148         const invalid = form.current?.querySelector('[aria-invalid="true"]')
149         invalid?.focus()
150         invalid?.scrollIntoView({ block: 'center' })
151       })
152       return
153     }
154     saveInProgress.current = true
155     setSaving(true)
156     try {
157       const saved = await onSave({ ...character, id: undefined, name: character.name.trim(), level:
    Number(character.level), stats: character.stats.map(Number) })
158       navigate('/biblioteca', { state: { message: `${saved.name}: personaje guardado desde Inspiración.`
    } })
159     } catch (failure) {
160       setSaveError(failure?.message || 'No se pudo guardar el personaje. Vuelve a intentarlo.')
161     } finally {
162       saveInProgress.current = false
163       setSaving(false)
164     }
165   }
166
167   const fieldProps = field => ({ field, locked: lockedFields.includes(field), onLock: toggleLock,
    onChange: () => generate(field), onEditEnd: () => finishEditing(field), error: errors[field] })
168   const inputProps = name => ({ id: `inspiration-input-${name}`, name, 'aria-labelledby':
    `inspiration-${name}-title`, 'aria-invalid': Boolean(errors[name]), 'aria-describedby': errors[name] ?
    `inspiration-${name}-error` : undefined })
169   const readingText = [character.name, character.type, character.race, `Nivel ${character.level}`,
    character.realm, character.story, `Rasgos: ${character.traits.join(', ')}`, ...attributeNames.map((name,
    index) => `${name}: ${character.stats[index]}`)].join('. ')
170
171   return <form ref={form} className="inspiration-page" noValidate onSubmit={handleSave}>
172     <fieldset className="form-contents" disabled={readingImage || saving} aria-label="Generador y edición
    del personaje">
173     <section className="inspiration-intro" aria-labelledby="inspiration-title">
```

```
      <span className="inspiration-emblem" aria-hidden="true"><Icon name="dice" size={32} /></span>
      <p className="eyebrow">EL AZAR TIENE UNA HISTORIA</p>
      <h2 id="inspiration-title">Una chispa.<br />Mil posibles leyendas.</h2>
      <p>Genera una ficha y edita cualquier campo aquí mismo. Lo que escribas quedará fijado para conservarlo.</p>
    </section>

    <div className="inspiration-toolbar">
      <button className="primary-button" type="button" onClick={() => generate()} disabled={allLocked} aria-describedby="inspiration-lock-hint"><Icon name="dice" size={20} />Otra inspiración</button>
      <button className="secondary-button inspiration-undo" type="button" onClick={undo} disabled={!history.length} aria-label="Recuperar idea anterior" title="Recuperar idea anterior"><Icon name="undo" size={20} /></button>
    </div>
    <div className="inspiration-guidance">
      <p id="inspiration-lock-hint">{allLocked ? 'Todos los campos están fijados. Puedes seguir editándolos o desbloquear alguno para volver a generar.' : 'Escribe directamente para editar. El dado genera otro valor y el candado lo conserva.'}</p>
      {lockedFields.length > 0 && <button className="text-button" type="button" onClick={() => { setLockedFields([]); setMessage('Todos los campos están desbloqueados.') }}>Quitar bloqueos ({lockedFields.length})</button>}
    </div>
    <p className="visually-hidden" role="status" aria-live="polite" aria-atomic="true">{message}</p>

    <article className="inspiration-draft" aria-label={`Borrador de ${character.name || 'tu personaje'}`}>
      <div className="inspiration-draft-heading"><h2 className="eyebrow">TU PRÓXIMO PERSONAJE</h2><span className="inspiration-draft-badge">Sin guardar</span></div>
      <div className="inspiration-portrait-card">
        <Field {...fieldProps('portrait')}>
          <Portrait character={{ ...character, name: character.name || 'tu personaje' }} />
          <p className="inspiration-portrait-caption">{character.imageDataUrl ? character.imageName : ['Retrato de hechicera', 'Retrato de pícaro', 'Iniciales del personaje'][character.portrait]}</p>
          <label className="inspiration-input-label" htmlFor="inspiration-portrait">Estilo de retrato</label>
          <select id="inspiration-portrait" name="portrait" value={character.portrait} onChange={event => edit('portrait', { portrait: Number(event.target.value), imageDataUrl: '', imageName: '' })}><option value={0}>Hechicera</option><option value={1}>Pícaro</option><option value={2}>Iniciales</option></select>
          <label className="inspiration-input-label" htmlFor="inspiration-image">Subir tu imagen</label>
          <input id="inspiration-image" name="portraitFile" type="file" accept="image/png,image/jpeg,image/webp" onChange={upload} aria-describedby="inspiration-image-hint" />
          <p className="form-hint" id="inspiration-image-hint">PNG, JPG o WebP. Hasta 750 KB.</p>
          {character.imageDataUrl && <button className="secondary-button" type="button" onClick={() => edit('portrait', { imageDataUrl: '', imageName: '' })}>Quitar imagen subida</button>}
          {imageError && <p className="field-error" role="alert">{imageError}</p>}
        </Field>
        <Field {...fieldProps('name')}><input {...inputProps('name')} className="inspiration-name" type="text" required minLength={2} maxLength={60} value={character.name} onChange={event => edit('name', { name: event.target.value })} /></Field>
      </div>

      <div className="inspiration-fields">
        <Field {...fieldProps('type')}><select {...inputProps('type')} className="inspiration-value" value={character.type} onChange={event => edit('type', { type: event.target.value })}>{characterTypes.map(type => <option key={type}>{type}</option>)}</select></Field>
        <Field {...fieldProps('race')}><select {...inputProps('race')} className="inspiration-value" value={character.race} onChange={event => edit('race', { race: event.target.value })}>{characterRaces.map(race => <option key={race}>{race}</option>)}</select></Field>
```

```
        <Field {...fieldProps('level')}><div className="inspiration-number"><input {...inputProps('level')} type="number" min={1} max={20} step={1} required value={character.level} onChange={event => edit('level', { level: event.target.value })} /><span aria-hidden="true">/ 20</span></div></Field>
        <Field {...fieldProps('realm')}><input {...inputProps('realm')} className="inspiration-value" type="text" maxLength={100} value={character.realm} onChange={event => edit('realm', { realm: event.target.value })} /></Field>
      </div>

      <Field {...fieldProps('traits')}>
        <ul className="trait-list inspiration-traits">{character.traits.map(trait => <li key={trait}><button type="button" aria-label={`Quitar rasgo ${trait}`} onClick={() => toggleTrait(trait)}>{trait}<Icon name="close" size={13} /></button></li>)}</ul>
        <details className="inspiration-trait-picker"><summary>Elegir rasgos ({character.traits.length}/6)</summary><div className="inspiration-trait-options">{[...new Set([...personalityTraits, ...character.traits])].map(trait => <label className="checkbox-label" key={trait}><input type="checkbox" name="traits" value={trait} checked={character.traits.includes(trait)} disabled={character.traits.length >= 6 && !character.traits.includes(trait)} onChange={() => toggleTrait(trait)} />{trait}</label>)}</div></details>
        <label className="inspiration-input-label" htmlFor="inspiration-custom-trait">Añadir un rasgo propio</label>
        <div className="inspiration-add-trait"><input id="inspiration-custom-trait" name="customTrait" type="text" maxLength={40} value={customTrait} placeholder="Por ejemplo, Curiosa" onChange={event => { setCustomTrait(event.target.value); setTraitError('') }} onKeyDown={event => { if (event.key === 'Enter') { event.preventDefault(); addTrait() } }} aria-describedby="inspiration-traits-hint" /><button className="secondary-button" type="button" disabled={!customTrait.trim() || character.traits.length >= 6} onClick={addTrait}>Añadir</button></div>
        <p className="form-hint" id="inspiration-traits-hint">Hasta seis rasgos. Pulsa un rasgo seleccionado para quitarlo.</p>
        {traitError && <p className="field-error" role="alert">{traitError}</p>}
      </Field>
      <Field {...fieldProps('stats')}>
        <dl className="inspiration-stats">{attributeNames.map((name, index) => <div className="inspiration-stat" key={name}><dt><label htmlFor={`inspiration-stat${index}`}>{name}</label></dt><dd><div className="inspiration-number"><input id={`inspiration-stat${index}`} name={`stat${index}`} type="number" min={1} max={20} step={1} required value={character.stats[index]} onChange={event => edit('stats', { stats: character.stats.map((stat, position) => position === index ? event.target.value : stat) })} aria-invalid={Boolean(errors[`stat${index}`])} aria-describedby={errors[`stat${index}`] ? `inspiration-stat${index}-error` : undefined} /><span aria-hidden="true">/ 20</span></div>{errors[`stat${index}`] && <p className="field-error" id={`inspiration-stat${index}-error`}>{errors[`stat${index}`]}</p>}</dd><div className="inspiration-stat-track" aria-hidden="true"><span style={{ width: `${Math.max(0, Math.min(20, Number(character.stats[index]) || 0)) / 20 * 100}%` }} /></div></div>)}</dl>
      </Field>
      <Field {...fieldProps('story')}><textarea {...inputProps('story')} className="inspiration-story" rows={7} maxLength={3000} value={character.story} onChange={event => edit('story', { story: event.target.value })} /><p className="form-hint">{character.story.length}/3000 caracteres</p></Field>
      {preferences.readAloud && <ReadAloud key={readingText} text={readingText} rate={preferences.readingRate} />}
    </article>
    </fieldset>

    <div className="inspiration-create">
      {readingImage && <p className="feedback" role="status">Cargando retrato…</p>}
      {submitted && Object.keys(errors).length > 0 && <p className="feedback error" role="alert">Revisa los campos indicados antes de guardar.</p>}
      {saveError && <p className="feedback error" role="alert">{saveError}</p>}
      <button className="primary-button" type="submit" disabled={readingImage || saving}><Icon name="book" size={18} />{saving ? 'Guardando…' : 'Guardar personaje'}</button>
```

```
235       <p>El personaje se guardará directamente en la biblioteca de este perfil.</p>
236     </div>
237   </form>
238 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 56 DE 62

PublicCharacterDetail.jsx

`src/pages/PublicCharacterDetail.jsx`

Consulta por id una ficha del backend.

Estado. personaje, cargando y error.

- Obtiene el id de la ruta y consulta obtenerPersonajePorId; limpia la ficha al cambiar id y evita actualizaciones después del desmontaje.
- Muestra nombre, visibilidad, historia, personalidad y fuerza, destreza, inteligencia y carisma.
- Incluye carga, error y enlace a Biblioteca. No ofrece edición, favoritos o duplicado.

Funciones y elementos principales

Nombre	Qué hace	Línea
`PublicCharacterDetail`	Presenta el detalle público y sus estados de consulta.	5

Código completo

68 líneas · SHA-256 del original: `16344be095736ef5e8c3a20c0692dd0099c2a90b58c12f1e07eb3af164579213`

```
import { useEffect, useState } from 'react'
import { Link, useParams } from 'react-router'
import { obtenerPersonajePorId } from '../services/api.js'

export default function PublicCharacterDetail() {
  const { id } = useParams()

  const [personaje, setPersonaje] = useState(null)
  const [cargando, setCargando] = useState(true)
  const [error, setError] = useState('')

  useEffect(() => {
    let activo = true

    setCargando(true)
    setError('')
    setPersonaje(null)

    obtenerPersonajePorId(id)
      .then(datos => {
        if (activo) setPersonaje(datos)
      })
      .catch(() => {
        if (activo) setError('No se pudo cargar el personaje.')
```

```
      })
      .finally(() => {
        if (activo) setCargando(false)
      })

    return () => {
      activo = false
    }
  }, [id])

  return (
    <section className="character-detail-page">
      <Link to="/biblioteca" className="secondary-button">
        Volver a biblioteca
      </Link>

      {cargando && <p role="status">Cargando ficha...</p>}

      {error && <p role="alert">{error}</p>}

      {personaje && (
        <article>
          <h2>{personaje.nombre}</h2>

          <p>{personaje.visibilidad}</p>

          <h3>Historia</h3>
          <p>{personaje.historia}</p>

          <h3>Personalidad</h3>
          <p>{personaje.personalidad}</p>

          <h3>Atributos</h3>
          <ul>
            <li>Fuerza: {personaje.fuerza}</li>
            <li>Destreza: {personaje.destreza}</li>
            <li>Inteligencia: {personaje.inteligencia}</li>
            <li>Carisma: {personaje.carisma}</li>
          </ul>
        </article>
      )}
    </section>
  )
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 57 DE 62

api.js

`src/services/api.js`

Servicios fetch para personajes públicos y autenticación de Spring Boot.

- Utiliza rutas relativas /api; no consume VITE_API_URL.
- El registro envía nombreUsuario, email y password como JSON; distingue errores 409 y 400.
- Login envía email y password, sin remember; login, me y logout declaran credentials: same-origin.
- GET /api/auth/me añade X-Requested-By: XMLHttpRequest y devuelve null ante 401.
- Registro, login, sesión y personajes esperan JSON en respuestas correctas; logout solo comprueba el estado HTTP.

Funciones y elementos principales

Nombre	Qué hace	Línea
`obtenerPersonajesPublicos`	GET /api/personajes/publicos.	1
`obtenerPersonajePorId`	GET /api/personajes/{id}, codificando el id.	11
`registrarUsuario`	POST /api/auth/register y traducción de errores.	20
`iniciarSesion`	POST /api/auth/login y error específico para 401.	48
`obtenerSesion`	GET /api/auth/me; 401 devuelve null.	72
`cerrarSesion`	POST /api/auth/logout; lanza error si falla.	91

Código completo

100 líneas · SHA-256 del original: `684a8abfa6a73cb4f8064d6a80a923171e32f9d712b311b59e768900130913f3`

```
export async function obtenerPersonajesPublicos() {
  const respuesta = await fetch('/api/personajes/publicos')

  if (!respuesta.ok) {
    throw new Error(`Error al obtener los personajes públicos: ${respuesta.status}`)
  }

  return respuesta.json()
}

```

```
export async function obtenerPersonajePorId(id) {
  const respuesta = await fetch(`/api/personajes/${encodeURIComponent(id)}`)

  if (!respuesta.ok) {
    throw new Error(`Error al obtener el personaje: ${respuesta.status}`)
  }

  return respuesta.json()
}
export async function registrarUsuario(datos) {
  const respuesta = await fetch('/api/auth/register', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      nombreUsuario: datos.name,
      email: datos.email,
      password: datos.password,
    }),
  })

  if (respuesta.status === 409) {
    throw new Error('Este correo electrónico ya está registrado.')
  }

  if (respuesta.status === 400) {
    throw new Error('Revisa los datos introducidos.')
  }

  if (!respuesta.ok) {
    throw new Error('No se pudo completar el registro.')
  }

  return respuesta.json()
}

export async function iniciarSesion(datos) {
  const respuesta = await fetch('/api/auth/login', {
    method: 'POST',
    credentials: 'same-origin',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      email: datos.email,
      password: datos.password,
    }),
  })

  if (respuesta.status === 401) {
    throw new Error('El correo o la contraseña son incorrectos.')
  }

  if (!respuesta.ok) {
    throw new Error('No se pudo iniciar sesión.')
  }

  return respuesta.json()
```

```
}

export async function obtenerSesion() {
  const respuesta = await fetch('/api/auth/me', {
    credentials: 'same-origin',
    headers: {
      'X-Requested-By': 'XMLHttpRequest',
    },
  })

  if (respuesta.status === 401) {
    return null
  }

  if (!respuesta.ok) {
    throw new Error('No se pudo comprobar la sesión.')
  }

  return respuesta.json()
}

export async function cerrarSesion() {
  const respuesta = await fetch('/api/auth/logout', {
    method: 'POST',
    credentials: 'same-origin',
  })

  if (!respuesta.ok) {
    throw new Error('No se pudo cerrar sesión.')
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 58 DE 62

generateCharacter.js

`src/utils/generateCharacter.js`

Genera todos los campos respetando bloqueos y evitando repetir valores anteriores.

- randomCharacter recibe previous, lockedFields y una función random opcional.
- Genera nombre, clase, raza, nivel, origen, retrato, rasgos, historia y cuatro atributos de 1 a 20.
- Copia los arrays de campos conservados y no escribe en almacenamiento.

Funciones y elementos principales

Nombre	Qué hace	Línea
`randomCharacter`	Devuelve un borrador completo a partir de catálogos y campos fijados.	6

Código completo

47 líneas · SHA-256 del original: `35e82d7bcd5992faa82e59d5ded05e33a2a497f74e22eb95ad3c969a196b81d2`

```
1  import { blankCharacter, characterRaces, characterTypes, personalityTraits } from
   '../data/characterOptions.js'
2  import { backgrounds, firstNames, goals, lastNames, realms, secrets } from '../data/inspiration.js'
3
4  const names = firstNames.flatMap(first => lastNames.map(last => `${first} ${last}`))
5
6  export function randomCharacter(previous = {}, lockedFields = [], random = Math.random) {
7    const locked = new Set(lockedFields)
8    const pick = (options, exclude) => {
9      const choices = options.filter(option => option !== exclude)
10     return choices[Math.floor(random() * choices.length)]
11   }
12   const integer = (min, max, exclude) => pick(Array.from({ length: max - min + 1 }, (_, index) => min +
   index), exclude)
13   const keep = key => locked.has(key) && previous[key] !== undefined
14   const draft = {
15     ...blankCharacter(),
16     name: keep('name') ? previous.name : pick(names, previous.name),
17     type: keep('type') ? previous.type : pick(characterTypes, previous.type),
18     race: keep('race') ? previous.race : pick(characterRaces, previous.race),
19     level: keep('level') ? previous.level : integer(1, 20, previous.level),
20     realm: keep('realm') ? previous.realm : pick(realms, previous.realm),
21     portrait: keep('portrait') ? previous.portrait : pick([0, 1, 2], previous.portrait),
22     imageDataUrl: keep('portrait') ? previous.imageDataUrl || '' : '',
23     imageName: keep('portrait') ? previous.imageName || '' : '',
24   }
25
26   if (keep('traits')) {
```

```
    draft.traits = [...previous.traits]
  } else {
    const available = [...personalityTraits]
    const count = integer(2, 4)
    draft.traits = Array.from({ length: count }, () => available.splice(Math.floor(random() * available.length), 1)[0])
    if (draft.traits.length === previous.traits?.length && draft.traits.every(trait => previous.traits.includes(trait))) {
      draft.traits[draft.traits.length - 1] = pick(available)
    }
  }

  draft.stats = keep('stats') ? [...previous.stats] : Array.from({ length: 4 }, (_, index) => integer(1, 20, previous.stats?.[index]))

  if (keep('story')) {
    draft.story = previous.story
  } else {
    const opening = pick(backgrounds.filter(text => !previous.story?.startsWith(text)))
    draft.story = `${opening} Ahora ${pick(goals)}. ${pick(secrets)}`
  }

  return draft
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 59 DE 62

readPortrait.js

`src/utils/readPortrait.js`

Lectura y comprobación compartida de retratos subidos.

- Valida tipo PNG/JPEG/WebP, tamaño máximo de 750 KB y decodificación de la imagen.
- Devuelve imageDataUrl e imageName para guardar en la ficha.

Funciones y elementos principales

Nombre	Qué hace	Línea
`readPortrait`	Lee y valida una imagen sin persistirla.	3

Código completo

20 líneas · SHA-256 del original: `e10bb7dced00f58044238c1c3bf80a9a61090faa9bf26247cd0d6b020664a8eb`

```
1  import { MAX_IMAGE_BYTES } from './characterStore.js'
2
3  export async function readPortrait(file) {
4    if (!['image/png', 'image/jpeg', 'image/webp'].includes(file.type) || file.size > MAX_IMAGE_BYTES) {
5      throw new Error('Selecciona una imagen PNG, JPG o WebP de hasta 750 KB.')
6    }
7    const imageDataUrl = await new Promise((resolve, reject) => {
8      const reader = new FileReader()
9      reader.onload = () => resolve(reader.result)
10     reader.onerror = () => reject(new Error('No se pudo leer el retrato.'))
11     reader.readAsDataURL(file)
12   })
13   await new Promise((resolve, reject) => {
14     const preview = new Image()
15     preview.onload = resolve
16     preview.onerror = () => reject(new Error('El archivo no contiene una imagen válida.'))
17     preview.src = imageDataUrl
18   })
19   return { imageDataUrl, imageName: file.name }
20 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 60 DE 62

PublicCharacters.jsx

`src/components/publiccharacters/PublicCharacters.jsx`

Listado remoto independiente al final de la biblioteca.

Estado. personajes, cargando y error.

- Al montar consulta obtenerPersonajesPublicos; muestra carga, error o lista vacía y evita actualizar tras desmontarse.
- Adapta idPersonaje a public-<id> y nombre a name, con type Personaje público, level null, image vacío, portrait null y favorite false.
- Envuelve CharacterCard interactive=false en un enlace a /personajes-publicos/:id. Sin imagen asignada muestra iniciales.
- Los filtros de la biblioteca personal no se aplican a esta consulta.

Funciones y elementos principales

Nombre	Qué hace	Línea
`PublicCharacters`	Carga y representa las tarjetas públicas.	7

Código completo

82 líneas · SHA-256 del original: `ebdcdb8bb089a49dd20dcb9d2f0203edd5dccfa94918445daacb437e0bde15aa`

```
1  import { useEffect, useState } from 'react'
2  import { obtenerPersonajesPublicos } from '../../services/api.js'
3  import CharacterCard from '../charactercard/CharacterCard.jsx'
4  import { Link } from 'react-router'
5
6
7  export default function PublicCharacters() {
8    const [personajes, setPersonajes] = useState([])
9    const [cargando, setCargando] = useState(true)
10   const [error, setError] = useState('')
11
12   useEffect(() => {
13     let activo = true
14
15     obtenerPersonajesPublicos()
16       .then(datos => {
17         if (activo) {
18           setPersonajes(datos)
19         }
20       })
```

```
      .catch(() => {
        if (activo) {
          setError('No se pudieron cargar los personajes públicos.')
        }
      })
      .finally(() => {
        if (activo) {
          setCargando(false)
        }
      })

    return () => {
      activo = false
    }
  }, [])

  return (
    <section aria-labelledby="public-characters-title">
      <div className="section-heading">
        <h2 id="public-characters-title">Personajes públicos</h2>
      </div>

      {cargando && <p role="status">Cargando personajes...</p>}

      {error && <p role="alert">{error}</p>}

      {!cargando && !error && personajes.length === 0 && (
        <p>No hay personajes públicos disponibles.</p>
      )}

      {!cargando && !error && (
        <div className="character-grid">
          {personajes.map(personaje => {
            const personajeAdaptado = {
              id: `public-${personaje.idPersonaje}`,
              name: personaje.nombre,
              type: 'Personaje público',
              level: null,
              image: '',
              portrait: null,
              favorite: false,
            }

            return (
              <Link
  key={personaje.idPersonaje}
  to={`/personajes-publicos/${personaje.idPersonaje}`}
  className="public-character-link"
  aria-label={`Consultar ficha pública de ${personaje.nombre}`}
>
  <CharacterCard
    character={personajeAdaptado}
    interactive={false}
  />
</Link>
            )
          })}
        </div>
      )}
```

```
80      </section>
81    )
82  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 61 DE 62

generateCharacter.test.js

`tests/generateCharacter.test.js`

Pruebas del generador completo y sus campos fijados.

- Comprueba validez de borradores, cambios al regenerar, conservación de campos y ausencia de mutaciones.

Código completo

62 líneas · SHA-256 del original: `a7b9a1a2c58318e8b38c24443e3535072c7ada5ccbced53c31fcd6260d517f24`

```
1  import assert from 'node:assert/strict'
2  import test from 'node:test'
3  import { inspirationFields } from '../src/data/inspiration.js'
4  import { normalizeCharacter } from '../src/utils/characterStore.js'
5  import { randomCharacter } from '../src/utils/generateCharacter.js'
6
7  function seededRandom() {
8    let seed = 7381
9    return () => {
10     seed = (Math.imul(seed, 1664525) + 1013904223) >>> 0
11     return seed / 4294967296
12   }
13 }
14
15 test('genera fichas completas válidas y varía todos los campos', () => {
16   const random = seededRandom()
17   const seen = Object.fromEntries(inspirationFields.map(({ key }) => [key, new Set()]))
18   let previous
19   for (let index = 0; index < 300; index++) {
20     const draft = randomCharacter(previous, [], random)
21     assert.doesNotThrow(() => normalizeCharacter({ ...draft, id: `generated-${index}` }))
22     assert.ok(draft.story.length > 100)
23     assert.equal(new Set(draft.traits).size, draft.traits.length)
24     assert.equal(draft.id, undefined)
25     assert.equal(draft.imageDataUrl, '')
26     for (const { key } of inspirationFields) seen[key].add(JSON.stringify(draft[key]))
27     previous = draft
28   }
29   for (const { key } of inspirationFields) assert.ok(seen[key].size > 1, `El campo ${key} debe variar`)
30 })
31
32 test('cada nueva combinación cambia todos los campos libres incluso con una secuencia aleatoria
   constante', () => {
33   for (const value of [0, 0.999999]) {
34     const random = () => value
35     const first = randomCharacter(undefined, [], random)
36     const next = randomCharacter(first, [], random)
37     assert.doesNotThrow(() => normalizeCharacter({ ...next, id: 'next' }))
38     for (const { key } of inspirationFields) assert.notDeepEqual(next[key], first[key], key)
```

```
    next.stats.forEach((stat, index) => assert.notEqual(stat, first.stats[index]))
  }
})

test('fijar campos conserva sus valores y sus imágenes sin mutar el borrador anterior', () => {
  const first = randomCharacter(undefined, [], () => 0)
  first.imageDataUrl = ''
  first.imageName = 'retrato.png'
  const original = structuredClone(first)
  const allFields = inspirationFields.map(({ key }) => key)
  const next = randomCharacter(first, allFields, () => 0.9)
  assert.deepEqual(next, original)
  assert.notEqual(next.stats, first.stats)
  assert.notEqual(next.traits, first.traits)
  assert.deepEqual(first, original)
})

test('regenerar solo los atributos conserva todos los demás campos', () => {
  const first = randomCharacter(undefined, [], () => 0)
  const locked = inspirationFields.filter(({ key }) => key !== 'stats').map(({ key }) => key)
  const next = randomCharacter(first, locked, () => 0.7)
  for (const key of locked) assert.deepEqual(next[key], first[key], key)
  next.stats.forEach((stat, index) => assert.notEqual(stat, first.stats[index]))
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 62 DE 62

.env.development

`.env.development`

Variable de entorno declarada para la dirección de la API.

- VITE_API_URL=http://localhost:8080/api.
- El servicio actual utiliza /api relativo sin leer import.meta.env; la dirección efectiva de desarrollo la fija el proxy de Vite.

Código completo

1 líneas · SHA-256 del original: `b1aea50188a63956872d37f0ecca36d84001edc3e344a81ef3b69ac45d2001d4`

```
1  VITE_API_URL=http://localhost:8080/api
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVOS PARA EXTRAER

Recursos y copia editable

El ZIP conserva el PDF, esta versión HTML y una carpeta codigo/ lista para abrir en el editor. Los archivos se leen del proyecto, sin alterar sus contenidos.

- Los PNG y TTF se incluyen en su formato original, junto con sus licencias.
- package-lock.json conserva la resolución de dependencias.
- El manifiesto enumera rutas y hashes para comprobar qué versión del código se documentó.
- Se excluyen dependencias instaladas, compilados, historial de Git y datos de los navegadores.

Para estudiar una función, usa primero el índice de archivos, consulta su explicación y localiza su número de línea. Para modificar o ejecutar el código, utiliza la copia editable del ZIP o el proyecto original.